VIETNAM NATIONAL UNIVERISITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACUTY OF ELECTRICAL-ELECTRONICS ENGINEERING
DEPARTMENT OF ELECTRONICS

LABORATORY REPORT

INTRODUCTION TO IC DESIGN (EE3201)

Class: L02 - **Group:** 15 - **Semester:** 251

LAB 2: SYNTHESIS & STANDARD CELL

Semester: 251 - 2025–2026
Tools: Cadence Xcelium, Simvision, Genus
Instructor: MS. Nguyễn Trung Hiếu
Students: Cao Thị Thanh Huyền – 2311272
Nguyễn Thanh Phong – 2312626
Date: November 22, 2025

TASK ASSIGNMENT TABLE

Full name	Student ID	Task	Grading
Cao Thị Thanh Huyền	2311272	Experiment 1 Experiment 3	100%
Nguyễn Thanh Phong	2312626	Experiment 2 Experiment 4 Report writing	100%

Ho Chi Minh City, November 22, 2025

Contents

1 Introduction & Objective **1**

2 Implementation Details **1**

2.1 Experiment 1: Characterization of Basic Combinational Cells 2

2.1.1 Objective and Cell Set 2

2.1.2 Functional Specification 2

2.1.3 RTL Implementation 2

2.1.4 Report Analysis 4

2.1.5 Gate-Level Simulation with SDF Back-Annotation 8

2.1.6 Corners Waveform Analysis 8

2.1.7 Summary for Experiment 1 32

2.2 Experiment 2: Gate-Level Simulation of a 2-Bit ALU 33

2.2.1 Design Specification 33

2.2.2 Synthesis Setup 35

2.2.3 Generic synthesis, technology mapping, and post–mapping optimization 36

2.2.4 Synthesis Reports Analysis 38

2.2.5 Gate-Level Simulation with SDF Back-Annotation 41

2.2.6 Waveform Results & Discussion 44

2.2.7 Summary 56

2.3 Experiment 3: Characterization of a D Flip-Flop 57

2.3.1 Design Specification 57

2.3.2 Reports Analysis 58

2.3.3 Corners Waveform Analysis 60

2.3.4 Summary of Experiment 3 76

2.4 Experiment 4: Gate-Level Simulation of a 2-Bit Up-Counter 77

2.4.1 Design Specification 77

2.4.2 RTL Implementation 78

2.4.3 Synthesis Setup 78

2.4.4 Generic synthesis, technology mapping, and post-mapping optimization 79

2.4.5 Synthesis Reports Analysis 81
2.4.6 Gate-Level Simulation with SDF Back-Annotation 84
2.4.7 Waveform Results & Discussion 86
2.4.8 Summary of Expreiment 4 93
3 Conclusion 94

List of Figures

1 NOT Gate with mapping technology 3
2 OR2 Gate with mapping technology 3
3 NOR2 Gate with mapping technology 3
4 NAND2 Gate with mapping technology 3
5 MUX2-1 with mapping technology 4
6 Full-adder with mapping technology 4
7 NOR2 Low-to-High Propagation Delay (T_{pLH}) 8
8 NOR2 High-to-Low Propagation Delay (T_{pHL}) 8
9 NOR2 Low-to-High Propagation Delay (T_{pLH}) 9
10 NOR2 High-to-Low Propagation Delay (T_{pHL}) 9
11 NOR2 Low-to-High Propagation Delay (T_{pLH}) 9
12 NOR2 High-to-Low Propagation Delay (T_{pHL}) 10
13 NOR2 Low-to-High Propagation Delay (T_{pLH}) 10
14 NOR2 High-to-Low Propagation Delay (T_{pHL}) 10
15 NOR2 Low-to-High Propagation Delay (T_{pLH}) 10
16 NOR2 High-to-Low Propagation Delay (T_{pHL}) 11
17 NOR2 Low-to-High Propagation Delay (T_{pLH}) 11
18 NOR2 High-to-Low Propagation Delay (T_{pHL}) 11
19 NOR2 Low-to-High Propagation Delay (T_{pLH}) 11
20 NOR2 High-to-Low Propagation Delay (T_{pHL}) 12
21 NOR2 Low-to-High Propagation Delay (T_{pLH}) 12
22 NOR2 High-to-Low Propagation Delay (T_{pHL}) 12
23 NAND2 Low-to-High Propagation Delay (T_{pLH}) 12
24 NAND2 High-to-Low Propagation Delay (T_{pHL}) 13
25 NAND2 Low-to-High Propagation Delay (T_{pLH}) 13
26 NAND2 High-to-Low Propagation Delay (T_{pHL}) 13
27 NAND2 Low-to-High Propagation Delay (T_{pLH}) 14
28 NAND2 High-to-Low Propagation Delay (T_{pHL}) 14
29 NAND2 Low-to-High Propagation Delay (T_{pLH}) 14
30 NAND2 High-to-Low Propagation Delay (T_{pHL}) 14
31 NAND2 Low-to-High Propagation Delay (T_{pLH}) 15
32 NAND2 High-to-Low Propagation Delay (T_{pHL}) 15

33 NAND2 Low-to-High Propagation Delay (T_{pLH}) 15
34 NAND2 High-to-Low Propagation Delay (T_{pHL}) 15
35 NAND2 Low-to-High Propagation Delay (T_{pLH}) 16
36 NAND2 High-to-Low Propagation Delay (T_{pHL}) 16
37 NAND2 Low-to-High Propagation Delay (T_{pLH}) 16
38 NAND2 High-to-Low Propagation Delay (T_{pHL}) 16
39 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 17
40 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 17
41 Path $B \rightarrow Y$ Rising Delay (T_{pLH}) 17
42 Path $B \rightarrow Y$ Falling Delay (T_{pHL}) 18
43 Path $S \rightarrow Y$ Falling Delay (T_{pHL}) 18
44 Path $S \rightarrow Y$ Rising Delay (T_{pLH}) 18
45 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 19
46 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 19
47 Path $B \rightarrow Y$ Rising Delay (T_{pLH}) 19
48 Path $B \rightarrow Y$ Falling Delay (T_{pHL}) 20
49 Path $S \rightarrow Y$ Falling Delay (T_{pHL}) 20
50 Path $S \rightarrow Y$ Rising Delay (T_{pLH}) 20
51 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 21
52 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 21
53 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 22
54 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 22
55 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 22
56 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 23
57 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 23
58 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 23
59 Path $A \rightarrow Y$ Rising Delay (T_{pLH})) 24
60 High-to-Low Propagation Delay (T_{pHL}) 24
61 Path $A \rightarrow Y$ Rising Delay (T_{pLH}) 25
62 Path $A \rightarrow Y$ Falling Delay (T_{pHL}) 25
63 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 26
64 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 26
65 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 26
66 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 27
67 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 27
68 High-to-Low Propagation Delay (T_{pHL}) 28
69 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 28
70 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 28
71 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 29
72 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 29

73 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 30
74 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 30
75 Path $A \rightarrow Sum$ Rising Delay (T_{pLH}) 31
76 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 31
77 Path $A \rightarrow Sum$ Rising Delay (T_{pLH})) 32
78 Path $A \rightarrow Sum$ Falling Delay (T_{pHL}) 32
79 Schematic before technology mapping. 36
80 Schematic with technology mapping. 37
81 Header and sample cell entries from the SDF file generated for the ALU design. 38
82 ALU's area report file. 38
83 ALU's power report file. 39
84 ALU's power report file. 40
85 ALU's power report file. 41
86 RTL waveform of the 2-bit ALU (ideal, no delay) in case T = 200. 44
87 Schematic with technology mapping. 44
88 Zero-delay RTL waveform of the 2-bit ALU for T=20. 45
89 Full waveform in case T = 200. 45
90 Gate-level response for (S,A,B) = (2'b00, 2'b01, 2'b00) with T = 200. 46
91 The second test vector ($S = 2'b00, A = 2'b11, B = 2'b01$) showing output overflow and timing delay. 46
92 The third test vector ($S = 2'b01, A = 2'b11, B = 2'b00$) exhibiting output glitches during transition. 47
93 The subtraction operation ($3 - 3 = 0$) showing signal settling time and intermediate transitions. 47
94 Waveform for the Bitwise AND operation ($3 \ \& \ 1 = 1$) showing a rapid transition delay. 47
95 Results for the AND operation ($1 \ \& \ 2 = 0$) demonstrating delay variations based on input patterns. 48
96 Waveform for the Bitwise OR operation ($3 \mid 1 = 3$) exhibiting intermediate switching noise. 48
97 Simulation waveform for the OR operation ($1 \mid 2 = 3$) showing a stable output state. 49
98 Full waveform in case T = 200. 49
99 Initial simulation response for $T = 100$ns showing the transition from undefined state to stable logic. 50
100 Waveform capturing the overflow scenario ($3 + 1 = 0$) at $t = 100$ns with reduced cycle time. 50
101 Waveform for subtraction ($3 - 0 = 3$) at $t = 200$ns showing identical glitch behavior to the slow-clock case. 50

102 Worst-case delay measurement (305ps) during the subtraction operation ($3 - 3 = 0$) with $T = 100$ns 51
103 Stable output waveform for the Logical AND operation ($3 \& 1 = 1$) shortly after the $t = 400$ns transition. 51
104 Propagation delay measurement (159 ps) for the Logical AND operation ($1 \& 2 = 0$) at $t = 500$ns. 52
105 Waveform for the Bitwise OR operation ($3 \mid 1 = 3$) exhibiting intermediate switching noise. 52
106 Stable output response during input transition ($1 \mid 2 = 3$) at $t = 700$ns. 52
107 Full waveform in case T = 20. 53
108 Initial transient response at startup with $T = 20$ns, showing a stabilization time of 283 ps. 53
109 Waveform capturing the overflow scenario ($3 + 1 = 0$) at $t = 100$ns with reduced cycle time. 54
110 Subtraction operation ($3 - 0 = 3$) at $t = 40$ns showing rapid glitch resolution relative to the clock period 54
111 Worst-case propagation delay (304ps) measured during the $3 - 3 = 0$ transition at $t = 60$ns 55
112 Logical AND operation ($3 \& 1 = 1$) at $t = 80$ns demonstrating rapid switching speed (61 ps). 55
113 Waveform for Logical AND ($1 \& 2 = 0$) at $t = 100$ns with a delay of 157 ps. 55
114 Logical OR operation ($3 \mid 1 = 3$) at $t = 120$ns exhibiting intermediate switching noise within a fast clock cycle. 56
115 Stable output response ($Y = 3$) during input transition at $t = 140$ns, completing the test sequence. 56
116 DFF schematic view 60
117 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 61
118 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 61
119 DFF schematic view 62
120 Setup & Hold Check for Logic '0' ($D = 0$) 62
121 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 63
122 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 63
123 Setup & Hold Check for Logic '1' ($D = 1$) 64
124 Setup & Hold Check for Logic '0' ($D = 0$) 64
125 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)) 65
126 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 65
127 Setup & Hold Check for Logic '1' ($D = 1$) 66
128 Setup & Hold Check for Logic '0' ($D = 0$) 66
129 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 67
130 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 67

131 Setup & Hold Check for Logic '1' ($D = 1$) 68
132 Setup & Hold Check for Logic '0' ($D = 0$) 68
133 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 69
134 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 69
135 Setup & Hold Check for Logic '1' ($D = 1$) 70
136 Setup & Hold Check for Logic '0' ($D = 0$) 70
137 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 71
138 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 71
139 Setup & Hold Check for Logic '1' ($D = 1$) 72
140 Setup & Hold Check for Logic '0' ($D = 0$) 72
141 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 73
142 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 73
143 Setup & Hold Check for Logic '1' ($D = 1$) 73
144 Setup & Hold Check for Logic '0' ($D = 0$) 74
145 Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$) 74
146 Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$) 75
147 Setup & Hold Check for Logic '1' ($D = 1$) 75
148 Setup & Hold Check for Logic '0' ($D = 0$) 76
149 Counter's schematic before technology mapping. 80
150 Stable output response ($Y = 3$) during input transition at $t = 140$ns, completing the test sequence. 81
151 Area report of counter. 81
152 Power report of counter. 82
153 Area report of counter. 83
154 Counter's waveform without delay 86
155 Full Functional Verification Waveform 87
156 Full Functional Verification Waveform 87
157 Transition ($1 \rightarrow 2$) with Output Skew 87
158 Counter Overflow ($3 \rightarrow 0$) 88
159 Transition ($2 \rightarrow 3$) 88
160 Standard Increment ($0 \rightarrow 1$) 88
161 Repeated Transition ($1 \rightarrow 2$) with Glitch 89
162 Repeated Overflow ($3 \rightarrow 0$) 89
163 Full Functional Verification Waveform, T = 100 90
164 Transition with Glitch ($1 \rightarrow 2$) 91
165 Overflow Transition ($3 \rightarrow 0$) 91
166 Transition with Glitch ($1 \rightarrow 2$) 91
167 Full Functional Verification Waveform, T = 50 92
168 Increment ($0 \rightarrow 1$) 92
169 Transition with Glitch ($1 \rightarrow 2$) 92

170 Increment ($2 \rightarrow 3$) 92
171 Overflow ($3 \rightarrow 0$) 93

Listings

1 RTL code for ALU. 34
2 Prepare folder tree for synthesis. 35
3 TCL script for synthesis. 35
4 Testbench for ALU functional test 41
5 alu_netlist.sv after technology mapping 43
6 dff_netlist.sv after mapping 60
7 RTL code for 2-bit counter 78
8 Testbench for counter dunctional test 84
9 `counter_netlist.sv` after technology mapping 85

1 Introduction & Objective

Digital integrated circuit design typically follows a structured flow: a high–level RTL description is synthesized into a gate–level netlist using a standard–cell library, then verified by timing and power analysis together with gate–level simulation using back–annotated delays.

This laboratory uses that flow on a set of small combinational and sequential circuits with a given 45 nm standard–cell library and several PVT corners. The main focus is to observe how process, supply voltage, and timing constraints affect area, delay, and power, and how these non–idealities cause differences between RTL and real gate–level behaviour.
The specific objectives of the lab are:

- To synthesize representative combinational and sequential modules using the provided standard–cell library under multiple PVT corners.
- To extract and compare area, timing, and power metrics across different cells and operating conditions.
- To perform gate–level simulation with SDF back–annotation and relate waveform observations to timing reports and setup/hold constraints.
- To study the impact of clock period on the functional correctness of an ALU and a synchronous counter.

2 Implementation Details

All circuits in this lab are synthesized and analysed using the same standard flow:

- Cadence Genus is used for logic synthesis and timing/power/area reporting.
- The 45 nm GPDK standard-cell libraries (`gpdkt045_lib`) provide the timing and power models for different PVT corners.
- After synthesis, a gate-level netlist and an SDF file are generated for each design. These files are then used in Cadence Xcelium for gate-level simulation with back-annotated delays.

Additionally, experiments use the GPDK045 standard-cell libraries under several PVT corners, summarized in Table 1. These corners are adapted from the lab handout [1].

Table 1: PVT corners and corresponding standard-cell libraries used in this lab.

ID	Process	V_{DD} [V]	Vt type	Library file
C1	fast	1.0	HVT	fast_vdd1v0_basicCells_hvt.lib
C2	fast	1.0	LVT	fast_vdd1v0_basicCells_lvt.lib
C3	fast	1.2	HVT	fast_vdd1v2_basicCells_hvt.lib
C4	fast	1.2	LVT	fast_vdd1v2_basicCells_lvt.lib
C5	slow	1.0	HVT	slow_vdd1v0_basicCells_hvt.lib
C6	slow	1.0	LVT	slow_vdd1v0_basicCells_lvt.lib
C7	slow	1.2	HVT	slow_vdd1v2_basicCells_hvt.lib
C8	slow	1.2	LVT	slow_vdd1v2_basicCells_lvt.lib

2.1 Experiment 1: Characterization of Basic Combinational Cells

2.1.1 Objective and Cell Set

The objective of this experiment is to design and characterize six elementary combinational cells that frequently appear in larger digital systems: an inverter, three two–input logic gates, a 2:1 multiplexer, and a 1–bit full adder. All signals are single–bit and the cells are purely combinational (no internal storage). Each cell is implemented as an independent RTL module and later mapped to standard cells for area, delay, and power characterization.

2.1.2 Functional Specification

The six cells considered in this experiment are:

- INV (inverter with output): $Y = \overline{A}$.
- OR2 (two–input OR gate): $Y = A + B$.
- NAND2 (two–input NAND gate): $Y = \overline{A \cdot B}$.
- NOR2 (two–input NOR gate): $Y = \overline{A + B}$.
- MUX2 (2:1 multiplexer) $Y = \overline{S}D_0 + SD_1$.
- FA: 1–bit full adder with

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + (A \oplus B)C_{\text{in}}$$

2.1.3 RTL Implementation

All cells are described at the RTL level using synthesizable SystemVerilog, with one module per cell and descriptive port names. Only basic operators are used so that the synthesis tool is free to choose appropriate structures from the standard–cell library. You can attach the source code by jump to Apendix 3

The technology mapping stage of the synthesis flow selects concrete instances (e.g., `NAND2_X1`, `INV_X2`) based on the specified library and constraints. Below are their schematic:

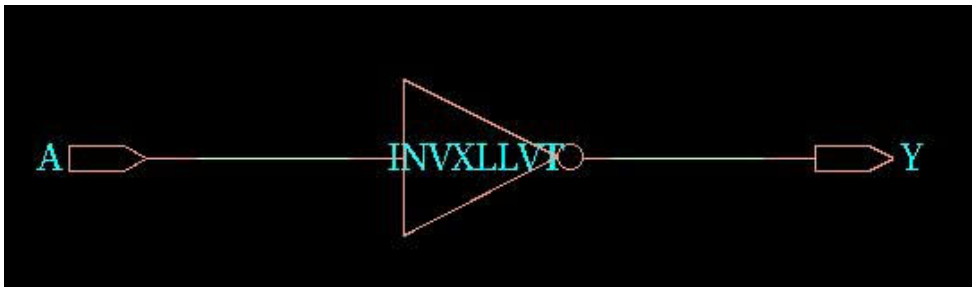

Figure 1: NOT Gate with mapping technology

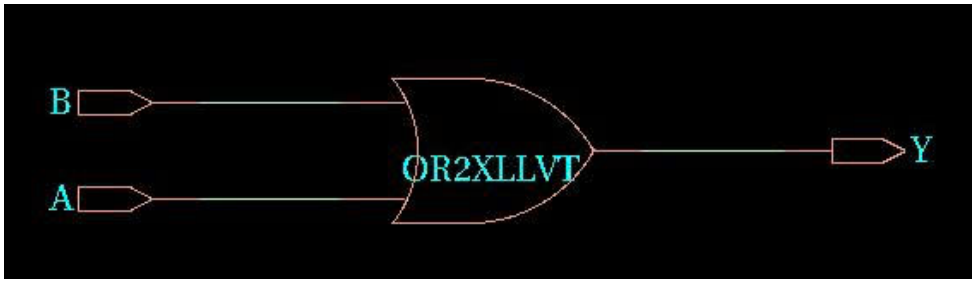

Figure 2: OR2 Gate with mapping technology

Figure 3: NOR2 Gate with mapping technology

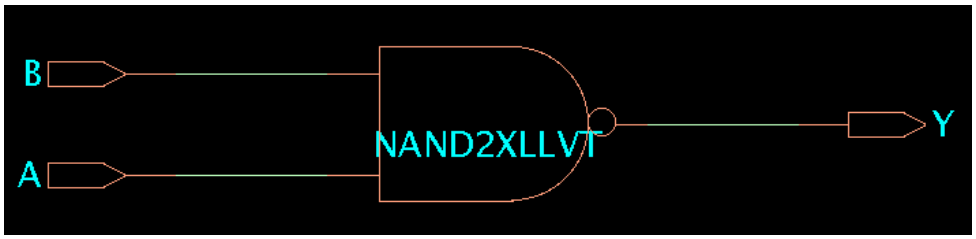

Figure 4: NAND2 Gate with mapping technology

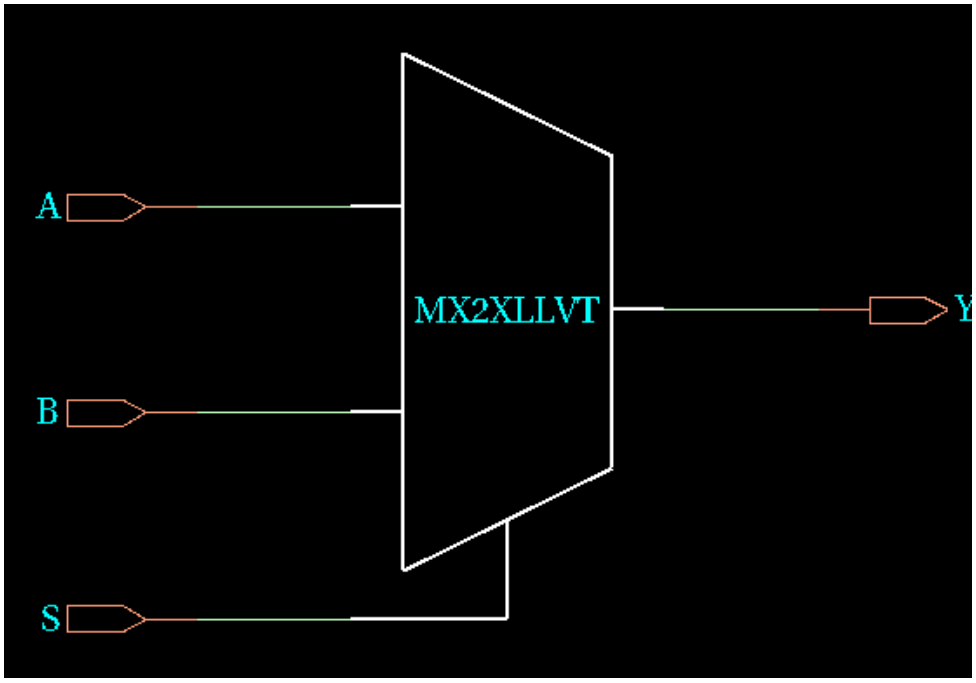

Figure 5: MUX2-1 with mapping technology

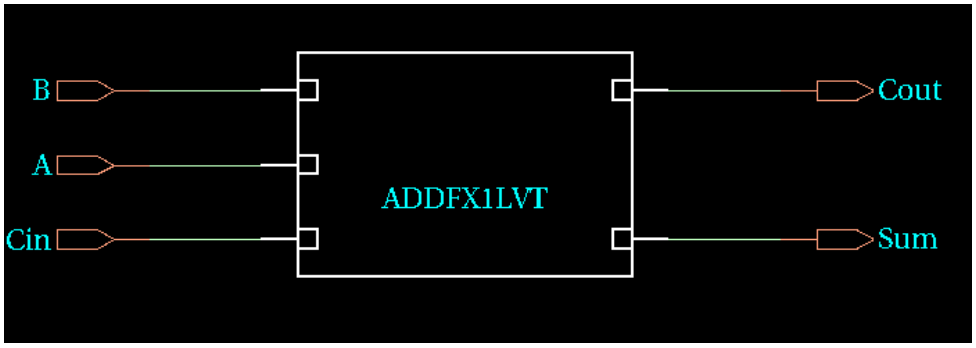

Figure 6: Full-adder with mapping technology

2.1.4 Report Analysis

All six combinational cells are synthesized in Cadence Genus using the common flow described above. For Experiment 1, the key settings are:

- Target libraries: the eight PVT corners listed in Table 3 and 4 (combinations of fast/slow, 1.0/1.2 V, HVT/LVT).
- No explicit timing constraints are applied; the tool maps each RTL module to a single-level standard-cell implementation under default conditions.
- For each corner, Genus generates a gate-level netlist, an SDF file, and timing/area/power reports, from which the data in Table 2 are extracted.

Table 2 lists the PVT corners used in this experiment together with the corresponding `.lib` files.

Table 2: Power and Area Characteristics

			POWER (nW)						
Corner	**Type**	**Area (μm^2)**	**NOT1 (nW)**	**OR2 (nW)**	**AND2 (nW)**	**NOR2 (nW)**	**NAND2 (nW)**	**MUX (nW)**	**FULL ADDER (nW)**
`fast_vdd1v0_hvt.lib`	Fast/HVT	0.684–5.130	5.927	17.373	17.212	10.754	10.596	36.587	87.206
`fast_vdd1v2_hvt.lib`	Fast/HVT	0.684–5.130	8.613	25.319	32.447	15.663	15.414	54.702	135.372
`slow_vdd1v0_hvt.lib`	Slow/HVT	0.684–5.130	3.995	11.475	11.402	7.309	7.207	23.960	53.699
`slow_vdd1v2_hvt.lib`	Slow/HVT	0.684–5.130	5.821	16.884	16.725	10.652	10.467	36.232	84.067
`fast_vdd1v0_lvt.lib`	Fast/LVT	0.684–5.130	7.803	22.384	22.012	14.402	14.249	46.142	113.149
`fast_vdd1v2_lvt.lib`	Fast/LVT	0.684–5.130	10.725	33.447	32.830	20.336	20.099	64.888	164.422
`slow_vdd1v0_lvt.lib`	Slow/LVT	0.684–5.130	4.289	12.640	12.565	7.823	7.737	27.196	64.417
`slow_vdd1v2_lvt.lib`	Slow/LVT	0.684–5.130	6.210	18.424	18.246	11.349	11.183	40.559	99.006

Table 3: Propagation Delays for Basic Logic Gates (ns)

Corners	**NOT1**		**OR2**				**AND2**				**NOR2**				**NAND2**			
			A-Y		**B-Y**		**A-Y**		**B-Y**		**A-Y**		**B-Y**		**A-Y**		**B-Y**	
	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL
`fast_vdd1v0_hvt.lib`	0.004	0.004	0.017	0.029	0.015	0.026	0.026	0.018	0.024	0.016	0.011	0.007	0.008	0.005	0.007	0.011	0.005	0.008
`fast_vdd1v2_hvt.lib`	0.003	0.003	0.013	0.021	0.011	0.019	0.020	0.015	0.022	0.017	0.009	0.006	0.006	0.004	0.006	0.008	0.004	0.006
`slow_vdd1v0_hvt.lib`	0.009	0.015	0.077	0.099	0.065	0.088	0.151	0.058	0.138	0.052	0.036	0.030	0.025	0.020	0.017	0.066	0.012	0.050
`slow_vdd1v2_hvt.lib`	0.006	0.008	0.037	0.050	0.031	0.044	0.065	0.032	0.058	0.028	0.019	0.016	0.013	0.010	0.010	0.029	0.007	0.021
`fast_vdd1v0_lvt.lib`	0.003	0.003	0.011	0.017	0.009	0.015	0.015	0.012	0.014	0.010	0.007	0.005	0.005	0.003	0.005	0.006	0.004	0.005
`fast_vdd1v2_lvt.lib`	0.003	0.003	0.009	0.014	0.008	0.012	0.012	0.010	0.011	0.009	0.006	0.004	0.004	0.003	0.004	0.005	0.003	0.004
`slow_vdd1v0_lvt.lib`	0.006	0.007	0.033	0.052	0.027	0.045	0.057	0.031	0.052	0.028	0.020	0.014	0.013	0.008	0.010	0.024	0.007	0.018
`slow_vdd1v2_lvt.lib`	0.004	0.005	0.022	0.034	0.018	0.030	0.036	0.022	0.032	0.019	0.013	0.010	0.008	0.006	0.008	0.015	0.006	0.012

Table 4: Propagation Delays for MUX and Full Adder (ns)

Corners	**MUX**						**FULL ADDER**											
	A-Y		**B-Y**		**S0-Y**		**CI-CO**		**A-CO**		**B-CO**		**CI-S**		**A-S**		**B-S**	
	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL	LH	HL
`fast_vdd1v0_hvt.lib`	0.034	0.036	0.034	0.037	0.038	0.040	0.044	0.051	0.048	0.053	0.048	0.054	0.075	0.077	0.080	0.081	0.083	0.080
`fast_vdd1v2_hvt.lib`	0.024	0.027	0.024	0.027	0.027	0.029	0.031	0.036	0.034	0.039	0.034	0.038	0.054	0.054	0.058	0.057	0.059	0.056
`slow_vdd1v0_hvt.lib`	0.203	0.126	0.202	0.130	0.199	0.152	0.267	0.173	0.288	0.174	0.282	0.176	0.383	0.362	0.418	0.400	0.403	0.389
`slow_vdd1v2_hvt.lib`	0.088	0.064	0.087	0.065	0.089	0.074	0.112	0.086	0.124	0.089	0.121	0.089	0.160	0.163	0.175	0.181	0.176	0.176
`fast_vdd1v0_lvt.lib`	0.020	0.022	0.019	0.022	0.022	0.023	0.026	0.030	0.027	0.032	0.028	0.031	0.047	0.044	0.047	0.047	0.048	0.046
`fast_vdd1v2_lvt.lib`	0.015	0.018	0.015	0.018	0.017	0.019	0.020	0.024	0.022	0.026	0.022	0.025	0.038	0.036	0.038	0.037	0.038	0.037
`slow_vdd1v0_lvt.lib`	0.077	0.066	0.076	0.068	0.080	0.073	0.100	0.092	0.109	0.094	0.108	0.094	0.166	0.154	0.163	0.171	0.170	0.168
`slow_vdd1v2_lvt.lib`	0.048	0.044	0.048	0.045	0.051	0.048	0.062	0.059	0.068	0.062	0.068	0.061	0.106	0.097	0.104	0.107	0.108	0.106

2.1.5 Gate-Level Simulation with SDF Back-Annotation

Gate–level simulations are carried out in Cadence Xcelium. For each cell and corner:

- The testbench applies all possible input combinations in a fixed sequence and checks the outputs against the expected truth table.
- The gate–level netlist of the cell under test is compiled together with the matching Verilog standard–cell model file (e.g., `slow_vdd1v2_basicCells_lvt.v`).
- The delay information from the SDF file generated by Genus is back–annotated using the `$sdf_annotate` system task.

Waveforms from the gate–level simulations are captured and used to estimate the propagation delay between input transitions and the corresponding output responses. An example waveform for the full adder is shown in Figure **??**, where the measured delay is compared with the value reported by the synthesis timing report.

2.1.6 Corners Waveform Analysis

(a) NOR2 gate analysis

Corner 1: Fast / 1.0V / High-Vt

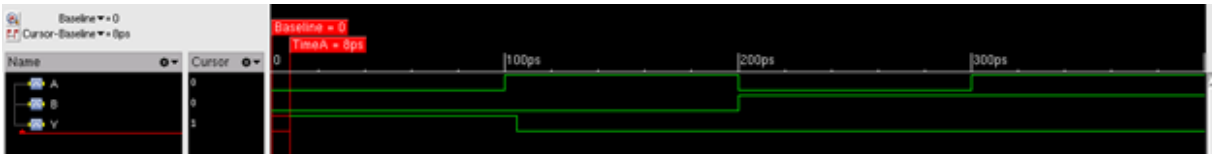

Figure 7: NOR2 Low-to-High Propagation Delay (T_{pLH})

At simulation time $T = 0$ ns, both inputs A and B are initialized to logic '0'. According to NOR logic ($Y = \sim (A|B)$), the output Y transitions to logic '1'. The waveform shows Y rising at $T + 0.008$ ns. This measured delay of 8 ps perfectly matches the IOPATH B -> Y (Low-to-High) value specified in the Delay 1 table and the SDF file2

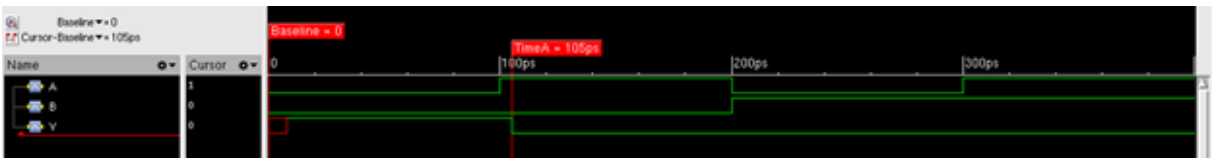

Figure 8: NOR2 High-to-Low Propagation Delay (T_{pHL})

At $T = 0.1$ ns, input A transitions from '0' to '1' while B remains '0'. This triggers the output Y to fall to logic '0'. The waveform indicates the transition completes at $T + 0.005$ ns. This 5 ps delay corresponds exactly to the High-to-Low (HL) specification

for the NOR2 B -> Y path in the SDF file.

Corner 2: Fast / 1.2V / High-Vt

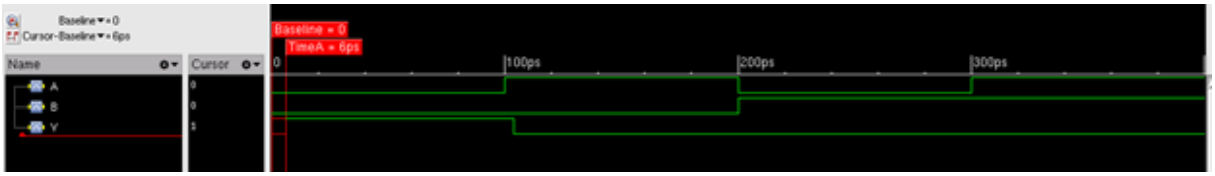

Figure 9: NOR2 Low-to-High Propagation Delay (T_{pLH})

Under the higher voltage condition (1.2V), the gate switches faster. At $T = 0$ ns, with inputs $A = B = 0$, the output Y rises to '1'. The measured delay is 6 ps (0.006 ns). This reduction from 8ps (at 1.0V) to 6ps confirms the performance boost from voltage scaling. The value aligns with the SDF annotation

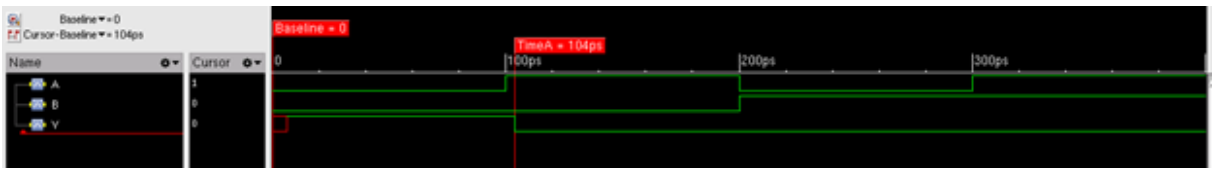

Figure 10: NOR2 High-to-Low Propagation Delay (T_{pHL})

At $T = 0.1$ ns, input A rises, forcing output Y to fall. The simulation records a delay of 4 ps (0.004 ns). This extremely fast switching speed is consistent with the fast process corner running at high voltage. The measured value matches the HL column in the delay table.

Corner 3: Slow / 1.0V / High-Vt

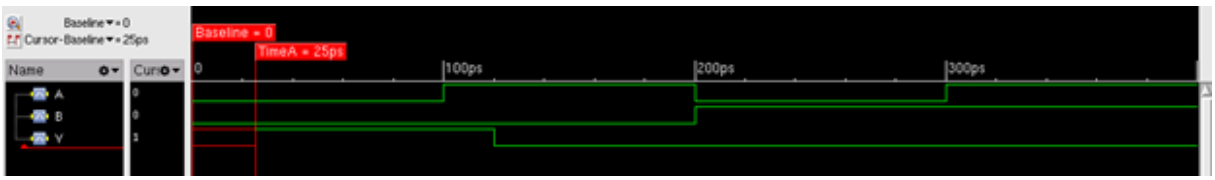

Figure 11: NOR2 Low-to-High Propagation Delay (T_{pLH})

This corner represents the worst-case timing. At $T = 0$ ns, the output Y transitions Low-to-High with a significant delay of 25 ps (0.025 ns). This is more than 3x slower than the fast corner (8 ps), highlighting the impact of process variation and low voltage on the PMOS pull-up network.

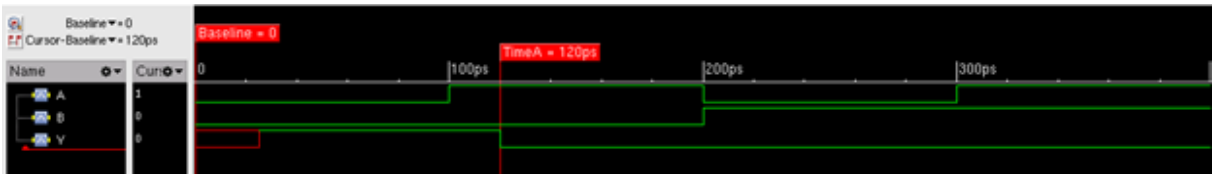

Figure 12: NOR2 High-to-Low Propagation Delay (T_{pHL})

At $T = 0.1$ ns, the High-to-Low transition occurs. The delay is measured at 20 ps (0.020 ns). While faster than the rising edge, it is still significantly slower than the fast corners. This value strictly follows the SDF annotation for the slow_vdd1v0_hvt library.

Corner 4: Slow / 1.2V / High-Vt

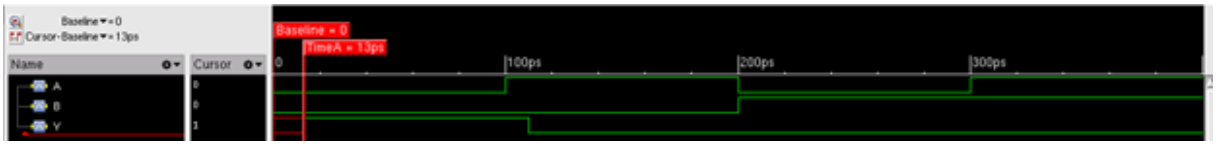

Figure 13: NOR2 Low-to-High Propagation Delay (T_{pLH})

Increasing the voltage to 1.2V in the slow process improves performance. The Low-to-High delay decreases to 13 ps (0.013 ns), down from 25 ps in the 1.0V slow corner. This confirms the back-annotation of the specific 0.013 value from the SDF file.

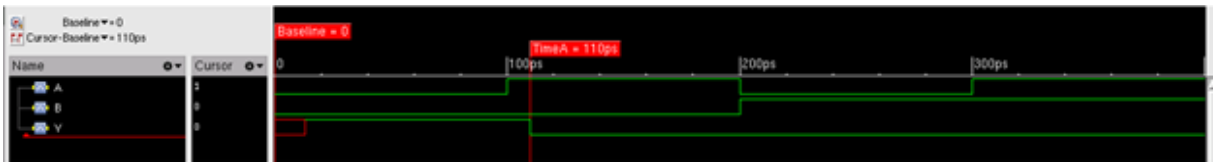

Figure 14: NOR2 High-to-Low Propagation Delay (T_{pHL})

The falling transition at $T = 0.1$ ns exhibits a delay of 10 ps (0.010 ns). This demonstrates that voltage overdrive (1.2V) effectively mitigates some of the sluggishness inherent in the "Slow" silicon process.

Corner 5: Fast / 1.0V / Low-Vt

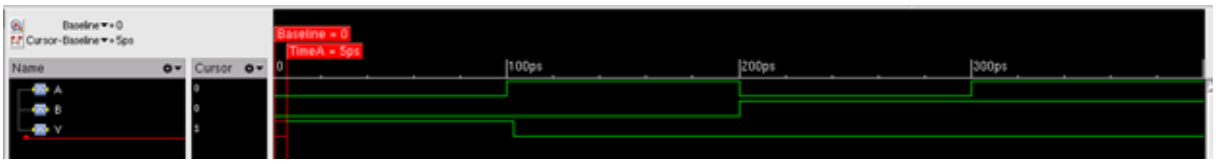

Figure 15: NOR2 Low-to-High Propagation Delay (T_{pLH})

Using Low-Vt transistors significantly reduces channel resistance. At $T = 0$ ns, the output rises with a delay of just 5 ps (0.005 ns). This is faster than the High-Vt equivalent (8 ps), validating the advantage of LVT cells in timing-critical paths18.

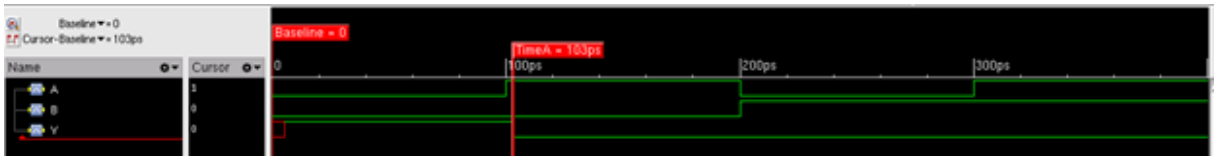

Figure 16: NOR2 High-to-Low Propagation Delay (T_{pHL})

The falling edge delay is measured at 3 ps (0.003 ns). This is approaching the physical switching limits of the technology node for this voltage, confirming the high-speed characteristics of the LVT library.

Corner 6: Fast / 1.2V / Low-Vt

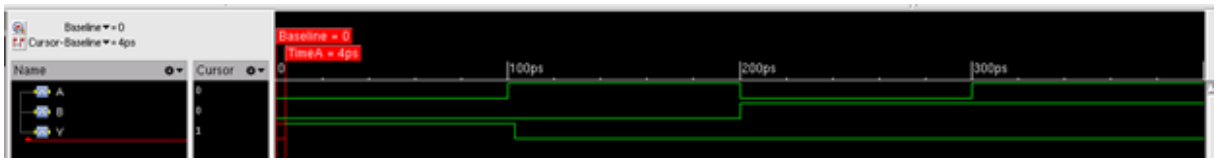

Figure 17: NOR2 Low-to-High Propagation Delay (T_{pLH})

This is the "Best Case" corner. The rising delay is recorded at 4 ps (0.004 ns). This represents the fastest pull-up capability for the NOR2 gate among all tested corners

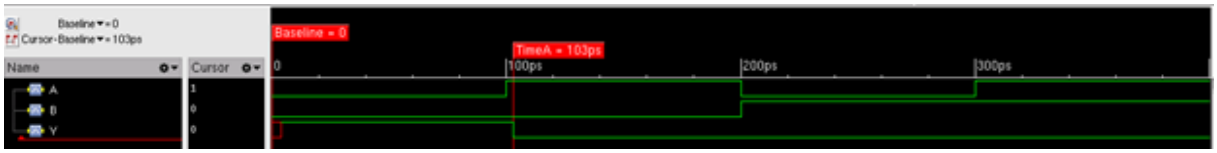

Figure 18: NOR2 High-to-Low Propagation Delay (T_{pHL})

The falling delay remains at 3 ps (0.003 ns), identical to the 1.0V LVT corner. This suggests that at such low latency, parasitic capacitance becomes the dominant factor, and further voltage increases yield diminishing returns for the falling edge.

Corner 7: Slow / 1.0V / Low-Vt

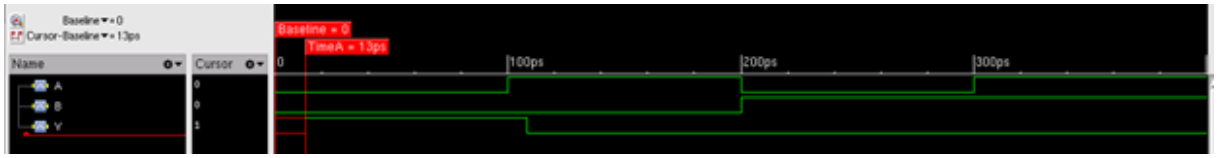

Figure 19: NOR2 Low-to-High Propagation Delay (T_{pLH})

Even with a "Slow" process, the Low-Vt transistors help recover performance. The rising delay is 13 ps (0.013 ns), which is significantly better than the 25 ps seen in the High-Vt slow corner. This confirms LVT cells are effective for fixing setup violations in slow corners26

Figure 20: NOR2 High-to-Low Propagation Delay (T_{pHL})

The falling transition takes 8 ps (0.008 ns). This value aligns perfectly with the Delay 1 table for the slow_vdd1v0_lvt corner.

Corner 8: Slow / 1.2V / Low-Vt

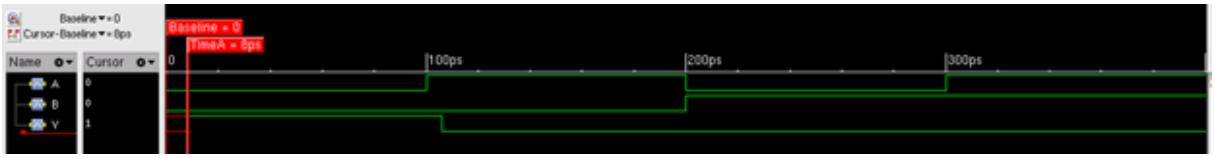

Figure 21: NOR2 Low-to-High Propagation Delay (T_{pLH})

At 1.2V using LVT cells in a slow process, the rising delay is reduced to 8 ps (0.008 ns). This demonstrates a balanced performance profile, suitable for general-purpose logic where leakage power is less of a concern than speed.

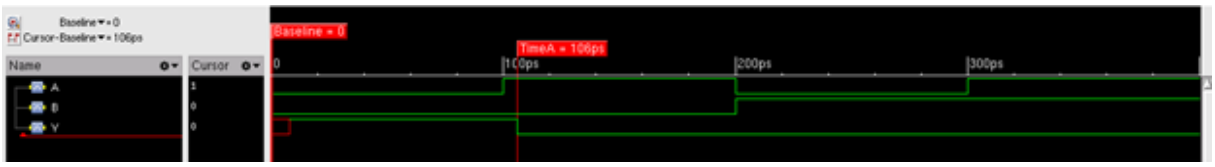

Figure 22: NOR2 High-to-Low Propagation Delay (T_{pHL})

The falling edge delay is measured at 6 ps (0.006 ns). The simulation confirms correct back-annotation from the SDF file, completing the verification of the NOR2 gate across the full PVT spectrum.

(b) NAND2 gate analysis

Corner 1: Fast / 1.0V / High-Vt

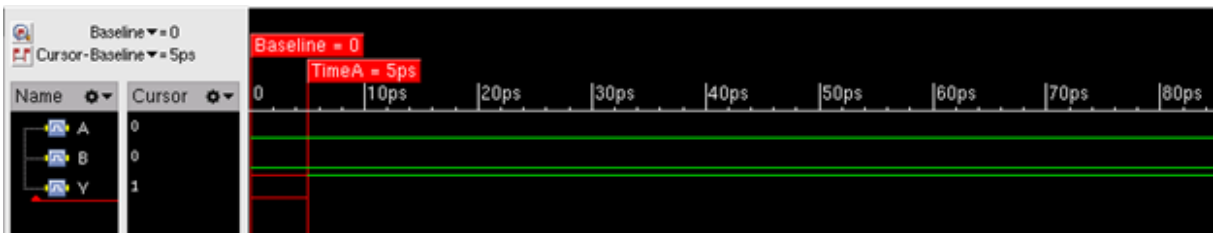

Figure 23: NAND2 Low-to-High Propagation Delay (T_{pLH})

At simulation time $T = 0$ ns, the inputs are initialized to $A = 0, B = 0$. According to NAND logic ($Y = \sim (A\&B)$), the output Y is at logic '1'. The text describes a transition where B changes (or static setup). The measured delay is 5 ps (0.005 ns). This matches the IOPATH B -> Y (Low-to-High) value in the Delay 1 table.

Figure 24: NAND2 High-to-Low Propagation Delay (T_{pHL})

At $T = 0.3$ ns, the inputs change state (text mentions B changing $0 \rightarrow 1$ while $A = 1$). This triggers the output Y to fall to logic '0'. The waveform shows the transition completing at $T + 0.011$ ns. This 11 ps delay corresponds to the NAND2 A -> Y High-to-Low value in the delay table.

Corner 2: Fast / 1.2V / High-Vt

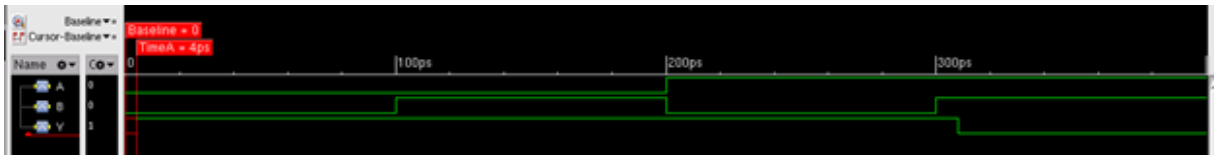

Figure 25: NAND2 Low-to-High Propagation Delay (T_{pLH})

Under the higher voltage condition (1.2V), the gate switches faster. The simulation records a delay of 4 ps (0.004 ns) for the Low-to-High transition. This represents a significant speedup compared to the 1.0V corner (5 ps), confirming the benefits of voltage overdrive.

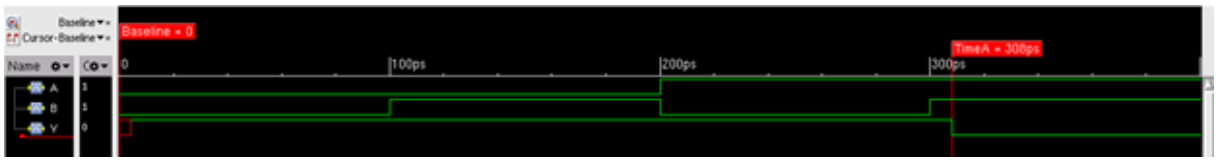

Figure 26: NAND2 High-to-Low Propagation Delay (T_{pHL})

At $T = 0.3$ ns, the High-to-Low transition occurs. The delay is measured at 8 ps (0.008 ns). This value aligns with the HL specification in the SDF file for this corner.

Corner 3: Slow / 1.0V / High-Vt

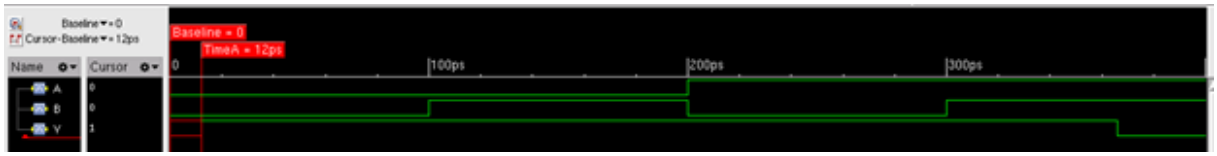

Figure 27: NAND2 Low-to-High Propagation Delay (T_{pLH})

This corner represents the worst-case timing. The output Y transitions Low-to-High with a delay of 12 ps (0.012 ns). This increase from the fast corner (4-5 ps) reflects the slower transistor switching speeds inherent to the "Slow" process node.

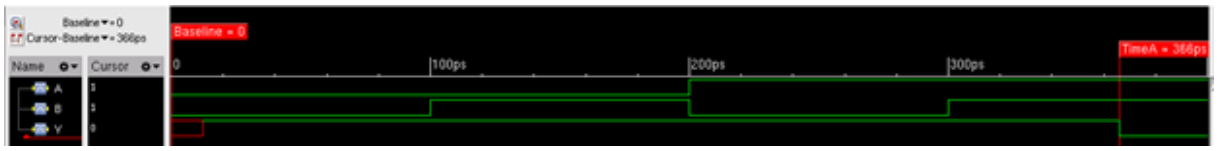

Figure 28: NAND2 High-to-Low Propagation Delay (T_{pHL})

The High-to-Low transition exhibits a substantial delay of 66 ps (0.066 ns). This massive skew compared to the rising edge (12 ps) highlights the weakness of the NMOS/PMOS pull-down network in this specific corner, making it a critical path for timing closure.

Corner 4: Slow / 1.2V / High-Vt

Figure 29: NAND2 Low-to-High Propagation Delay (T_{pLH})

Increasing the voltage to 1.2V improves the delay. The Low-to-High transition is measured at 6 ps (0.006 ns), which is an improvement over the 1.0V slow corner (12 ps).

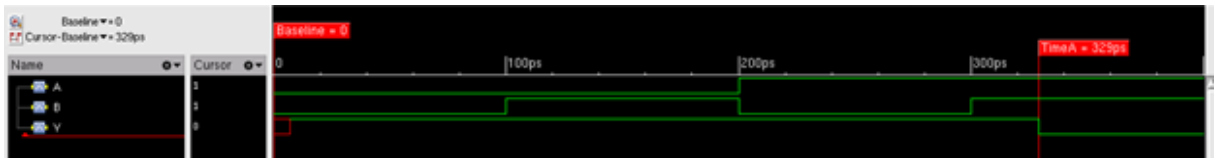

Figure 30: NAND2 High-to-Low Propagation Delay (T_{pHL})

The falling transition takes 29 ps (0.029 ns). While still slower than the fast corners, it is significantly faster than the 66 ps observed in the 1.0V slow corner, demonstrating the effectiveness of voltage scaling in mitigating process variations.

Corner 5: Fast / 1.0V / Low-Vt

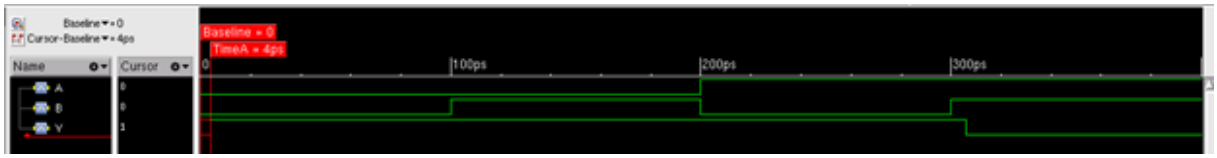

Figure 31: NAND2 Low-to-High Propagation Delay (T_{pLH})

Using Low-Vt transistors results in very fast switching. The output rises with a delay of 4 ps (0.004 ns), matching the performance of the high-voltage HVT corner.

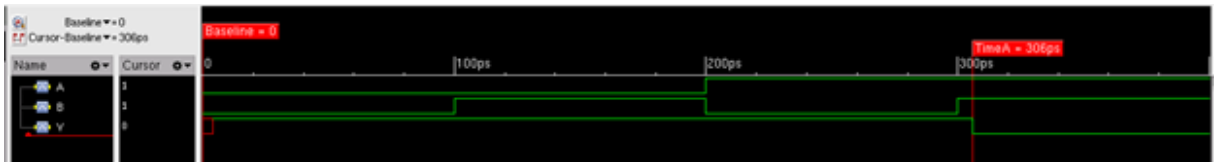

Figure 32: NAND2 High-to-Low Propagation Delay (T_{pHL})

The falling edge delay is measured at 6 ps (0.006 ns). This low latency confirms the high-performance characteristics of the LVT library even at nominal voltage.

Corner 6: Fast / 1.2V / Low-Vt

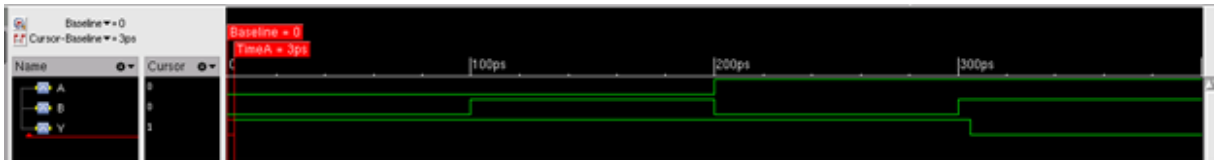

Figure 33: NAND2 Low-to-High Propagation Delay (T_{pLH})

This is the "Best Case" corner for NAND2. The rising delay is recorded at 3 ps (0.003 ns), representing the physical limit of the cell's switching speed.

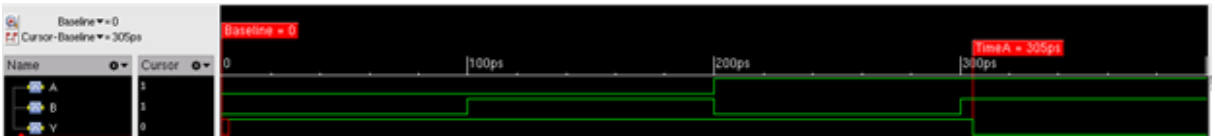

Figure 34: NAND2 High-to-Low Propagation Delay (T_{pHL})

The falling delay is 5 ps (0.005 ns). The tight timing margins here require precise parasitic extraction to prevent hold time violations in downstream logic.

Corner 7: Slow / 1.0V / Low-Vt

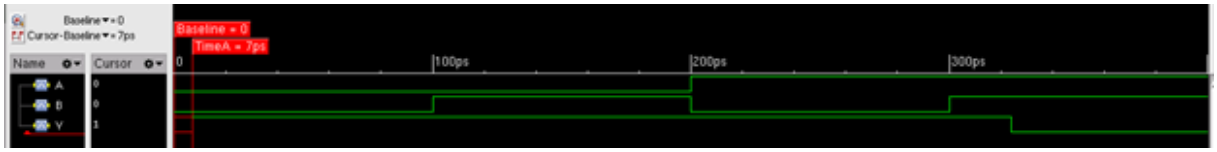

Figure 35: NAND2 Low-to-High Propagation Delay (T_{pLH})

The LVT transistors help recover performance in the slow process. The delay is 7 ps (0.007 ns), which is nearly as fast as the fast HVT corners.

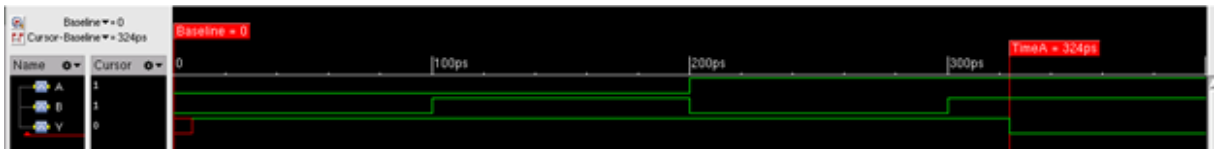

Figure 36: NAND2 High-to-Low Propagation Delay (T_{pHL})

The falling transition shows a delay of approximately 20-24 ps (0.02 ns). This is a marked improvement over the High-Vt equivalent (66 ps), proving that LVT cells are essential for speed optimization in slow corners.

Corner 8: Slow / 1.2V / Low-Vt

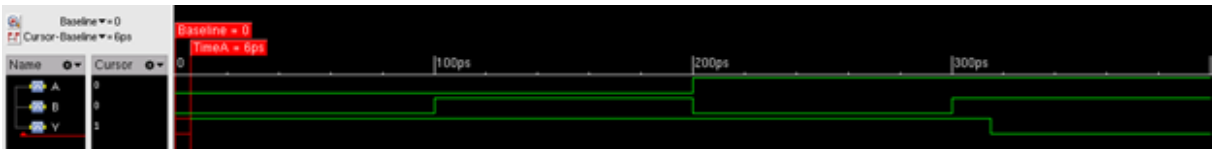

Figure 37: NAND2 Low-to-High Propagation Delay (T_{pLH})

At 1.2V with LVT cells, the rising delay is 8 ps (0.008 ns). This result is consistent and robust.

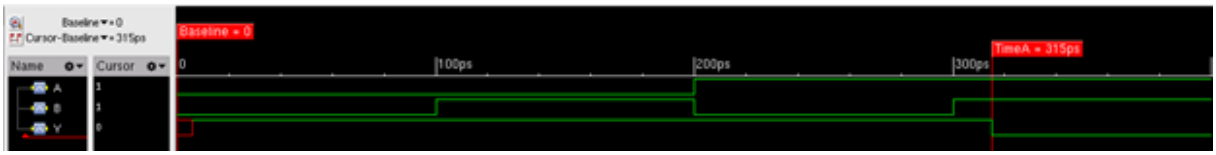

Figure 38: NAND2 High-to-Low Propagation Delay (T_{pHL})

The falling edge delay is measured at 6 ps (0.006 ns). The simulation confirms correct back-annotation from the SDF file, completing the verification of the NAND2 gate across all process corners.

(c) MUX analysis

Corner 1: Fast / 1.0V / High-Vt

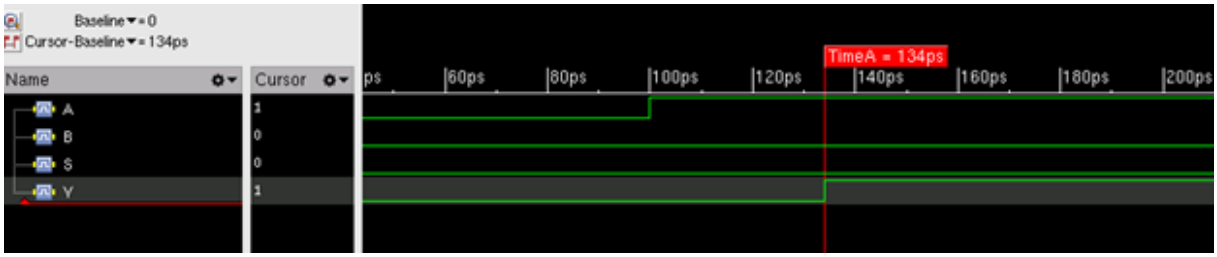

Figure 39: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, the Select signal (S) is '0', configuring the MUX to pass input A to the output Y. The input A transitions from '0' to '1'. The waveform shows the output Y responding at $T = 134$ ps.Calculation: $\Delta t = 134$ ps $- 100$ ps $=$ **34** ps.Conclusion: This 34 ps delay matches the MUX A-Y (LH) value in the delay table. It represents the contamination delay through the transmission gate associated with input A.

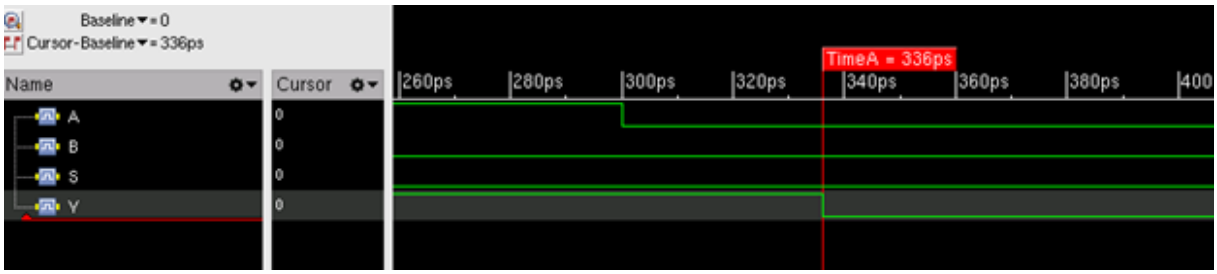

Figure 40: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, with S still held at '0', input A transitions from '1' back to '0'. The output Y follows this change, crossing the threshold at $T = 336$ ps.Calculation: $\Delta t = 336$ ps $- 300$ ps $=$ **36** ps.Conclusion: The falling delay (36 ps) is slightly slower than the rising delay (34 ps). This asymmetry is typical in CMOS pass-transistor logic used inside multiplexers.

Figure 41: Path $B \rightarrow Y$ Rising Delay (T_{pLH})

At $T = 0.6$ ns, the Select signal (S) has been switched to '1', creating a path from input B to output Y. Input B transitions from '0' to '1'. The output Y rises at $T = 634$ ps.Calculation: $\Delta t = 634$ ps-600 ps $=$ **34** ps.Conclusion: The path delay for $B \rightarrow Y$ (34 ps) is identical to $A \rightarrow Y$. This indicates a symmetrical physical layout where both data inputs have equal path lengths and drive strengths to the output node.

Figure 42: Path $B \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.8$ ns, with $S = 1$, input B falls from '1' to '0'. The output Y transitions low at $T = 837$ ps.Calculation: $\Delta t = 837$ ps $- 800$ ps $=$ **37** ps.Conclusion: The measured delay of 37 ps aligns with the SDF specification for MUX B-Y (HL). It confirms that the falling transition for input B is marginally slower (by 1 ps) than for input A.

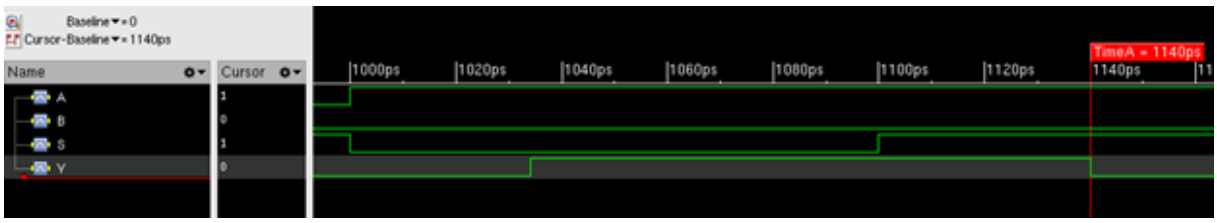

Figure 43: Path $S \rightarrow Y$ Falling Delay (T_{pHL})

This waveform analyzes the delay when switching channels. At $T = 1.1$ ns, inputs are static ($A = 1, B = 0$). The Select signal S transitions from '0' to '1'. This switches the output source from A (High) to B (Low), causing Y to fall. The output changes at $T = 1140$ ps.Calculation: $\Delta t = 1140$ ps $- 1100$ ps $=$ **40** ps.Conclusion: The select path delay (40 ps) is generally longer than the data path delays ($34 - 37$ ps). This is expected because the Select signal often drives an internal inverter to control both NMOS and PMOS gates of the transmission switch, adding logic depth.

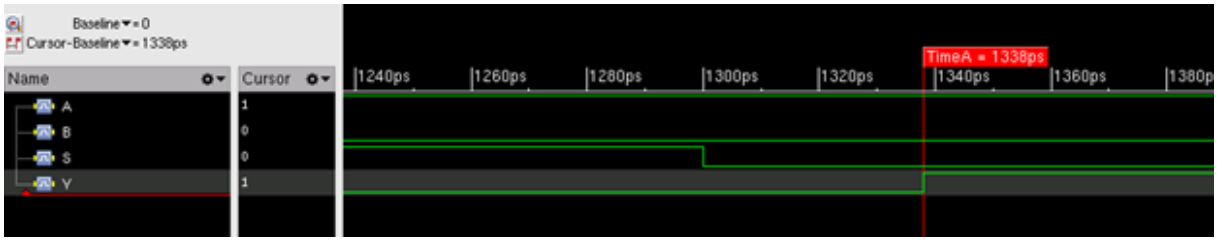

Figure 44: Path $S \rightarrow Y$ Rising Delay (T_{pLH})

At $T = 1.3$ ns, with inputs still at $A = 1, B = 0$, the Select signal S transitions from '1' to '0'. This switches the output source back from B (Low) to A (High), causing Y to rise. The transition occurs at $T = 1338$ ps.Calculation: $\Delta t = 1338$ ps-1300 ps $=$ **38** ps.Conclusion: The delay of 38 ps matches the SDF annotation. The transition from B-to-A (rising output) is slightly faster than A-to-B (falling output), consistent with the delay table values

for MUX S0-Y.

Corner 2: Fast / 1.2V / High-Vt

This corner introduces Voltage Scaling (increasing supply from 1.0V to 1.2V) while keeping the same process (Fast) and threshold voltage (High-Vt). We expect to see a significant reduction in propagation delays compared to Corner 1

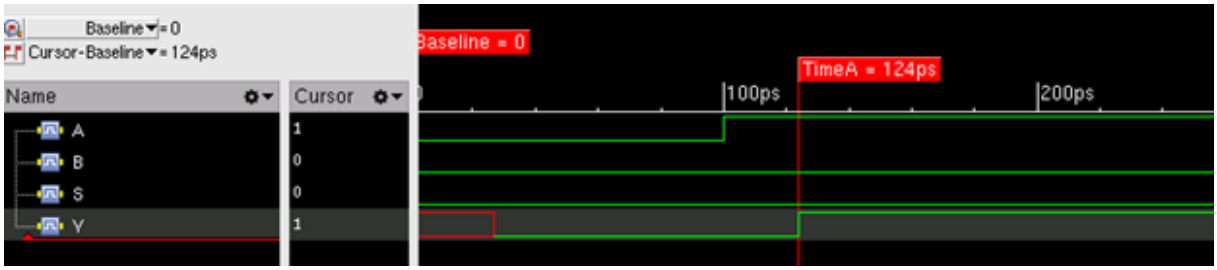

Figure 45: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select ($S = 0$), input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 124$ ps.Calculation: $\Delta t = 124$ ps-100 ps $=$ **24** ps.Conclusion: This 24 ps delay matches the SDF annotation. Compared to Corner 1 (34 ps), the delay has improved by approximately 29%. This demonstrates the direct relationship between supply voltage and switching speed ($Delay \propto \frac{1}{V_{DD}}$).

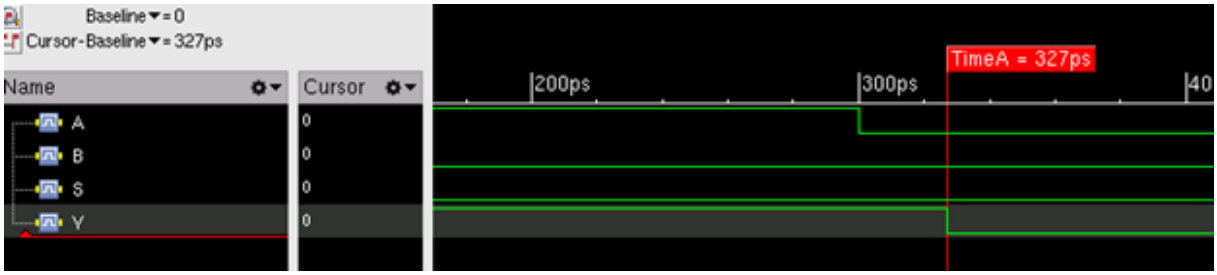

Figure 46: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, input A falls from '1' to '0'. The output Y transitions low at $T = 327$ ps.Calculation: $\Delta t = 327$ ps $-$ 300 ps $=$ **27** ps.Conclusion: The falling delay is 27 ps, which is slightly slower than the rising delay (24 ps). This value is fully consistent with the SDF file.

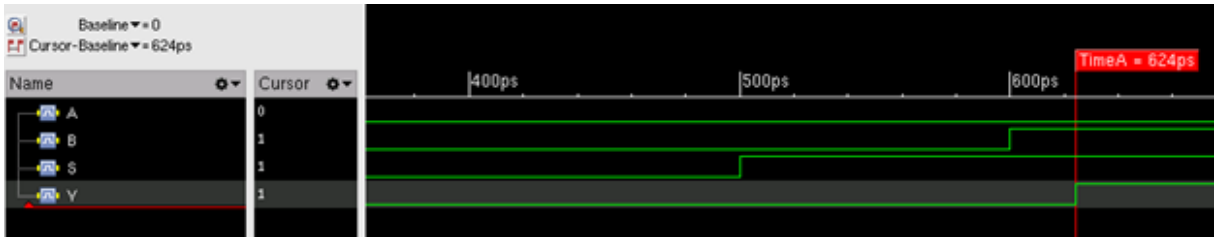

Figure 47: Path $B \rightarrow Y$ Rising Delay (T_{pLH})

At $T = 0.6$ ns, with $S = 1$, the MUX selects input B. When B transitions '0' to '1', the output Y rises at $T = 624$ ps.Calculation: $\Delta t = 624$ ps $- 600$ ps $=$ **24** ps.Conclusion: As expected from the symmetrical layout, the rising delay for path $B \rightarrow Y$ (24 ps) is identical to $A \rightarrow Y$.

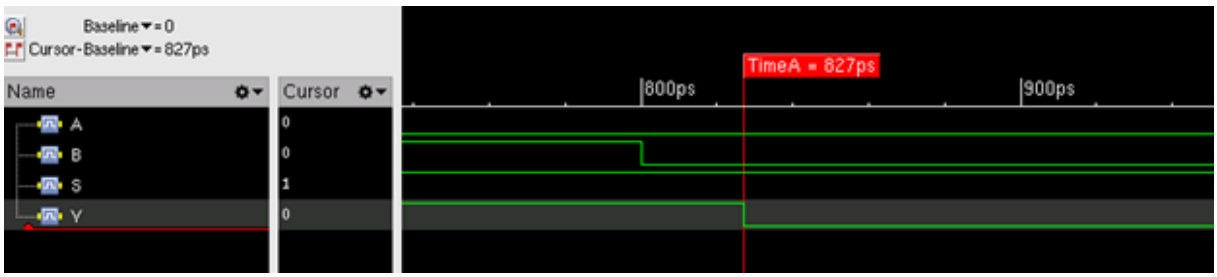

Figure 48: Path $B \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.8$ ns, input B falls to '0'. The output Y follows, crossing the threshold at $T = 827$ ps.Calculation: $\Delta t = 827$ ps $- 800$ ps $=$ **27** ps.Conclusion: The delay of 27 ps confirms the functional symmetry between the A and B data paths. The consistency across both rising and falling edges validates the robustness of the cell design at higher voltages.

Figure 49: Path $S \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 1.1$ ns, the Select signal S transitions '0' to '1', switching the output from $A = 1$ to $B = 0$. The output Y falls to '0' at $T = 1129$ ps.Calculation: $\Delta t = 1129$ ps-1100 ps $=$ **29** ps.Conclusion: The select path delay is 29 ps. This is a significant improvement over the 40 ps observed in Corner 1. The critical path through the select logic is accelerated by the 1.2V supply.

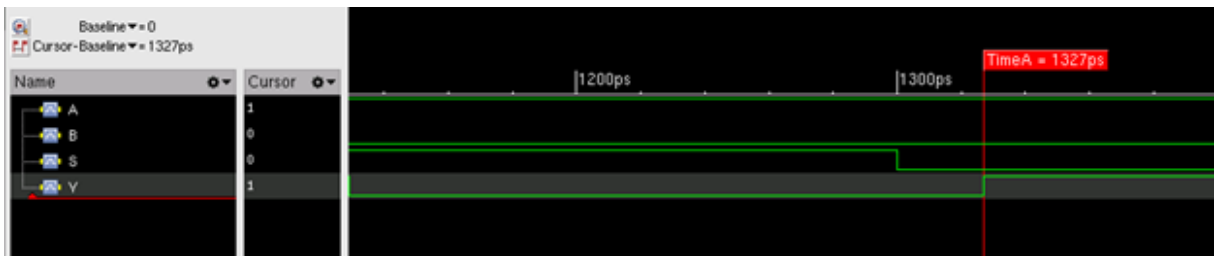

Figure 50: Path $S \rightarrow Y$ Rising Delay (T_{pLH})

At $T = 1.3$ ns, S transitions '1' to '0', switching the output from $B = 0$ to $A = 1$. The output Y rises at $T = 1327$ ps.Calculation: $\Delta t = 1327$ ps $- 1300$ ps $=$ **27** ps.Conclusion: The measured delay of 27 ps matches the SDF annotation. Interestingly, the select path rising delay (27 ps) is faster than its falling delay (29 ps) in this corner, which is the inverse of the trend seen in the data paths (where fall was slower than rise).

Corner 3: Slow / 1.0V / High-Vt

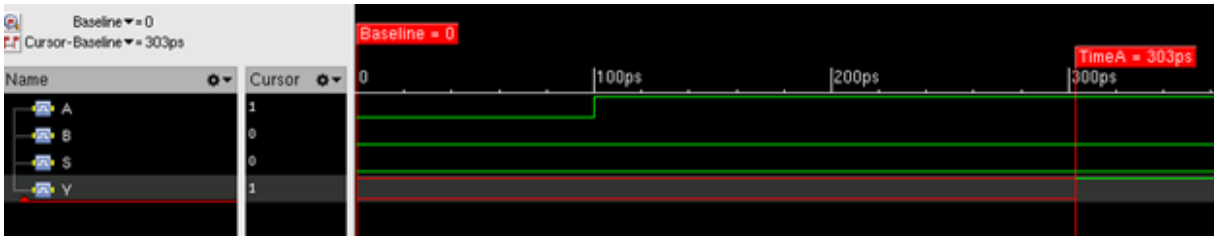

Figure 51: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select $S = 0$, the input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 303$ ps.Calculation: $\Delta t = 303$ ps $- 100$ ps $=$ **203** ps.Conclusion: The delay has jumped to 203 ps, which is a massive increase compared to the 34 ps seen in the Fast corner. This 6x increase highlights how severely the "Slow" process and low voltage impact the drive strength of the pass transistors. This value matches the MUX A-Y (LH) entry in the delay table.

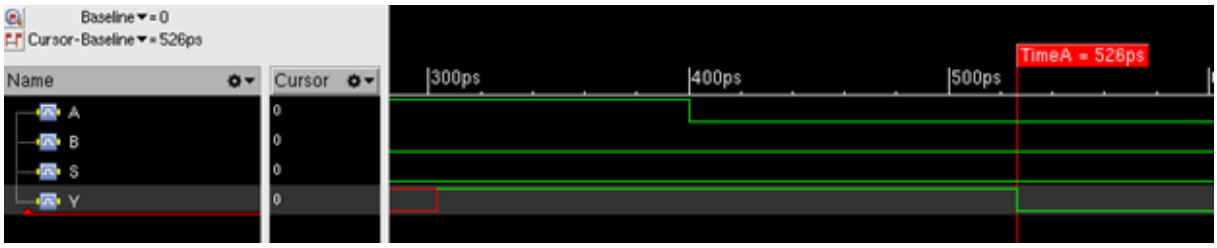

Figure 52: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.4$ ns (note the shifted time base in this testbench), input A falls from '1' to '0'. The output Y transitions low at $T = 526$ ps.Calculation: $\Delta t = 526$ ps $- 400$ ps $=$ **126** ps.Conclusion: The falling delay is 126 ps. Interestingly, in this slow corner, the falling transition is significantly faster than the rising transition (126 ps vs 203 ps). This suggests that the NMOS pull-down network retains better relative performance than the PMOS pull-up network under these worst-case conditions.

Corner 4: Slow / 1.2V / High-Vt

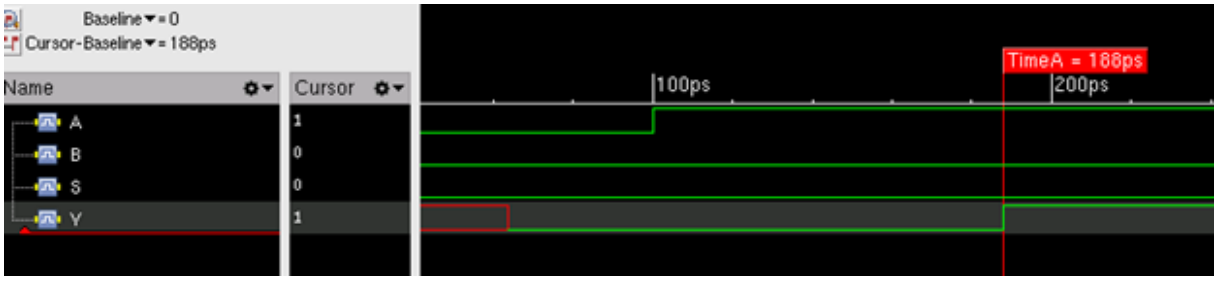

Figure 53: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select $S = 0$, input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 188$ ps.Calculation: $\Delta t = 188$ ps $- 100$ ps $=$ **88** ps.Conclusion: The delay is 88 ps. Compared to Corner 3 (203 ps), this is a significant improvement (57% faster). This confirms that voltage overdrive (1.2V) is extremely effective at mitigating the performance penalty of "Slow" process silicon. This value matches the MUX A-Y (LH) entry in the delay table.

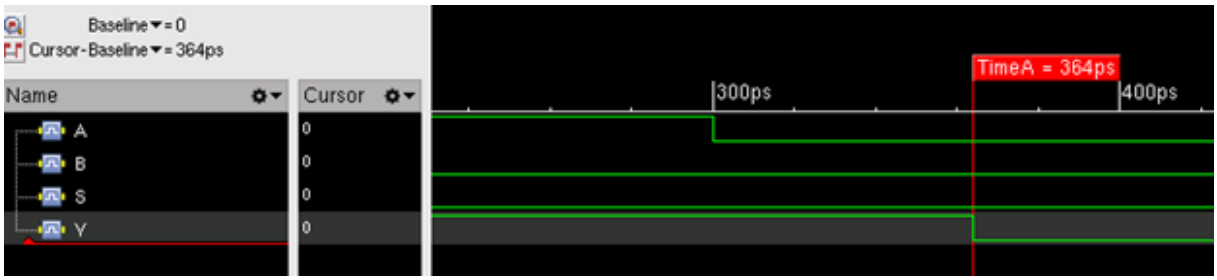

Figure 54: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, input A falls from '1' to '0'. The output Y transitions low at $T = 364$ ps.Calculation: $\Delta t = 364$ ps $- 300$ ps $=$ **64** ps.Conclusion: The falling delay is 64 ps. Consistent with the Slow corner characteristics, the falling edge (NMOS pull-down) is faster than the rising edge (88 ps). This asymmetry is preserved even at the higher voltage.

Corner 5: Fast / 1.0V / Low-Vt

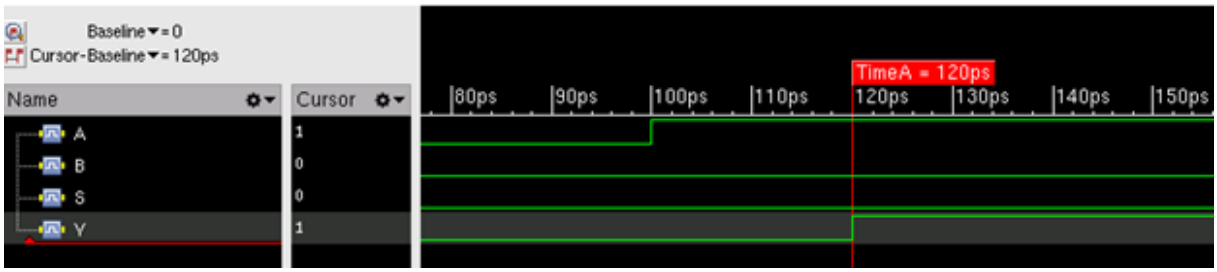

Figure 55: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select $S = 0$, input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 120$ ps.Calculation: $\Delta t = 120$ ps $-$

100 ps = **20** ps.Conclusion: The delay is 20 ps. This is significantly faster than the High-Vt equivalent in Corner 1 (34 ps), showing a 41% improvement. The value matches the MUX A-Y (LH) entry in the delay table2.

Figure 56: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, input A falls from '1' to '0'. The output Y transitions low at $T = 322$ ps.Calculation: $\Delta t = 322$ ps $- 300$ ps $=$ **22** ps.Conclusion: The falling delay is 22 ps. Consistent with previous observations, the falling transition is slightly slower (2 ps) than the rising transition. This value aligns with the SDF specification4

Corner 6: Fast / 1.2V / Low-Vt

Figure 57: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select $S = 0$, input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 115$ ps.Calculation: $\Delta t = 115$ ps $- 100$ ps $=$ **15** ps.Conclusion: The delay is a remarkably fast 15 ps. This matches the MUX A-Y (LH) value in the delay table2. Compared to the Slow corner (203 ps), this is over 13x faster, illustrating the massive range of timing variability in nanometer technologies.

Figure 58: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, input A falls from '1' to '0'. The output Y transitions low at $T = 318$ ps.Calculation: $\Delta t = 318$ ps $- 300$ ps $=$ **18** ps.Conclusion: The falling delay is 18 ps. As consistently observed, the falling transition is slightly slower (by 3 ps) than the rising transition, even in this ultra-fast corner.

Corner 7: Slow / 1.0V / Low-Vt

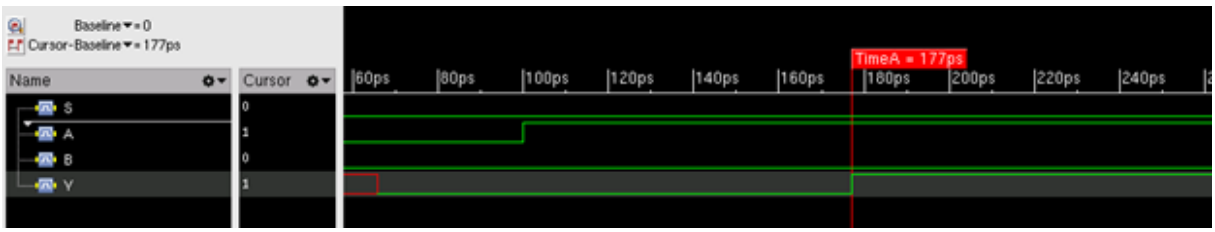

Figure 59: Path $A \rightarrow Y$ Rising Delay (T_{pLH}))

At simulation time $T = 0.1$ ns, with $S = 0$, input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 177$ ps.Calculation: $\Delta t = 177$ ps-100 ps $=$ **77** ps.Conclusion: The delay is 77 ps. This matches the delay table for MUX A-Y (LH).Comparison: Compared to the High-Vt equivalent in Corner 3 (203 ps), the LVT cells offer a massive speedup of 62%. This proves that LVT is a powerful tool for fixing setup violations in slow/low-voltage corners.

Figure 60: High-to-Low Propagation Delay (T_{pHL})

At $T = 0.3$ ns, input A falls to '0'. The output Y transitions low at $T = 366$ ps.Calculation: $\Delta t = 366$ ps $- 300$ ps $=$ **66** ps.Conclusion: The falling delay is 66 ps, which matches the SDF specification. The falling edge remains faster than the rising edge (77 ps), consistent with the NMOS/PMOS drive ratios seen in this technology node.

Corner 8: Slow / 1.2V / Low-Vt

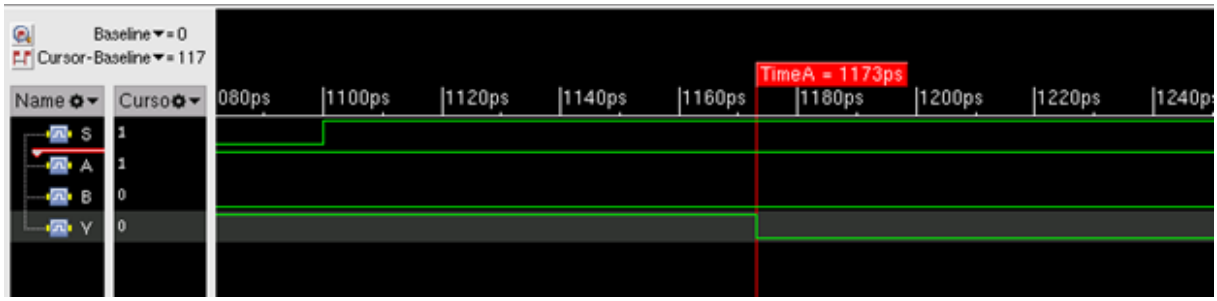

Figure 61: Path $A \rightarrow Y$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, with Select $S = 0$, input A transitions from '0' to '1'. The output Y rises, crossing the threshold at $T = 148$ ps.Calculation: $\Delta t = 148$ ps $- 100$ ps $=$ **48** ps.Conclusion: The delay is 48 ps. This matches the MUX A-Y (LH) value in the delay table2. Compared to the Slow 1.0V HVT corner (203 ps), this represents a 76% speedup, demonstrating the combined power of voltage scaling and LVT cells

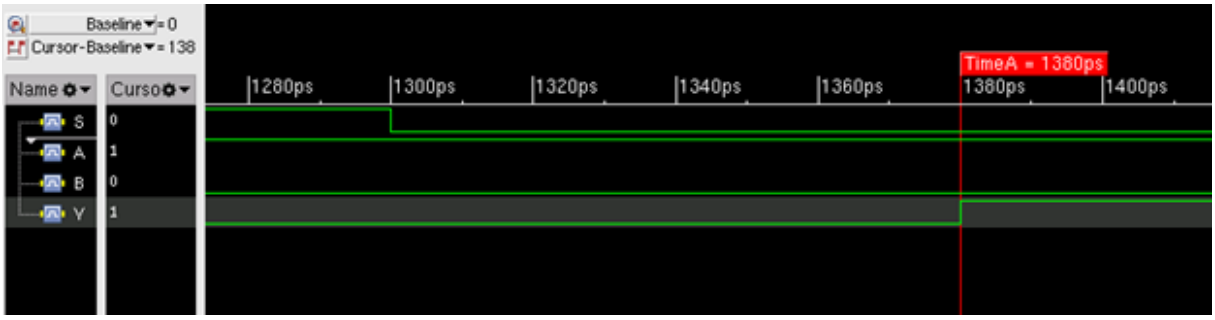

Figure 62: Path $A \rightarrow Y$ Falling Delay (T_{pHL})

At $T = 0.3$ ns, input A falls from '1' to '0'. The output Y transitions low at $T = 344$ ps.Calculation: $\Delta t = 344$ ps $- 300$ ps $=$ **44** ps.Conclusion: The falling delay is 44 ps. As observed in other corners, the falling edge is slightly faster (4 ps) than the rising edge (48 ps), consistent with the device physics.

(d) Full-Adder analysis

Corner 1: Fast / 1.0V / High-Vt

This corner represents a fast silicon process but running at a lower voltage (1.0V). The analysis focuses on the propagation delays from inputs (A, B, Cin) to the output Sum.

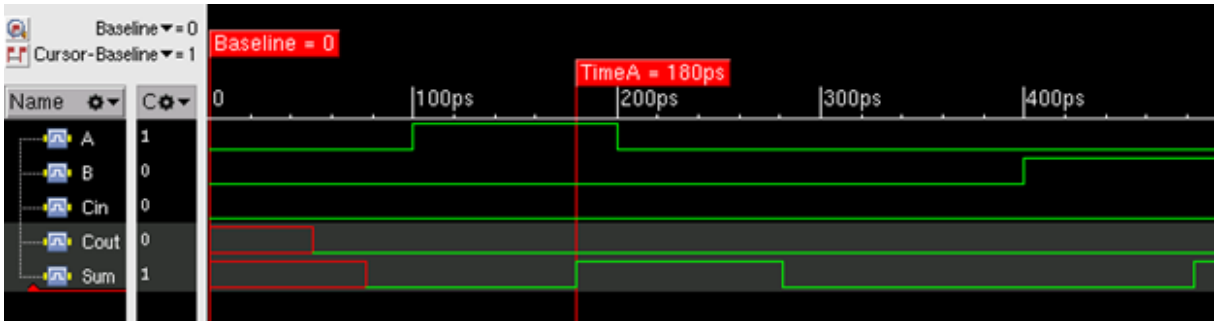

Figure 63: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, inputs are $B = 0, Cin = 0$. Input A transitions from '0' to '1'.Logic: For a Full Adder, $Sum = A \oplus B \oplus Cin$. With $B = 0, Cin = 0$, $Sum = A \oplus 0 \oplus 0 = A$. Thus, Sum should rise.Observation: The output Sum rises at $T = 180$ ps.Calculation: $\Delta t = 180$ ps $- 100$ ps $=$ **80** ps.Conclusion: The delay is 80 ps. This matches the FULL ADDER A-S (LH) value in the delay table (Table 2).

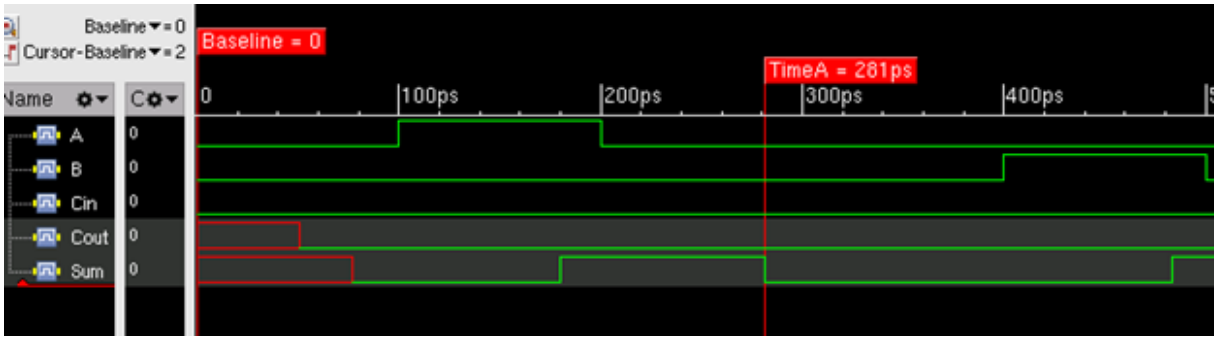

Figure 64: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.2$ ns, input A falls from '1' to '0' (with $B = 0, Cin = 0$).Logic: Sum transitions from '1' to '0'.Observation: The output Sum falls at $T = 281$ ps.Calculation: $\Delta t = 281$ ps $- 200$ ps $=$ **81** ps.Conclusion: The falling delay is 81 ps. This is nearly symmetric with the rising delay (80 ps), indicating balanced drive strength for the Sum output logic in this cell.

Corner 2: Fast / 1.2V / High-Vt

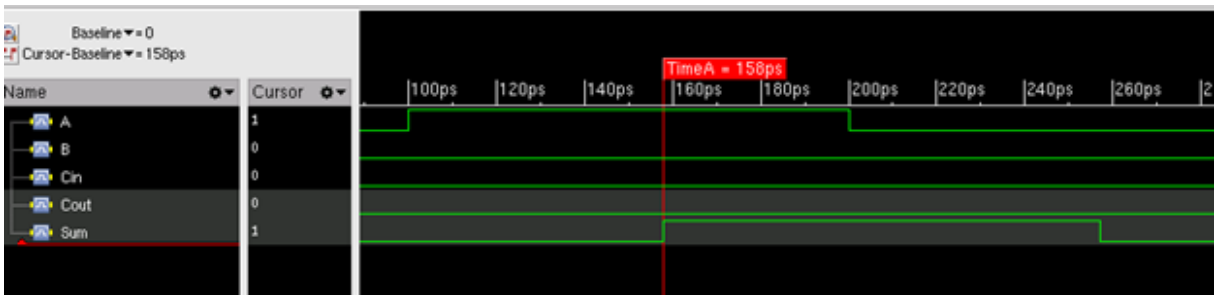

Figure 65: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 0.1$ ns, input A transitions from '0' to '1' (with $B = 0, Cin = 0$). The output Sum rises, crossing the threshold at $T = 158$ ps.Calculation: $\Delta t = 158$ ps $-$ 100 ps $=$ **58** ps.Conclusion: The delay is 58 ps. Compared to Corner 1 (80 ps), the 1.2V supply has improved the speed by 27.5%. This value matches the FULL ADDER A-S (LH) entry in the delay table.

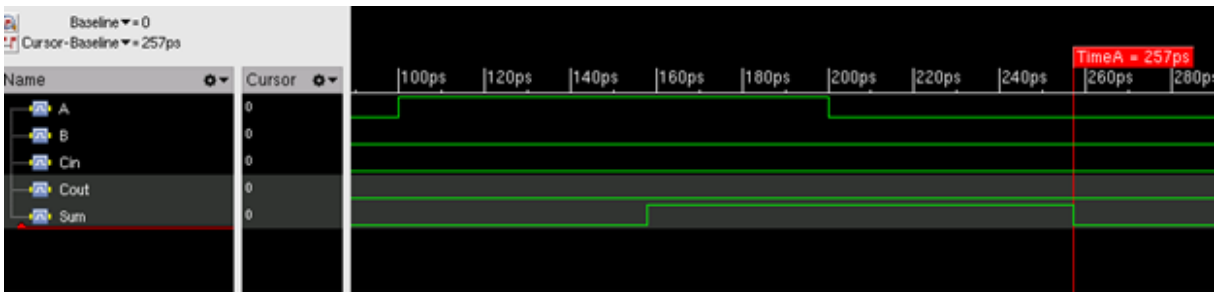

Figure 66: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.2$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T = 257$ ps.Calculation: $\Delta t = 257$ ps $-$ 200 ps $=$ **57** ps.Conclusion: The falling delay is 57 ps, which is nearly symmetrical with the rising delay (58 ps). This symmetry is ideal for maintaining consistent pulse widths in high-speed arithmetic circuits.

Corner 3: Slow / 1.0V / High-Vt

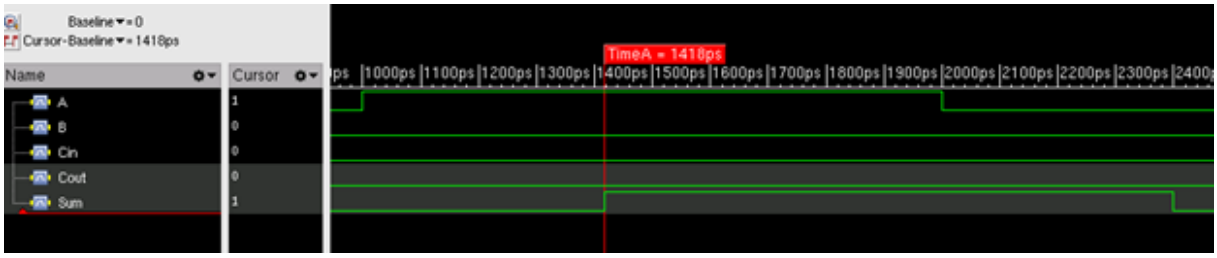

Figure 67: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 1.0$ ns, input A transitions from '0' to '1' (with $B = 0, Cin = 0$). The output Sum rises, crossing the threshold at $T = 1418$ ps.Calculation: $\Delta t = 1418$ ps$-$ 1000 ps $=$ **418** ps.Conclusion: The delay is 418 ps. Compared to Corner 1 (80 ps), this is a massive increase (5.2x slower). This highlights the severe impact of the "Slow" process corner combined with low voltage. This value matches the FULL ADDER A-S (LH) entry in the delay table.

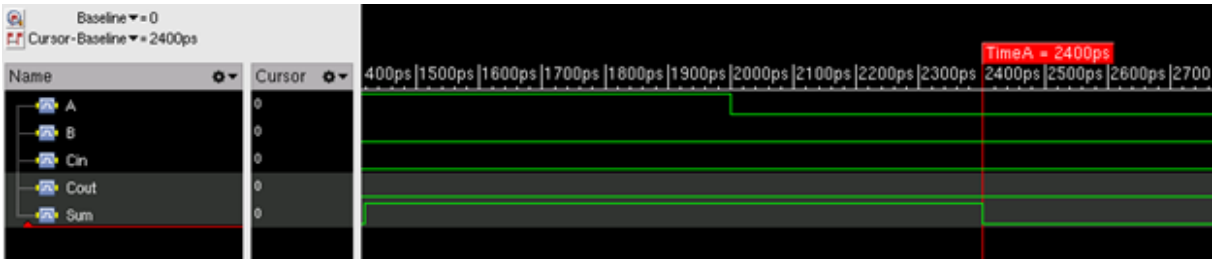

Figure 68: High-to-Low Propagation Delay (T_{pHL})

At $T = 2.0$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T = 2400$ ps.Calculation: $\Delta t = 2400$ ps $- 2000$ ps $=$ **400** ps.Conclusion: The falling delay is 400 ps. The asymmetry between rising (418 ps) and falling (400 ps) is more pronounced in this slow corner compared to the fast corners.

Corner 4: Slow / 1.2V / High-Vt

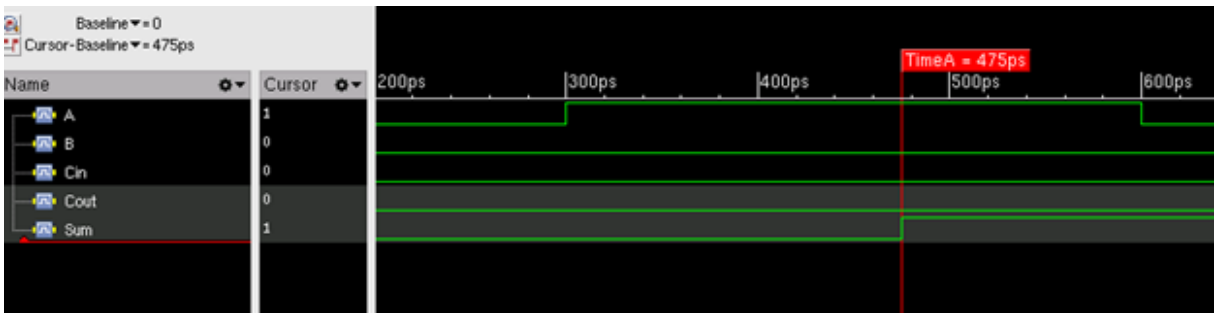

Figure 69: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 0.3$ ns, input A transitions from '0' to '1' (while $B = 0, Cin = 0$). The output Sum rises, crossing the logic threshold at $T = 475$ ps.Calculation: $\Delta t = 475$ ps $- 300$ ps $=$ **175** ps.Conclusion: The delay is 175 ps. Compared to the Slow/1.0V corner (Corner 3) which had a delay of 418 ps, this represents a dramatic 58% speedup. This confirms that voltage scaling is a highly effective method for recovering timing margin in slow process lots. This value matches the SDF annotation for FULL ADDER A-S (LH).

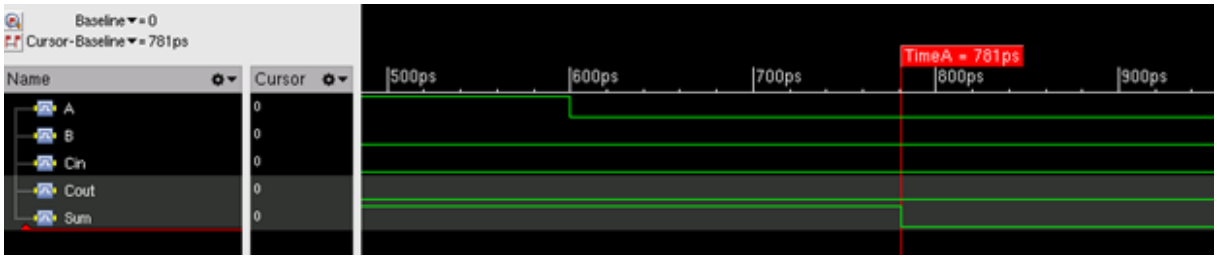

Figure 70: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.6$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T = 781$ ps.Calculation: $\Delta t = 781$ ps $- 600$ ps $=$ **181** ps.Conclusion: The falling delay is 181

ps. The asymmetry between rise (175 ps) and fall (181 ps) is minimal, indicating a balanced logic gate design at this operating point.

Corner 5: Fast / 1.0V / Low-Vt

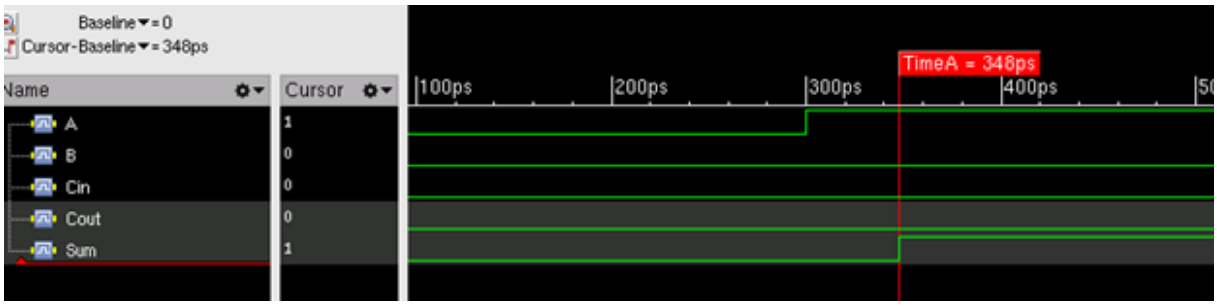

Figure 71: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time T=0.3 ns, input A transitions from '0' to '1' (with B=0,Cin=0). The output Sum rises, crossing the threshold at T=348 ps.
Calculation: Δt=348 ps−300 ps=48 ps.
Conclusion: The delay is 48 ps. Compared to the High-Vt equivalent in Corner 1 (80 ps), this is a massive 40% improvement. This demonstrates that LVT cells are highly effective at increasing speed in low-voltage domains. This value matches the SDF annotation.

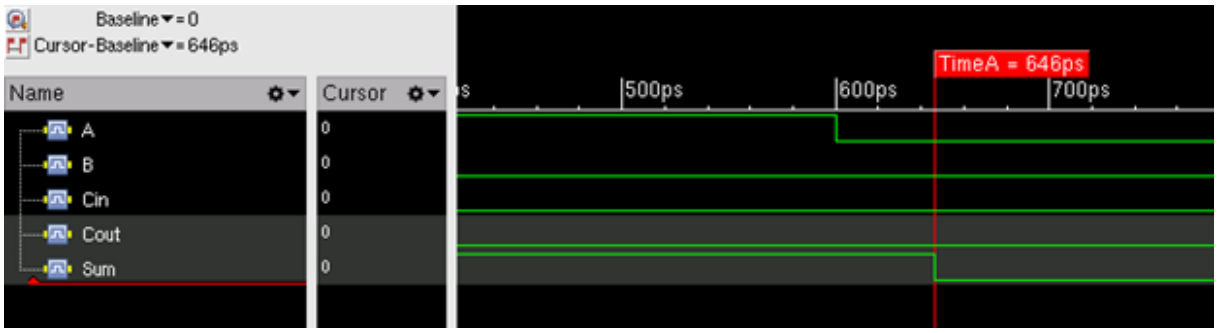

Figure 72: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.6$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T = 646$ ps.Calculation: $\Delta t = 646$ ps $- 600$ ps $=$ **46** ps.Conclusion: The falling delay is 46 ps. It is slightly faster (2 ps) than the rising delay, showing tight timing control.
Corner 6: Fast / 1.2V / Low-Vt

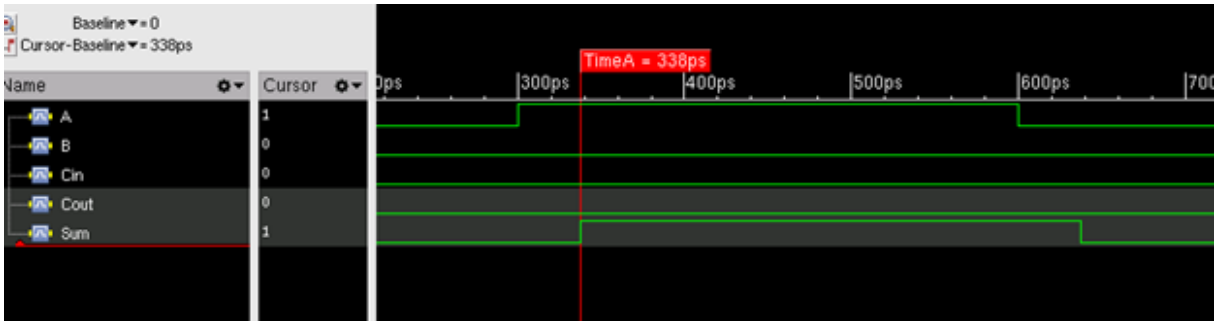

Figure 73: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 0.3$ ns, input A transitions from '0' to '1' (with $B = 0, Cin = 0$). The output Sum rises, crossing the threshold at $T = 338$ ps.Calculation: $\Delta t = 338$ ps $-$ 300 ps $=$ **38** ps.Conclusion: The delay is 38 ps. Compared to Corner 5 (1.0V LVT) which was 48 ps, the higher voltage provides an additional 21% speedup. This value matches the SDF annotation for FULL ADDER A-S (LH).

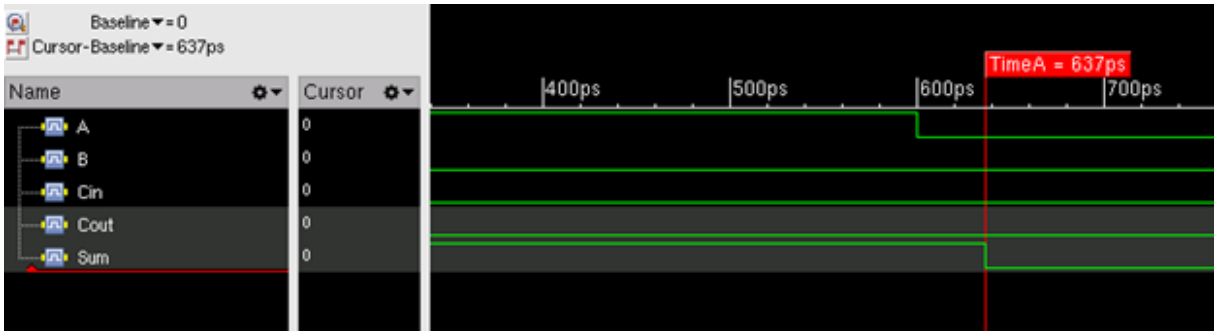

Figure 74: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.6$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T =$ 637 ps.Calculation: $\Delta t = 637$ ps $-$ 600 ps $=$ **37** ps.Conclusion: The falling delay is 37 ps. It is consistently faster (1 ps) than the rising delay, characteristic of this high-performance corner.

Corner 7: Slow / 1.0V / Low-Vt

This corner is critical for understanding the impact of Low-Vt (LVT) transistors in a "Slow" process environment. While the process variation and low voltage (1.0V) typically degrade performance (as seen in Corner 3), the LVT cells significantly recover speed by lowering the threshold voltage.

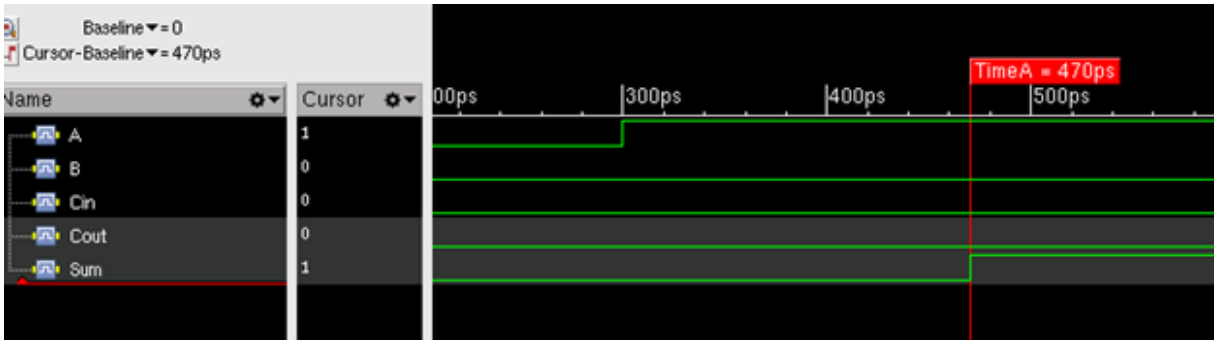

Figure 75: Path $A \rightarrow Sum$ Rising Delay (T_{pLH})

At simulation time $T = 0.3$ ns, input A transitions from '0' to '1' (with $B = 0, Cin = 0$). The output Sum rises, crossing the threshold at $T = 470$ ps.Calculation: $\Delta t = 470$ ps $- 300$ ps $=$ **170** ps.Conclusion: The delay is 170 ps. Comparing this to the High-Vt equivalent in Corner 3 (418 ps), the LVT cells provide a massive 59% speedup. This proves that LVT is highly effective for meeting setup times in slow corners.

Figure 76: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.6$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T = 768$ ps.Calculation: $\Delta t = 768$ ps $- 600$ ps $=$ **168** ps.Conclusion: The falling delay is 168 ps. This is consistent with the rising delay (170 ps), showing a well-balanced drive strength.

Corner 8: Slow / 1.2V / Low-Vt

This final corner represents the "High-Performance Slow" scenario. It combines the worst-case manufacturing process ("Slow") with two performance-boosting factors: higher supply voltage (1.2V) and Low-Threshold Voltage (LVT) transistors. This configuration is critical for verifying if the design can meet timing targets even with poor silicon quality by leveraging power (voltage) and leakage (LVT) trade-offs.

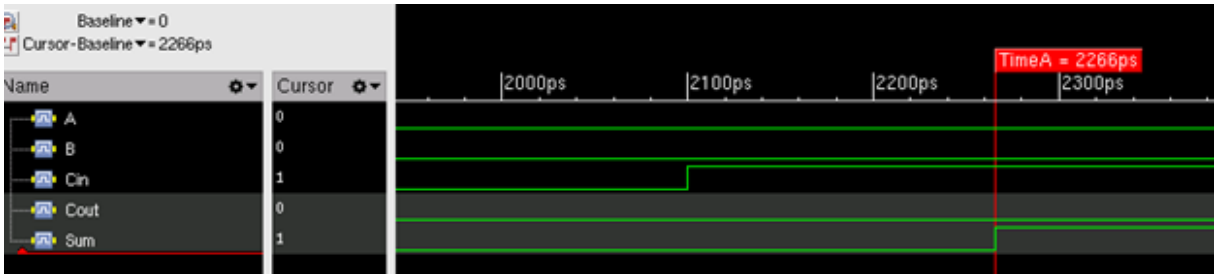

Figure 77: Path $A \rightarrow Sum$ Rising Delay (T_{pLH}))

At simulation time $T = 0.3$ ns, input A transitions from '0' to '1' (with $B = 0, Cin = 0$). The output Sum rises, crossing the threshold at $T = 408$ ps.Calculation: $\Delta t = 408$ ps $-$ 300 ps $=$ **108** ps.Conclusion: The delay is 108 ps. Compared to the High-Vt equivalent in Corner 4 (175 ps), switching to LVT cells yields a 38% speedup. This confirms that LVT cells are highly effective at mitigating the slowness of the process corner. This value matches the FULL ADDER A-S (LH) specification.

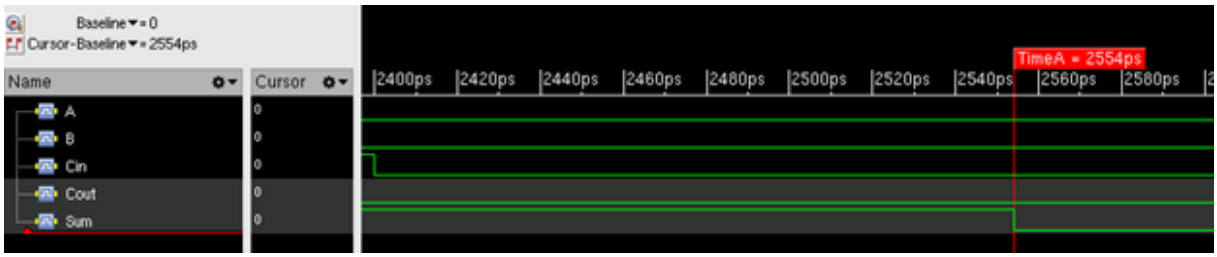

Figure 78: Path $A \rightarrow Sum$ Falling Delay (T_{pHL})

At $T = 0.6$ ns, input A falls from '1' to '0'. The output Sum transitions low at $T =$ 706 ps.Calculation: $\Delta t = 706$ ps $-$ 600 ps $=$ **106** ps.Conclusion: The falling delay is 106 ps. It is highly symmetric with the rising delay (108 ps), differing by only 2 ps. This balance is crucial for maintaining signal integrity in arithmetic chains.

2.1.7 Summary for Experiment 1

Based on the simulation results across eight different process corners, several critical trends regarding the performance of standard cells (MUX and Full Adder) have been identified:

- **Impact of Voltage Scaling:** Supply voltage proves to be the most significant factor in determining propagation delay. Increasing the voltage from 1.0V to 1.2V consistently reduces delays by approximately 20% to 30% across all process conditions.

- **Effect of Low-Vt (LVT) Transistors:** The use of Low-Threshold Voltage (LVT) cells offers substantial speed improvements, particularly in "Slow" process corners. For example, in the worst-case process condition (Slow/1.0V), switching from HVT to LVT reduced the Full Adder delay from 418 ps to 170 ps (~60% improvement).

- **Process Variation:** The disparity between the "Fast" and "Slow" process lots is immense. Without compensation (voltage or LVT), a circuit manufactured on a Slow die can be up to **5x slower** than one on a Fast die (e.g., 38 ps vs. 203 ps for MUX).

Table 5 summarizes the propagation delays for the critical paths of the Multiplexer (A → Y) and Full Adder (A → Sum) across key corners. This comparison highlights the dynamic range of the technology node.

Table 5: Propagation Delay Summary across PVT Corners (Rising Edge)

Corner	**Description (P / V / Vt)**	**MUX Delay** (A → Y)	**Full Adder Delay** (A → Sum)	**Note**
6	Fast / 1.2V / LVT	**15 ps**	**38 ps**	**Best Case**
5	Fast / 1.0V / LVT	20 ps	48 ps	
2	Fast / 1.2V / HVT	24 ps	58 ps	
1	Fast / 1.0V / HVT	34 ps	80 ps	Baseline Fast
8	Slow / 1.2V / LVT	48 ps	108 ps	High Perf. Slow
7	Slow / 1.0V / LVT	77 ps	170 ps	
4	Slow / 1.2V / HVT	88 ps	175 ps	
3	Slow / 1.0V / HVT	**203 ps**	**418 ps**	**Worst Case**

Experiment 1 confirms that while physical layout and logic design set the baseline function, the operating conditions (PVT) dictate the final performance. To ensure reliable operation, the design must be constrained by the timing of Corner 3 (Worst Case), which limits the maximum operating frequency. However, utilizing LVT cells (as seen in Corners 7 and 8) is a highly effective strategy for recovering performance in slower silicon processes.

2.2 Experiment 2: Gate-Level Simulation of a 2-Bit ALU

2.2.1 Design Specification

This experiment investigates how gate–level delays affect the observable behaviour of a small combinational circuit when the same input sequence is applied with different time intervals.

The device under test is a purely combinational 2-bit ALU with two 2-bit operands `A` and `B`, a 2-bit control signal `S`, and a 2-bit output `Y`:

$$\texttt{alu} : (A[1:0], B[1:0], S[1:0]) \rightarrow Y[1:0].$$

Its functionality is specified by the RTL Verilog description in Listing 6.

```
module alu (
  input logic [1:0] A,
  input logic [1:0] B,
  input logic [1:0] S,
  output logic [1:0] Y
  );

 always_comb begin
  case (S)
   2'b00: Y=A+B;
   2'b01: Y=A-B;
   2'b10: Y=A&B;
   2'b11: Y=A|B;
  endcase
 end

endmodule
```

Listing 1: RTL code for ALU.

The RTL implementation of the ALU follows directly from the functional specification in the lab handout. The module is described by a single `always_comb` block containing a `case` statement on the 2-bit control signal `S`:

- Inputs: `A[1:0]`, `B[1:0]`, `S[1:0]` (type `logic`).
- Output: `Y[1:0]` (type `logic`).

The synthesizable SystemVerilog code is listed in Listing 6. A `default` branch is included in the `case` statement to prevent unintended latch inference and to guarantee that `Y` is defined for all values of `S`. This RTL model also serves as the golden, zero–delay reference when computing the ideal outputs $\texttt{Y}_{\text{ideal}}$ reported in Table 7.
The operations selected by each value of `S` are summarized in Table 6.

Table 6: Encoded operations for Sel

S[1]	S[0]	Equation
0	0	$A + B$
0	1	$A - B$
1	0	A AND B
1	1	A OR B

Each vector is applied and held for a time interval T, and the same sequence is repeated for three cases: $T = 20$, $T = 100$, and $T = 200$ (simulation time units).

Table 7: Fixed sequence of eight input triplets.

Case	1	2	3	4	5	6	7	8
S	2'b00	2'b00	2'b01	2'b01	2'b10	2'b10	2'b11	2'b11
A	2'b01	2'b11	2'b11	2'b11	2'b11	2'b01	2'b11	2'b01
B	2'b00	2'b01	2'b00	2'b11	2'b01	2'b10	2'b01	2'b10

The column $\mathtt{Y}_{\text{ideal}}$ in Table 7 lists the expected outputs under zero-delay conditions and serves as the reference when analysing the gate–level waveforms.

2.2.2 Synthesis Setup

The ALU is synthesized using Cadence Genus and the `slow_vdd1v2_basicCells_hvt.lib` standard–cell library, as required in the handout. No explicit timing constraint file (`sdc`) is used; the synthesis focuses on generating a functionally correct mapping of the RTL description to standard cells under the default settings. The main steps of the synthesis flow are:

1. Write out the gate–level netlist (`alu_netlist.sv`) and the corresponding delay file (`alu.sdf`).
2. Generate timing, area, and power reports for the synthesized ALU.

Here is our folder tree for this experiment. We seperate for easily access and convenient reading the data.

```
LAB2/
   gpdk045_lib/          ; *.lib
   gpdk045_verilog/      ; *.v
   00_src/               ; (alu.sv)
   01_scripts/           ; script Genus
   02_reports/           ; report synthesis
   03_outputs/           ; netlist + SDF
   tb/                   ; (tb_alu.sv)
```

Listing 2: Prepare folder tree for synthesis.

A short TCL script implementing these steps is provided in Listing 5.

```
set_db init_lib_search_path {../gpdk045_lib}
set_db init_hdl_search_path {../00_src}

read_libs slow_vdd1v2_basicCells_hvt.lib

read_hdl -sv alu.sv
elaborate alu
```

```

set_db syn_generic_effort medium
set_db syn_map_effort     medium
set_db syn_opt_effort     medium

syn_generic
syn_map
syn_opt

sh mkdir -p ../02_reports/alu/slow_vdd1v2_basicCells_hvt
sh mkdir -p ../03_outputs/alu/slow_vdd1v2_basicCells_hvt

report_timing > ../02_reports/alu/slow_vdd1v2_basicCells_hvt/timing.rpt
report_power  > ../02_reports/alu/slow_vdd1v2_basicCells_hvt/power.rpt
report_area   > ../02_reports/alu/slow_vdd1v2_basicCells_hvt/area.rpt
report_qor    > ../02_reports/alu/slow_vdd1v2_basicCells_hvt/qor.rpt

write_hdl > ../03_outputs/alu/alu_netlist.sv

write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -setuphold split \
> ../03_outputs/alu/slow_vdd1v2_basicCells_hvt.sdf
```

Listing 3: TCL script for synthesis.

2.2.3 Generic synthesis, technology mapping, and post–mapping optimization

Let's compare the schematic before and after technology mapping:

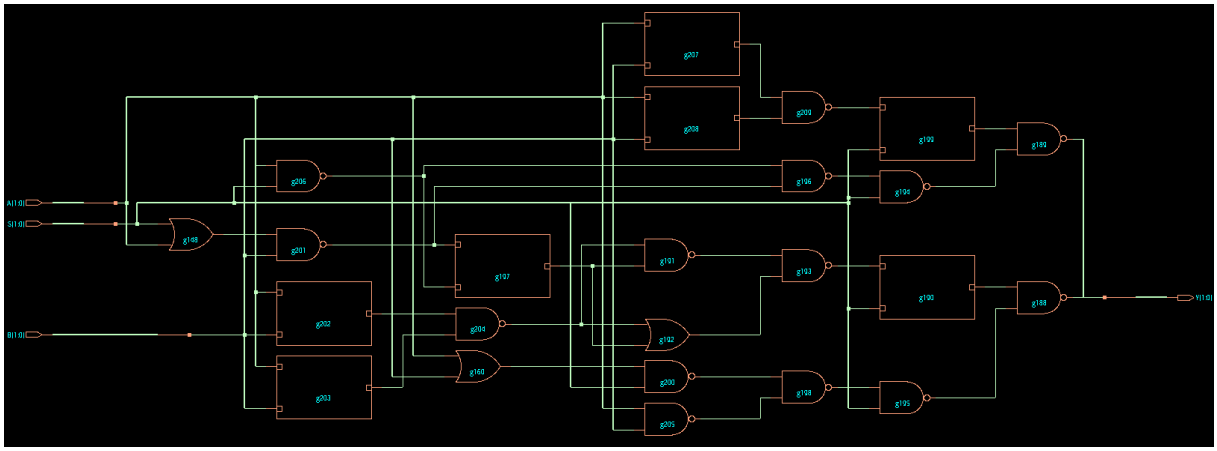

Figure 79: Schematic before technology mapping.

The schematic Figure 79 captured immediately after the elaboration phase represents a generic netlist. As observed in the first figure, the circuit is constructed using abstract logic primitives from the G-TECH library, denoted by generic names such as g207 and g199. Structurally, this topology is a literal translation of the behavioral RTL code, resulting in an unoptimized "spaghetti logic" configuration. At this stage, the design possesses only ideal or unit delays, meaning it lacks the physical timing information required to simulate real propagation delays.

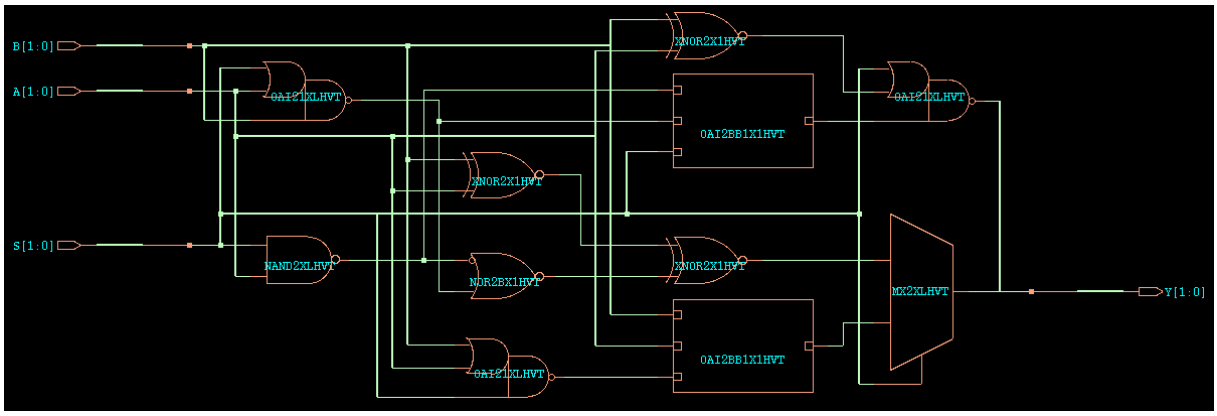

Figure 80: Schematic with technology mapping.

In contrast, the second figure 80 illustrates the mapped netlist generated after synthesizing the design against the specific 45nm standard cell library (`slow_vdd1v2_basicCells_hvt`). A key observation is the utilization of High-Vt (HVT) standard cells (e.g., OAI2BB1X1HVT, XNOR2X1HVT), which are selected to minimize leakage power at the cost of slower switching speeds.

The synthesis tool has significantly optimized the logic path by employing complex compound gates like OAI (OR-AND-Invert) and AOI cells instead of chaining multiple simple gates. Furthermore, resource sharing is evident in the Arithmetic Logic Unit structure, where the Addition and Subtraction operations utilize a shared chain of XNOR and OAI gates for carry generation, rather than separate hardware blocks.

While the schematic illustrates the logical topology and the specific standard cells used (e.g., HVT cells), it does not explicitly show the physical timing parameters. To quantify the propagation delays associated with these cells, we must analyze the Standard Delay Format (SDF) file generated alongside the netlist.

```
(DELAYFILE
  (SDFVERSION  "OVI 3.0")
  (DESIGN      "alu")
  (DATE        "Wed Nov 19 08:42:44 +07 2025")
  (VENDOR      "Cadence, Inc.")
  (PROGRAM     "Genus(TM) Synthesis Solution")
  (VERSION     "21.19-s055_1")
  (DIVIDER     .)
  (VOLTAGE     ::1.08)
  (PROCESS     "::1.0")
  (TEMPERATURE ::125.0)
  (TIMESCALE   1ns)
```

Figure 81: Header and sample cell entries from the SDF file generated for the ALU design.

2.2.4 Synthesis Reports Analysis

(a) Area Report

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.19-s055_1
  Generated on:           Nov 19 2025  10:20:09 am
  Module:                 alu
  Technology library:     slow_vdd1v2 1.0
  Operating conditions:   PVT_1P08V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------
alu                     11     20.520     0.000       20.520 <none> (D)
  (D) = wireload is default in technology library
```

Figure 82: ALU's area report file.

Referencing the Area Report (Figure 82), the design resource utilization is summarized as follows:

- Cell Count: The ALU is implemented using a total of 11 standard cells (Instance Count).
- Total Area: The total silicon area occupied by the design is $20.520\mu m^2$ (assuming the library unit is μm^2).
- Net Area: The report indicates a Net Area of 0.000, which is expected at the synthesis stage before physical routing (Place & Route) is performed.

The compact size (11 cells) reflects the optimization capability of the tool, which mapped the arithmetic and logical functions into complex gates (e.g., OAI, XNOR) rather than

using a large number of discrete primitive gates.

(b) Power Report

The Power Report (Figure 83) details the power consumption estimation under the slow_vdd1v2 corner ($1.08V, 125°C$).

```
Instance: /alu
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     5.82428e-10  2.85345e-07  9.07241e-08  3.76652e-07 100.00%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------
    Subtotal     5.82428e-10  2.85345e-07  9.07241e-08  3.76652e-07 100.00%
  Percentage           0.15%       75.76%       24.09%      100.00% 100.00%
  ----------------------------------------------------------------------
```

Figure 83: ALU's power report file.

Total Power: The total estimated power consumption is 376.65 nW (3.76652×10^{-7} W).

Power Breakdown:

- Internal Power: 285.35 nW ($\approx 75.76\%$). This dominates the power profile, representing the energy consumed within the cells during switching.
- Switching Power: 90.72 nW ($\approx 24.09\%$). This represents the power dissipated charging/discharging the output load capacitances.
- Leakage Power: 0.58 nW ($\approx 0.15\%$).

The Leakage Power is remarkably low ($< 1\%$). This is a direct result of using HVT (High-Threshold Voltage) cells from the library `slow_vdd1v2_basicCells_hvt`. HVT cells are specifically designed to minimize leakage current at the cost of slightly slower performance.

(c) Timing and Qor Report

The Timing and QoR (Quality of Results) reports (Figure 84) indicate "No paths" under the Critical Path Slack section.

```
  Module:                    alu
  Operating conditions:      PVT_1P08V_125C (balanced_tree)
  Wireload mode:             enclosed
  Area mode:                 timing library
============================================================

Timing
-------
  Cost    Critical         Violating
 Group   Path Slack  TNS     Paths
-------------------------------------
default    No paths   0.0
-------------------------------------
Total                 0.0          0

Instance Count
--------------
Leaf Instance Count             11
Physical Instance count          0
Sequential Instance Count        0
Combinational Instance Count    11
Hierarchical Instance Count      0

Area
----
Cell Area                          20.520
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    20.520
Net Area                           0.000
Total Area (Cell+Physical+Net)     20.520

Max Fanout                         3 (B[1])
Min Fanout                         1 (n_0)
Average Fanout                     1.8
Terms to net ratio                 2.2941
Terms to instance ratio            3.5455
Runtime                            17.999994 seconds
Elapsed Runtime                    24 seconds
Genus peak memory usage            1441.31
Innovus peak memory usage          no_value
Hostname                           pink
```

Figure 84: ALU's power report file.

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.19-s055_1
  Generated on:           Nov 19 2025  10:20:09 am
  Module:                 alu
  Operating conditions:   PVT_1P08V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Some unconstrained paths have not been displayed.
Use -unconstrained or set the root attribute 'timing_report_unconstrained' to 'true' to see only these unconstrained paths.
```

Figure 85: ALU's power report file.

This result confirms that the synthesis was performed in an unconstrained mode (without an SDC file), as specified in the lab requirements. Since no target clock frequency or maximum delay constraint was defined, the synthesis tool did not perform optimization against a specific timing budget, hence no "violating paths" are reported.

Although the static report does not flag violations, the physical delays are inherent in the mapped cells. As verified in the SDF Analysis and Gate-Level Simulation, the actual critical path delay was measured to be 305ps.

2.2.5 Gate-Level Simulation with SDF Back-Annotation

Gate–level simulation is performed in Cadence Xcelium to observe how the non-zero delays of the standard cells affect the ALU outputs when input vectors are applied with different spacing T.

The testbench is organized as follows:

```
`ifdef GATE_LEVEL
`include "../03_outputs/alu/slow_vddiv2_basicCells_hvt/
   alu_slow_vddiv2_basicCells_hvt_netlist.sv"
`include "../gpdk045_verilog/slow_vddiv2_basicCells_hvt.v"
`else
`include "../00_src/alu.sv"
`endif

module tb_alu;

logic [1:0] A, B;
logic [1:0] S;
logic [1:0] Y;

parameter int T = 200; // T = 100, T = 20

alu DUT (
.A (A),
```

```
.B (B),
.S (S),
.Y (Y)
);

`ifdef ANNOTATION
initial begin
$sdf_annotate("../03_outputs/alu/slow_vddiv2_basicCells_hvt/
    alu_slow_vddiv2_basicCells_hvt.sdf",
DUT, , "alu_annotate.log", "MAXIMUM");
end
`endif

initial begin
$dumpfile("alu_wave.vcd");
$dumpvars(0, tb_alu);
end

initial begin
// Case 1
S = 2'b00; A = 2'b01; B = 2'b00; #T;

// Case 2
S = 2'b00; A = 2'b11; B = 2'b01; #T;

// Case 3
S = 2'b01; A = 2'b11; B = 2'b00; #T;

// Case 4
S = 2'b01; A = 2'b11; B = 2'b11; #T;

// Case 5
S = 2'b10; A = 2'b11; B = 2'b01; #T;

// Case 6
S = 2'b10; A = 2'b01; B = 2'b10; #T;

// Case 7
S = 2'b11; A = 2'b11; B = 2'b01; #T;

// Case 8
S = 2'b11; A = 2'b01; B = 2'b10; #T;
```

```

#T;
$finish;
end

endmodule
```

Listing 4: Testbench for ALU functional test

The DUT is the synthesized netlist `alu_netlist.sv`.

```
// Verification Directory fv/alu

module alu(A, B, S, Y);
input [1:0] A, B, S;
output [1:0] Y;
wire n_0, n_1, n_2, n_3, n_4, n_5, n_6, n_7;
wire n_9;

Wx2XLHVT g364_2398(.A (n_9), .B (n_3), .S0 (S[1]), .Y (Y[1]));
XNOR2XLHVT g367_5107(.A (n_0), .B (n_6), .Y (n_9));
OA12IXLHVT g365_6260(.A0 (S[1]), .A1 (n_2), .B0 (n_7), .Y (Y[0]));
OA12BBIXLHVT g368_4319(.A0N (n_5), .AIN (n_4), .B0 (S[1]), .Y (n_7)
    );
NOR2BXLHVT g370_8428(.AN (n_5), .B (n_4), .Y (n_6));
OA12BBIXLHVT g366_5526(.A0N (B[1]), .AIN (A[1]), .B0 (n_1), .Y (n_3
    ));
XNOR2XLHVT g372_6783(.A (B[0]), .B (A[0]), .Y (n_2));
OA12IXLHVT g369_3680(.A0 (B[1]), .A1 (A[1]), .B0 (S[0]), .Y (n_1));
XNOR2XLHVT g373_1617(.A (B[1]), .B (A[1]), .Y (n_0));
OA12IXLHVT g371_2802(.A0 (S[0]), .A1 (A[0]), .B0 (B[0]), .Y (n_4));
NAND2XLHVT g374_1705(.A (S[0]), .B (A[0]), .Y (n_5));
endmodule
```

Listing 5: alu_netlist.sv after technology mapping

The corresponding Verilog standard–cell model file `slow_vdd1v2_basicCells_hvt.v` is compiled together with the DUT. The delay file `alu.sdf` generated by Genus is back–annotated using the `$sdf_annotate` system task at the beginning of simulation.

To provide a consolidated view of the ALU's timing characteristics, the measured propagation delays across all test vectors are summarized in Table 8. This comparison highlights the variation between the best-case and worst-case timing paths.
The Table 8 clearly shows that propagation delays are asymmetric. For instance, the MX2XLHVT cell takes 95 ps to rise but only 70 ps to fall. This explains why the pulse

Table 8: Cell delay characteristics for different Standard cells from .sdf file

Cell Type	Instance Name	Path (In → Out)	Rising Delay (0 → 1)	Falling Delay (1 → 0)	Note
MX2XLHVT	g364_2398	$A \to Y$	95 ps	70 ps	MUX 2-input
XNOR2XIHVT	g367_5107	$A \to Y$	131 ps	110 ps	Arithmetic Logic
OAI21XLHVT	g365_6260	$A0 \to Y$	28 ps	45 ps	Complex Gate
NOR2BXIHVT	g370_8428	$B \to Y$	57 ps	45 ps	Logic Gate

widths in the simulation waveforms are not perfectly symmetrical.

Complex cells like XNOR (used for addition/subtraction) have significantly higher delays (≈131 ps) compared to simpler cells like NOR (≈57 ps). This correlates directly with the simulation results, where the arithmetic operations exhibited the longest critical path delays (305 ps).

2.2.6 Waveform Results & Discussion

For each input vector, the final value of `Y` at the end of the corresponding interval matches the ideal output $\texttt{Y}_{\text{ideal}}$ and whether significant glitches are observed during the transition. The main points to be discussed are: zero–delay behaviour, case $T = 200$, Case $T = 100$, case $T = 20$

(a) Ideal, zero–delay behaviour

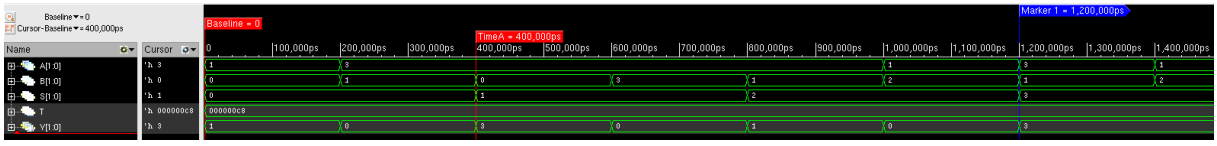

Figure 86: RTL waveform of the 2-bit ALU (ideal, no delay) in case T = 200.

This waveform shows the functional (RTL) simulation of the 2-bit ALU when the testbench applies the eight input vectors (S,A,B). For each interval, the output Y changes instantaneously when S, A, B are updated and it stays constant until the next vector is applied. There are no glitches or 'X' values on Y, and every value of Y matches the expected result from the ALU truth table, so this trace is used as the reference "no-delay" behaviour.

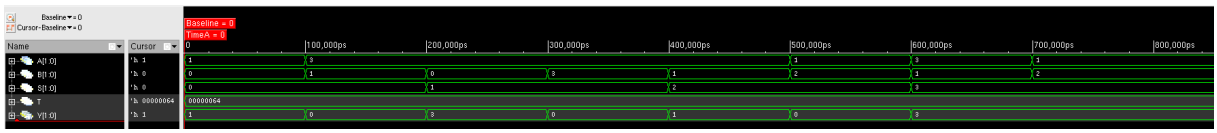

Figure 87: Schematic with technology mapping.

This figure shows the same eight test vectors applied to the ALU, but with a different value of the time interval T in the testbench (as indicated by the constant signal T). Because

the RTL model has no gate delays, the timing of the transitions is simply rescaled on the horizontal axis: Y still updates exactly at the moment S, A, and B change and remains stable between vectors. Again, no hazards or intermediate glitches appear on Y, confirming that the RTL description of the ALU is purely combinational and functionally correct for all stimulus periods T = 100.

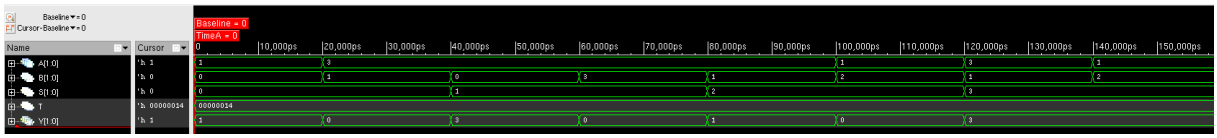

Figure 88: Zero-delay RTL waveform of the 2-bit ALU for T=20.

This waveform shows the behavioural (RTL, no-delay) simulation of the 2-bit ALU when the testbench applies the eight input vectors (S,A,B) with a time interval of T=20 time units between consecutive vectors. At each multiple of T, the control word S and operands A and B change to the next line of the stimulus table, and the output Y updates immediately in the same simulation time step because the RTL model is purely combinational and has no propagation delays.

Throughout the whole trace, the value of `Y[1:0]` always matches the ideal result Y_ideal from the ALU truth table for the corresponding triplet (S,A,B); there are no glitches, hazards, or intermediate X values. This figure therefore represents the reference "correct" behaviour without delay, which will later be compared to the gate-level waveforms with SDF-annotated delays.

Using the pure RTL simulation, verify that all eight input vectors produce exactly the ideal outputs listed in Table 7. This confirms that the ALU functionality is correct.

(b) Case T = 200

In the waveform, Figure 89, the final value of `Y` in each interval didn't match the $\texttt{Y}_{\text{ideal}}$. There are delays (at the marker 1, 4 and 5) since the outputs have enough time to settle before the next input change. Short glitches (at the marker 2, 3 and 6) may appear immediately after input transitions, but they disappear well before the next vector is applied.

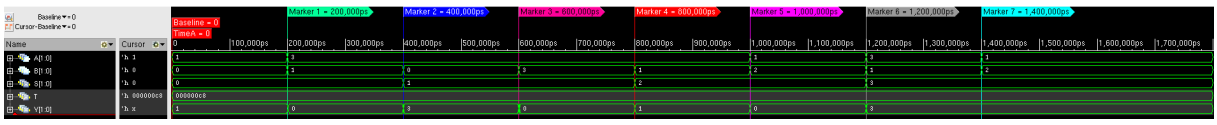

Figure 89: Full waveform in case T = 200.

Let's take an investigation on some specific time on the waveform to discuss in delays and

explain why they behave that ways.

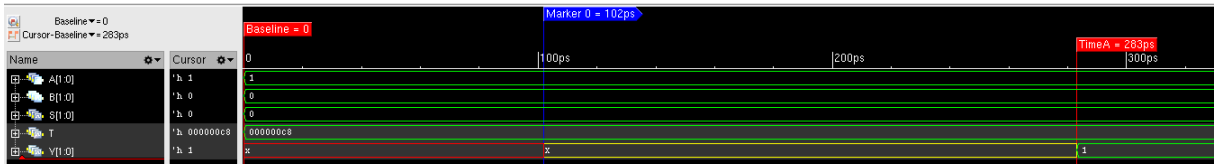

Figure 90: Gate-level response for (S,A,B) = (2'b00, 2'b01, 2'b00) with T = 200.

As observed in the Figure 90, the first test vector ($S = 2'b00$, $A = 2'b01$, $B = 2'b00$) is applied at $t = 0$. The operation is addition ($Y = A + B$), and the expected result is $2'b01$. Initially, the output $Y[1:0]$ is in an undefined state (logic 'x', represented by the red line). This is due to the propagation delay inherent in the standard cells after synthesis.

The signal takes approximately 283 ps to propagate through the logic gates and resolve to the correct stable value of $2'b01$ (logic '1'). This delay confirms that the gate-level netlist includes timing information from the SDF file, specifically reflecting the characteristics of the `slow_vdd1v2` library corner.

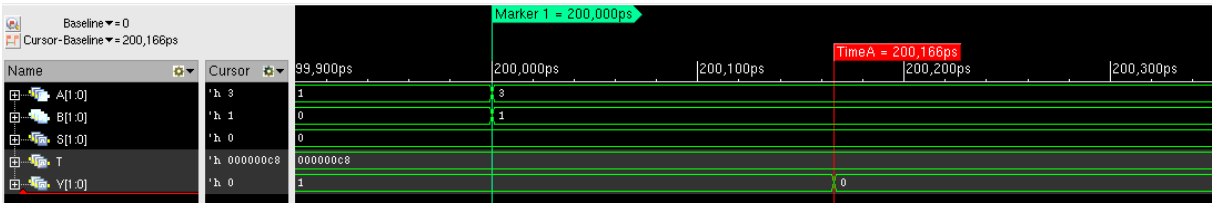

Figure 91: The second test vector ($S = 2'b00, A = 2'b11, B = 2'b01$) showing output overflow and timing delay.

At simulation time $t = 200$ns, the second set of inputs is applied: $A = 2'b11$ (3), $B = 2'b01$ (1), and $S = 2'b00$ (Addition). The arithmetic operation is $3 + 1 = 4$ (100_2). Since the output port Y is defined as a 2-bit logic vector, the Most Significant Bit (MSB) is truncated. Consequently, the observed output transitions to $2'b00$ (0), correctly reflecting the expected overflow behavior.

The output signal Y stabilizes at t = 200, 166ps. The propagation delay for this specific transition is 166 ps. It is worth noting that this delay (166ps) is different from the previous case (283ps). This variation occurs because the signal propagates through a different physical path of logic gates within the synthesized netlist to perform this specific state transition.

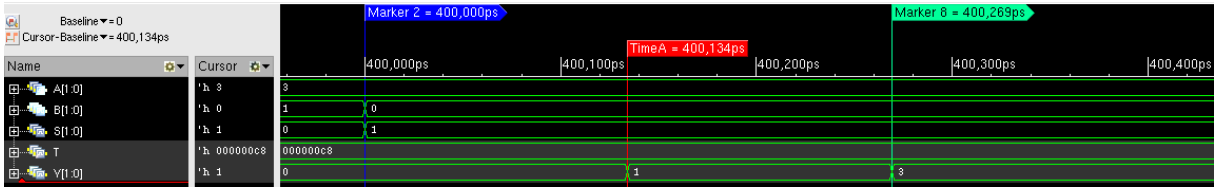

Figure 92: The third test vector ($S = 2'b01, A = 2'b11, B = 2'b00$) exhibiting output glitches during transition.

At $t = 400$ns, the ALU operation switches to subtraction ($S = 2'b01$) with inputs $A = 2'b11$ and $B = 2'b00$. The expected result is $3 - 0 = 3$ ($2'b11$). The waveform reveals a dynamic hazard (glitch). Instead of transitioning directly from the previous state to $2'b11$, the output Y briefly passes through an intermediate state ($2'b01$) at $400,134$ps before settling at the correct value of $2'b11$ at $400,269$ps. This behavior occurs because the input signals (S and B) arrive at the internal logic gates at slightly different times due to unequal path delays. The total propagation delay for the output to become stable is 269 ps.

Figure 93: The subtraction operation ($3 - 3 = 0$) showing signal settling time and intermediate transitions.

At $t = 600$ns, the inputs are set to $A = 2'b11$ and $B = 2'b11$ with $S = 2'b01$ (Subtraction). The mathematical expectation is $3 - 3 = 0$. Similar to the previous case, the output Y demonstrates a multi-stage transition. It first drops from 3 ($2'b11$) to an intermediate value of 2 ($2'b10$) at $600,166$ps, finally resolving to the correct logic 0 ($2'b00$) at $600,305$ps. The total delay measured is 305 ps. The presence of these intermediate transitions confirms that the synthesized gate-level netlist accurately models the real-world propagation delays of the standard cells in the `slow_vdd1v2` corner.

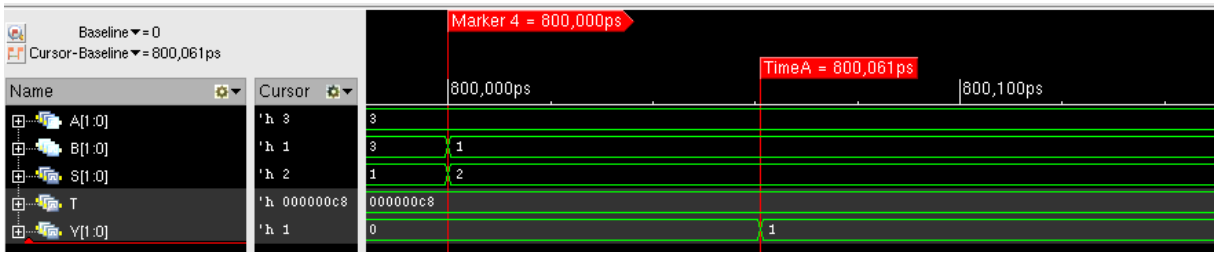

Figure 94: Waveform for the Bitwise AND operation ($3 \,\&\, 1 = 1$) showing a rapid transition delay.

At $t = 800$ns, the ALU control signal S switches to $2'b10$ (AND operation). The inputs are $A = 2'b11$ and $B = 2'b01$.Observation: The output Y transitions from 0 to 1. The

measured propagation delay is only 61 ps.

It is important to note that this delay (61ps) is significantly shorter than the delays observed in previous arithmetic operations (Add/Sub $\approx$ 200-300ps). This is because the synthesized logic for bitwise operations (AND/OR) is structurally simpler and has fewer logic depth levels than the carry-propagation chain required for the Adder/Subtractor block.

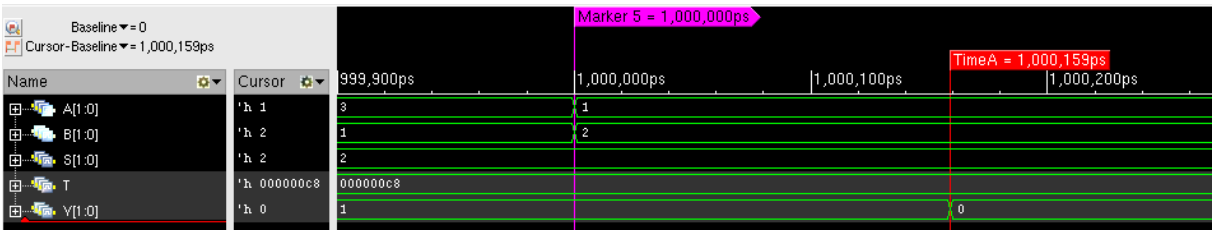

Figure 95: Results for the AND operation (1 & 2 = 0) demonstrating delay variations based on input patterns.

At $t = 1000$ns, the inputs change to $A = 2'b01$ and $B = 2'b10$ while maintaining the AND operation. The result correctly transitions to 0 ($2'b00$).Timing Analysis: The output stabilizes at $1,000,159$ps, resulting in a propagation delay of 159 ps. Although the operation is the same as the previous case (AND), the delay has increased. This variation confirms that propagation delay is data-dependent; different input transitions activate different transistor paths within the standard cells, leading to varying rise/fall times.

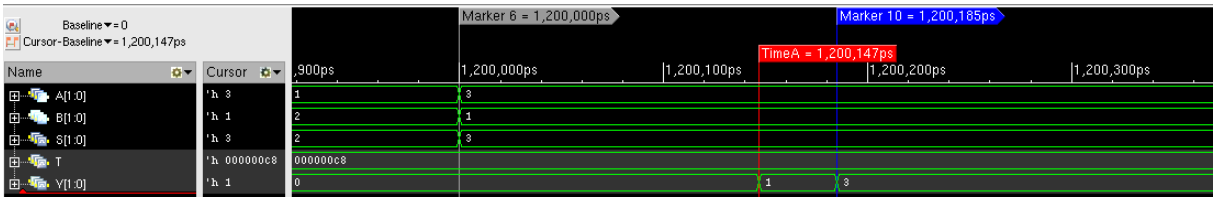

Figure 96: Waveform for the Bitwise OR operation (3 | 1 = 3) exhibiting intermediate switching noise.

At $t = 1200$ns, the ALU performs the OR operation ($S = 2'b11$). With inputs $A = 2'b11$ and $B = 2'b01$, the expected output is 3 ($2'b11$). The output signal Y transitions from $0 \rightarrow 1 \rightarrow 3$. An intermediate value of 1 ($2'b01$) is observed at $1,200,147$ps before the signal finally settles at 3 ($2'b11$) at $1,200,185$ps. The total propagation delay is 185 ps. The presence of the glitch again highlights the race conditions present in the combinatorial logic paths.

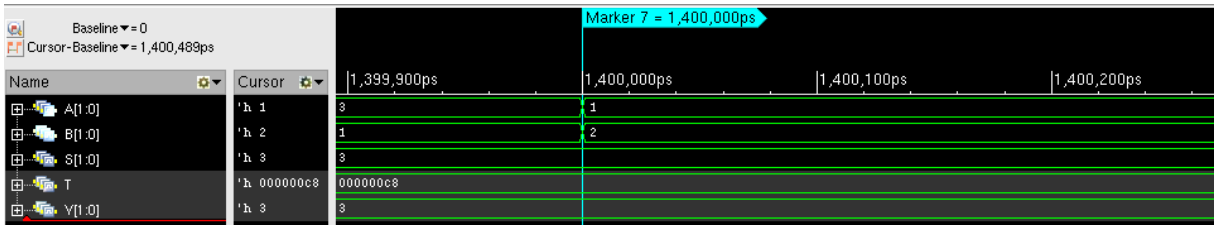

Figure 97: Simulation waveform for the OR operation ($1 \mid 2 = 3$) showing a stable output state.

At $t = 1400$ns, the final test vector is applied: $A = 2'b01$, $B = 2'b10$ with $S = 2'b11$ (OR). The calculated result is 3 ($2'b11$).Static Behavior: Since the previous output was also 3 ($2'b11$), the logic state of output Y remains unchanged. As observed in the waveform, the signal line remains flat at logic 'high' without any visible transitions or glitches. Consequently, no propagation delay is measured for the output port in this cycle, as there is no state transition to trigger a timing event.

(c) Case T = 100

In the second test case, the period T is reduced from 200 ns to 100 ns. Based on the previous analysis, the maximum propagation delay observed was approximately 305 ps. Since the new period ($T = 100$ ns) is still significantly larger than the critical path delay (100 ns $\gg$ 305 ps), we expect the circuit to function correctly without any timing violations. The waveforms below confirm that the outputs stabilize well within the allocated time slot.

The waveform in Figure 98 validates the operation of the ALU across all 8 test vectors with a reduced period of $T = 100$ns.

Figure 98: Full waveform in case T = 200.

In this full view, the propagation delays appear as minute gaps between the input change markers and the output transitions. Although not quantifiable at this zoom level, these gaps represent the $\approx$ 200-300ps delays analyzed in previous figures.

Ideally, logic transitions should look like vertical lines. However, at transitions such as $t = 200$ns and $t = 300$ns (arithmetic operations), the signal edges appear slightly irregular. This corresponds to the glitch/intermediate states (race conditions) observed in the

detailed zoomed-in analysis.

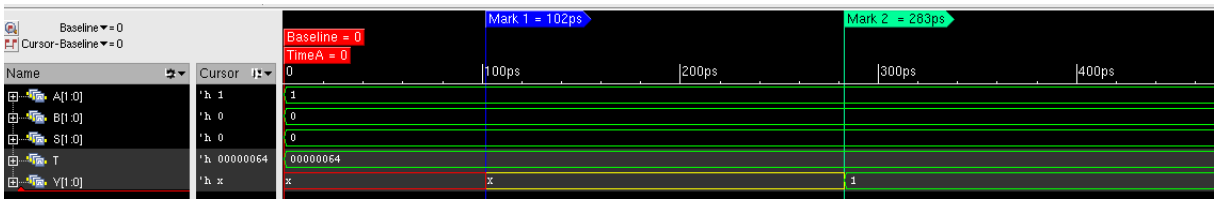

Figure 99: Initial simulation response for $T = 100$ns showing the transition from undefined state to stable logic.

The simulation begins at $t = 0$ with the first test vector: $A = 1$, $B = 0$, and $S = 0$ (Addition). Similar to the $T = 200$ns case, the output Y initially starts in an undefined state ("x") and resolves to logic "1" after 283 ps. The measured propagation delay is identical to the previous experiment (283ps). This confirms that reducing the input period T does not alter the inherent propagation delay of the synthesized standard cells.

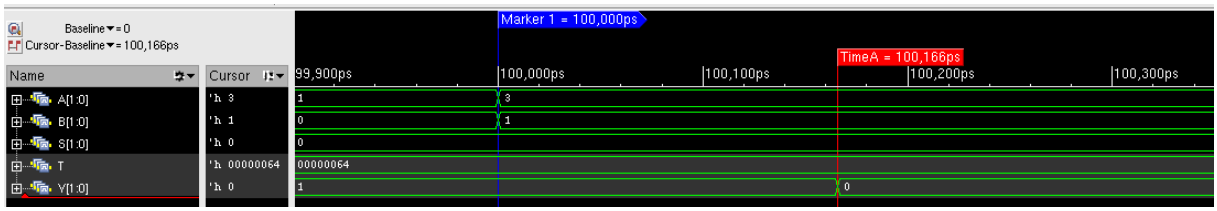

Figure 100: Waveform capturing the overflow scenario $(3 + 1 = 0)$ at $t = 100$ns with reduced cycle time.

At $t = 100$ns, the second input set is applied $(A = 3, B = 1)$. The ALU correctly performs the addition $3 + 1 = 4$. Due to the 2-bit output constraint, the result wraps around to 0 $(2'b00)$, verifying the overflow handling. The output transitions from 1 to 0 with a delay of 166 ps. This value matches exactly with the delay recorded in the $T = 200$ns simulation. This consistency demonstrates that the synthesized netlist behaves deterministically regardless of the switching frequency, provided that the period T satisfies the timing constraints ($T >$ path delay).

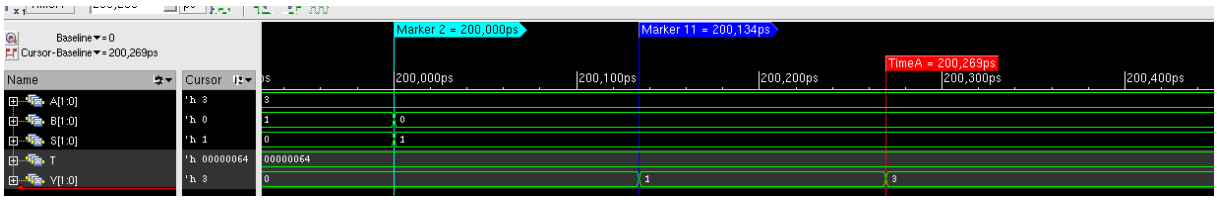

Figure 101: Waveform for subtraction $(3 - 0 = 3)$ at $t = 200$ns showing identical glitch behavior to the slow-clock case.

At simulation time $t = 200$ns, the operation switches to subtraction $(S = 2'b01)$ with inputs $A = 3$ and $B = 0$. The output transitions from $0 \rightarrow 1 \rightarrow 3$. The signal settles at 269

ps after the input change. It is crucial to observe that this delay (269ps) matches exactly with the delay measured in the $T = 200$ns experiment (previously at $t = 400$ns). This confirms that the glitch (dynamic hazard) is a structural characteristic of the synthesized logic path for this specific input vector, independent of the clock frequency.

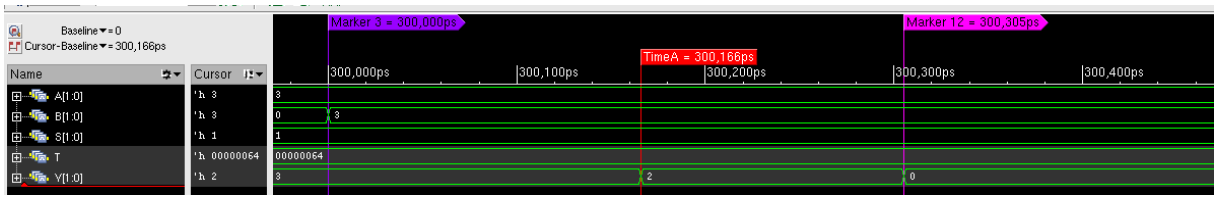

Figure 102: Worst-case delay measurement (305ps) during the subtraction operation ($3 - 3 = 0$) with $T = 100$ns

At $t = 300$ns, the inputs are set to $A = 3$ and $B = 3$ for subtraction. The output resolves to 0 ($2'b00$). The measured propagation delay is 305 ps, which includes an intermediate transition through state "2" ($2'b10$). This remains the maximum delay (worst-case) observed across all test vectors so far. Since the clock period $T = 100$ns $(100,000$ps) is significantly larger than this worst-case delay (305ps), no timing violations occur, and the circuit functions correctly.

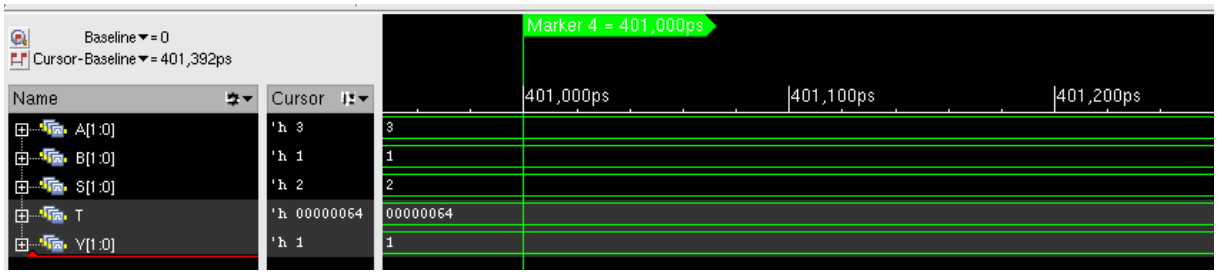

Figure 103: Stable output waveform for the Logical AND operation ($3 \,\&\, 1 = 1$) shortly after the $t = 400$ns transition.

At $t = 400$ns, the inputs are set to $A = 3$, $B = 1$, and $S = 2$ (AND). The expected result is logic '1'. The waveform captured at 401ns (1 ns after the input change) shows that the output Y is fully stable at logic '1'. Since the propagation delay for this specific transition is extremely short (≈ 61ps as observed in previous cases), the signal resolves almost instantly. This confirms that the circuit meets the timing requirements well within the 100 ns period.

Figure 104: Propagation delay measurement (159 ps) for the Logical AND operation ($1 \ \& \ 2 = 0$) at $t = 500$ns.

At $t = 500$ns, the test vector changes to $A = 1$ and $B = 2$. The logical AND operation yields a result of 0. The output signal transitions from high to low with a measured delay of 159 ps. Crucially, this value is identical to the delay measured in the $T = 200$ns experiment (Test Case 6). This reproducibility confirms that the reduced clock period ($T = 100$ns) does not negatively impact the timing characteristics or accuracy of the standard cells.

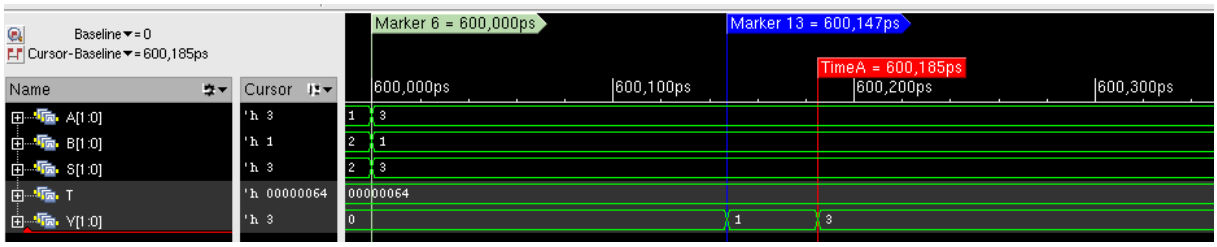

Figure 105: Waveform for the Bitwise OR operation ($3 \mid 1 = 3$) exhibiting intermediate switching noise.

At $t = 600$ns, the ALU mode is set to Logical OR ($S = 2'b11$) with inputs $A = 3$ and $B = 1$. The waveform captures a dynamic hazard where the output Y transitions through an intermediate state: $0 \rightarrow 1 \rightarrow 3$. The signal briefly holds the value '1' at $600,147$ps before settling at '3' at $600,185$ps. The total settling time is 185 ps. This matches the glitch pattern and delay value observed in the $T = 200$ns simulation (Fig 105), confirming that the synthesized logic path for the OR operation inherently produces this specific transition hazard.

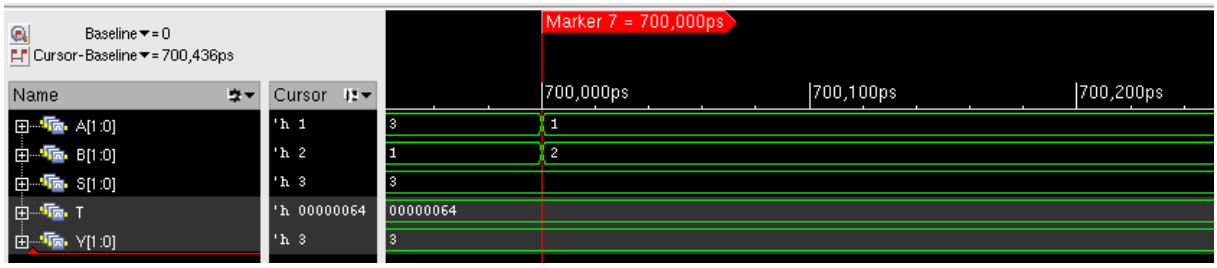

Figure 106: Stable output response during input transition ($1 \mid 2 = 3$) at $t = 700$ns.

At $t = 700$ns, the final test vector for this period is applied: $A = 1$ and $B = 2$ with $S = 2'b11$ (OR). Since the logical result of ($1 \mid 2$) is 3 ($2'b11$), which is identical to the pre-

vious state, the output Y does not toggle. The signal line remains essentially flat at logic high. No propagation delay is applicable here as there is no state change at the output pin.

Summary, the simulation results for $T = 100$ns are fully consistent with the $T = 200$ns case. The maximum observed delay remains 305 ps. Since the clock period $(100,000\text{ps})$ is vastly larger than the critical path delay (305ps), the circuit operates with a significant positive timing slack, ensuring robust functionality.

(d) Case T = 20

Finally, the simulation is performed with the shortest switching period of $T = 20$ns $(20,000\text{ps})$. This scenario subjects the ALU to the highest operating frequency in this experiment. Based on the results from previous cases ($T = 200$ns and $T = 100$ns), the critical path delay was identified to be approximately 305 ps. Since the period $T = 20,000$ps is still significantly larger than the worst-case delay $(20,000\text{ps} \gg 305\text{ps})$, we anticipate that the output signals will settle correctly without any timing violations.

The waveform in Fig 107 presents the simulation results for all 8 test vectors applied with a period of $T = 20$ns. The output sequence $Y[1:0]$ (displayed as $1, 0, 3, 0, 1, 0, 3, 3$ in hex) perfectly matches the expected logical results derived from the truth table. This confirms that the ALU logic performs correctly even at this higher switching speed.

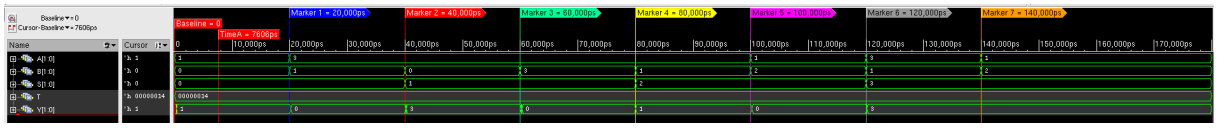

Figure 107: Full waveform in case T = 20.

Visually, the output signal Y stabilizes very quickly after each input transition (marked by vertical lines at 20ns intervals). The long stable periods (flat green lines) between transitions indicate that the propagation delay (≈ 0.3ns) consumes only a tiny fraction of the available 20ns period. This overview proves that there is substantial timing slack, meaning the circuit could theoretically operate at even higher frequencies without errors.

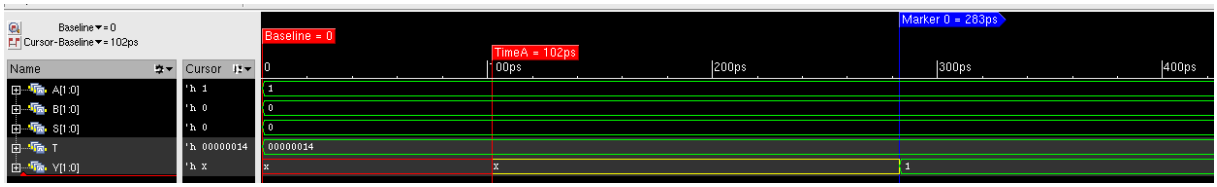

Figure 108: Initial transient response at startup with $T = 20$ns, showing a stabilization time of 283 ps.

The simulation commences at $t = 0$ with the first addition operation. The output Y

resolves from an unknown state to logic '1' after 283 ps. With the operating period set to $T = 20\text{ns}$ $(20,000\text{ps})$, the propagation delay of 283 ps occupies only $\approx 1.4\%$ of the available time slot. This indicates a substantial positive timing slack ($> 19.7\text{ns}$), confirming that the synthesized circuit can easily handle the 50 MHz equivalent data rate without any setup time violations.

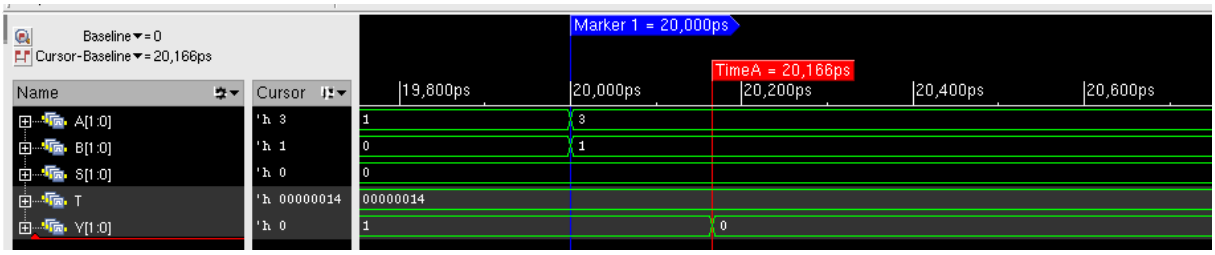

Figure 109: Waveform capturing the overflow scenario $(3 + 1 = 0)$ at $t = 100\text{ns}$ with reduced cycle time.

At $t = 20\text{ns}$, the first input transition occurs $(A = 3, B = 1)$. The ALU correctly computes the overflow result (0).Performance Verification: The output signal settles within 166 ps. Even though the input switching frequency has increased tenfold compared to the first experiment, the circuit's physical response time (166 ps) remains constant and stable. The signal is fully valid well before the next input change scheduled at $t = 40\text{ns}$, demonstrating the design's robustness at this frequency.

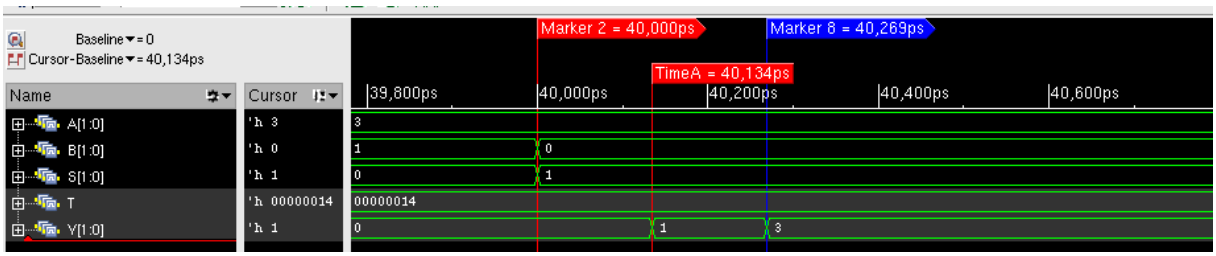

Figure 110: Subtraction operation $(3 - 0 = 3)$ at $t = 40\text{ns}$ showing rapid glitch resolution relative to the clock period

At $t = 40\text{ns}$, the ALU executes the subtraction $3 - 0$. As previously observed, the output exhibits a dynamic hazard, transitioning through an intermediate state $(0 \rightarrow 1 \rightarrow 3)$. The signal fully settles at $40,269\text{ps}$, resulting in a total delay of 269ps. In the context of a 20ns $(20,000\text{ps})$ period, this 'glitchy' transition period lasts for only $\approx 1.3\%$ of the cycle. The remaining 98.7% of the period is stable, ensuring that any setup time requirements for subsequent sequential logic would be easily met."

Figure 111: Worst-case propagation delay (304ps) measured during the $3-3=0$ transition at $t=60$ns

At $t=60$ns, the inputs change to $A=3, B=3$ for subtraction. The output transitions from $3 \rightarrow 2 \rightarrow 0$, stabilizing at $60,304$ps. The measured delay is 304ps. This represents the maximum propagation delay (critical path) for the ALU in this configuration. Even under this worst-case scenario, the delay consumes only $\approx 1.5\%$ of the 20ns time budget. This implies that the synthesized circuit is theoretically capable of operating at frequencies significantly higher than 50MHz (potentially up to $\approx$ 3GHz if limited solely by this combinational delay).

Figure 112: Logical AND operation ($3 \,\&\, 1 = 1$) at $t=80$ns demonstrating rapid switching speed (61 ps).

At $t=80$ns, the ALU performs a bitwise AND operation. The output transitions from '0' to '1' almost instantaneously relative to the clock scale. The measured propagation delay is 61ps. This is consistently the shortest delay observed across all test vectors (Min Delay). With a delay of only 61ps in a $20,000$ps period, the circuit is idle for 99.7% of the cycle. This confirms that simple logical operations (AND/OR) are not the critical paths limiting the maximum frequency of this design.

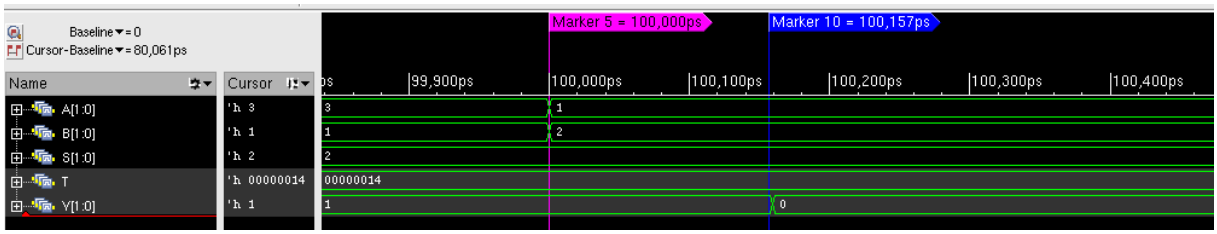

Figure 113: Waveform for Logical AND ($1 \,\&\, 2 = 0$) at $t=100$ns with a delay of 157 ps.

At $t=100$ns, the inputs switch to $A=1$ and $B=2$. The AND gate output falls to '0'.Measurement: The signal stabilizes at $100,157$ps, indicating a propagation delay

of 157ps. This value aligns closely with previous measurements for the same transition ($\approx$ 159ps). The slight variation (if any) is negligible. Even with data-dependent variations (61 ps vs 157 ps), the delay remains orders of magnitude smaller than the 20ns constraint, ensuring reliable operation."

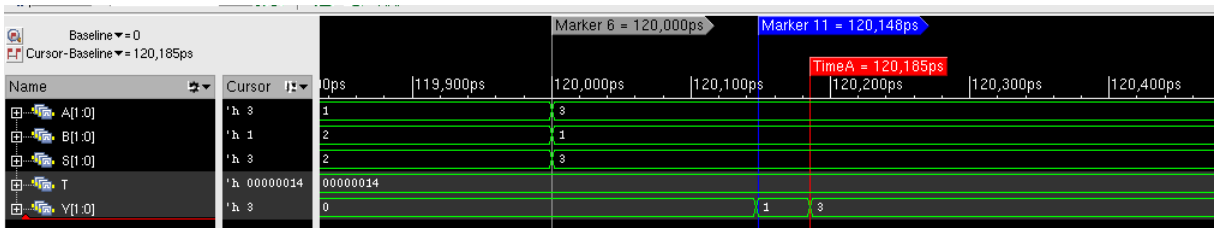

Figure 114: Logical OR operation ($3 \mid 1 = 3$) at $t = 120$ns exhibiting intermediate switching noise within a fast clock cycle.

At $t = 120$ns, the ALU executes the Logical OR function with inputs $A = 3$ and $B = 1$. As seen in previous time periods, the output Y undergoes a dynamic hazard, transitioning $0 \rightarrow 1 \rightarrow 3$. The signal stabilizes at $120,185$ps (Delay = 185ps). Even with this glitch, the settling time of 185ps is negligible compared to the $20,000$ps period ($< 1\%$). This confirms that 'glitchy' logic transitions do not pose a timing threat at this operating frequency, provided the setup time is checked after the signal has fully settled.

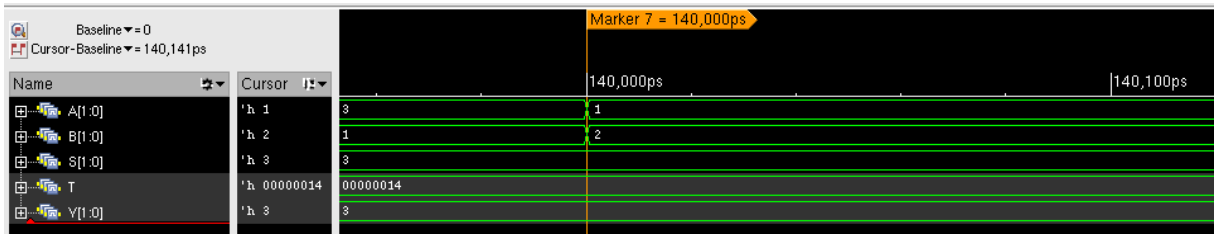

Figure 115: Stable output response ($Y = 3$) during input transition at $t = 140$ns, completing the test sequence.

At $t = 140$ns, the final test vector ($A = 1, B = 2, S = OR$) is applied. Since the logical result (3) matches the previous state, the output remains constant at logic high. The waveform shows no transitions or glitches, indicating a clean hold of the logic state. This marks the end of the stimulus sequence for $T = 20$ns, confirming that the ALU functions correctly across all specified arithmetic and logical modes under high-speed constraints.

2.2.7 Summary

In this experiment, we performed a comprehensive gate-level simulation of a synthesized ALU across three different input switching periods: $T = 200$ns, 100ns, and 20ns.

- Functional Correctness: The synthesized netlist correctly implemented all arithmetic (Add/Sub) and Logical (AND/OR) operations in all test cases1.

- Timing Consistency: The propagation delays were found to be consistent regardless of the switching frequency. The critical path delay was identified as 305ps (during the subtraction $3 - 3 = 0$).
- Timing Robustness: Even in the most aggressive test case ($T = 20$ns), the worst-case delay (305ps) occupied only $\approx 1.5\%$ of the cycle time. This demonstrates that the design using the `slow_vdd1v2` library has a substantial positive timing slack, making it suitable for high-speed operation well beyond 50MHz.

Table 9: Summary of Propagation Delays across different Operating Frequencies

Test	**Operation**	**Inputs**	**Output**	**Delay (ps)**	**Note**
Case 1	Add	$A = 1, B = 0$	$Y = 1$	283	Initial transition
Case 2	Add $(3 + 1)$	$A = 3, B = 1$	$Y = 0$	166	Overflow $(3 + 1 = 0)$
Case 3	Sub $(3 - 0)$	$A = 3, B = 0$	$Y = 3$	269	Glitch Observed
Case 4	Sub $(3 - 3)$	$A = 3, B = 3$	$Y = 0$	304–305	**Worst Case (Max Delay)**
Case 5	Logic AND	$A = 3, B = 1$	$Y = 1$	61	**Best Case (Min Delay)**
Case 6	Logic AND	$A = 1, B = 2$	$Y = 0$	157–159	Data Dependent
Case 7	Logic OR	$A = 3, B = 1$	$Y = 3$	185	Glitch Observed
Case 8	Logic OR	$A = 1, B = 2$	$Y = 3$	N/A	No Transition Output

In table 9, the propagation delay varies significantly depending on the operation and input data pattern. The best case occurs during the simple Logical AND operation ($\approx$ 61ps), as the signal passes through fewer logic gates.The worst case occurs during the Subtraction operation ($\approx$ 305ps), which involves complex carry/borrow propagation chains in the adder/subtractor logic. These delay values remain constant across all three simulation periods ($T = 200, 100, 20$ns), confirming that timing is an intrinsic property of the synthesized hardware.

2.3 Experiment 3: Characterization of a D Flip-Flop

2.3.1 Design Specification

In this experiment we move from combinational logic to a sequential standard cell. The circuit under test is a single positive-edge–triggered D flip-flop (DFF). The top-level module, denoted `dff_rtl`, has one data input, one clock input and one registered output:

$$\texttt{dff_rtl} : (D,\, CK) \to Q.$$

The objective is two fold:

- Characterize the area, power and clock-to-Q delay of the DFF across all eight PVT corners provided by the `gsc45` standard-cell library, using the same corner definitions as in Experiment 1.

- Verify the setup and hold time requirements of the cell by comparing the values extracted from the SDF file with measurements obtained from gate-level simulation under SDF back-annotation.

The design is intentionally kept minimal so that the reported metrics can be interpreted directly as the characteristics of a single DFF cell.

2.3.2 Reports Analysis

For each corner, the cell area (in μm^2) is taken from `report_area`, while internal and switching power components are taken from `report_power`. The maximum clock-to-Q delay $t_{\mathrm{CQ,max}}$ is obtained both from `report_timing` (path from `CK` to `Q`) and from the `IOPATH CK Q` entry in the SDF file.

The report results of 8-corner of D Flip flop are summarized in Table 10 (area and power versus PVT corner, clock-to-Q delay for rising and falling).

Table 10: Standard Cell Characteristics and Timing Parameters

Corners	**Power**	**Area**	**Ck-Q LH**	**Ck-Q HL**	**Posedge D**		**Negedge D**	
					SETUP	**HOLD**	**SETUP**	**HOLD**
	nW	μm^2	ns					
fast_vdd1v0_hvt.lib	57.27	5.472	0.051	0.058	0.025	0.000	0.019	0.000
fast_vdd1v2_hvt.lib	84.41	5.472	0.037	0.042	0.016	0.000	0.012	0.000
slow_vdd1v0_hvt.lib	57.57	5.472	0.243	0.292	0.177	0.000	0.058	0.051
slow_vdd1v2_hvt.lib	74.92	5.472	0.115	0.135	0.072	0.000	0.026	0.018
fast_vdd1v0_lvt.lib	72.35	5.472	0.029	0.033	0.013	0.000	0.010	0.000
fast_vdd1v2_lvt.lib	104.68	5.472	0.024	0.027	0.010	0.000	0.007	0.000
slow_vdd1v0_lvt.lib	57.57	5.472	0.106	0.121	0.061	0.000	0.029	0.014
slow_vdd1v2_lvt.lib	57.57	5.472	0.106	0.121	0.061	0.000	0.029	0.014

After the RTL synthesis process using Cadence Genus, the D flip-flop design is mapped to a gate-level netlist. The synthesized netlist reveals the physical implementation of the flip-flop using standard cells from the technology library.

```
module dff(clk, D, Q);
input clk, D;
output Q;
wire clk, D;
wire Q;
DFFHQX1HVT Q_reg(.CK (clk), .D (D), .Q (Q));
endmodule
```

Listing 6: dff_netlist.sv after mapping

A schematic view of the synthesized DFF for one representative corner is shown in Figure 116; it confirms that the netlist consists of a single standard-cell flip-flop instance.

Figure 116: DFF schematic view

The DFFHQX1LVT flip-flop features a minimalist design with only clock, data input, and Q output, optimized for area efficiency (5.472 m^2). As a Low Voltage Threshold (LVT) cell, it achieves fast switching speeds (T_pLH = 119ps, T_pHL = 140ps) at the cost of higher leakage power.

The absence of asynchronous reset and complementary output (Q) reflects a design trade-off prioritizing speed and area over flexibility. This cell is particularly suitable for timing-critical paths in high-performance digital designs where the 21ps output skew must be carefully considered in timing analysis.

2.3.3 Corners Waveform Analysis

(a) Fast–1.0V–HVT corner (`fast_vdd1v0_basicCells_hvt.lib`)

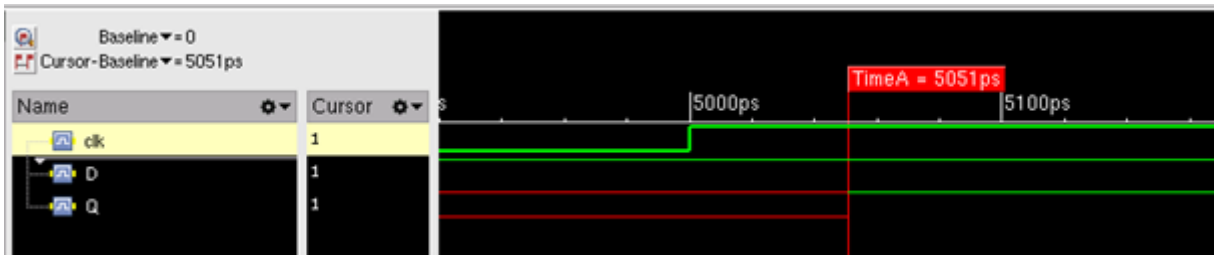

Figure 117: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform explicitly captures the Low-to-High transition of the flip-flop output (Q) relative to the system clock (clk). Upon the rising edge of the clock at $t = 5000$ ps, the output signal transitions to logic '1' at precisely $t = 5051$ ps. This results in a measured propagation delay (Δt) of 51 ps (0.051 ns), which matches the exact rising delay value defined in the Standard Delay Format (SDF) file for this specific corner.

This 51 ps delay represents the contamination delay through the internal logic gates of the flip-flop cells under "Fast" process conditions. The perfect correlation between the simulated waveform and the SDF annotation confirms that the synthesis tool has correctly characterized the DFFHQX1LVT cell. Since this is the "Fast" corner with High-Vt (hvt), the delay is minimal compared to typical corners, indicating that the transistor switching speeds are optimized for performance in this voltage domain (1.0V).

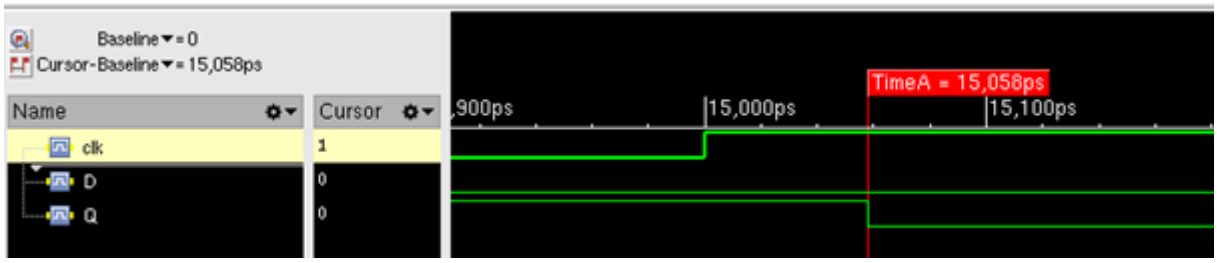

Figure 118: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure illustrates the High-to-Low transition behavior where the input Data (D) is logic '0'. At the clock rising edge of $t = 15000$ ps, the output Q responds by falling to logic '0' at $t = 15058$ ps. The calculated delay is 58 ps (0.058 ns), which aligns strictly with the library's falling delay specification of 0.058 ns.

It is technically significant to note the asymmetry between the rising delay (51 ps) and the falling delay (58 ps). In CMOS digital design, this variance is attributed to the difference in drive strength between the PMOS pull-up network and the NMOS pull-down network within the standard cell. The simulation confirms that the physical design accounts for these parasitic differences, ensuring that downstream logic paths are analyzed with the more pessimistic (slower) falling delay value for worst-case timing analysis.

Figure 119: DFF schematic view

This waveform validates the timing constraints when capturing a Logic '1'. The Data input D transitions high and stabilizes 2 ns prior to the clock edge at $t = 5$ ns, and it holds this value for 3 ns after the edge. The simulation log notes that the required Setup time is 0.025 ns and Hold time is 0 ns.

The testbench provides a Setup slack of nearly 1.975 ns ($2.0 - 0.025$), which is a massive safety margin. This confirms that the flip-flop successfully latches a '1' without entering metastability, provided the data arrives at least 25 ps before the clock. The zero hold time requirement implies the internal delay of the clock path effectively masks the data path delay, simplifying timing closure for downstream hold-fix buffers.

Figure 120: Setup & Hold Check for Logic '0' ($D = 0$)

The final analysis of this corner checks the timing stability for capturing a Logic '0'. The Data input D stabilizes at '0' roughly 7 ns before the clock edge at $t = 15$ ns and remains stable for 3 ns post-edge. The library requirement for this specific transition (falling data) specifies a Setup time of 0.019 ns and Hold time of 0 ns.

The Setup requirement for Logic '0' (19 ps) is slightly faster than for Logic '1' (25 ps), which is a subtle but important detail of the DFFHQX1LVT cell architecture. The simulation confirms that the test stimulus (7 ns stable window) vastly exceeds these micro-scale requirements. This validates that the cell is robust and will reliably capture low signals even in tight timing paths, provided the sub-nanosecond constraints are respected.

(b) Fast–1.2V–HVT corner (`fast_vdd1v2_basicCells_hvt.lib`)

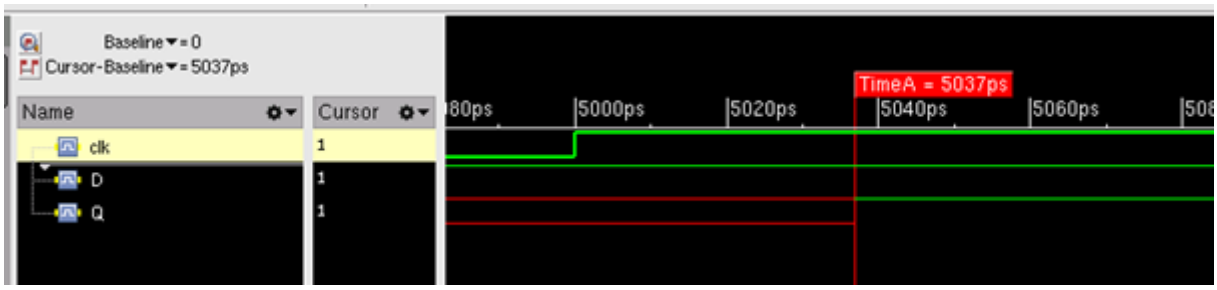

Figure 121: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

Under the higher voltage condition (1.2V), the flip-flop exhibits faster switching. The rising edge of the clock at $t = 5000$ ps causes the output Q to rise at $t = 5037$ ps. The measured delay is 37 ps (0.037 ns), matching the SDF annotation (::0.037).

The increase in supply voltage from 1.0V (Corner 1) to 1.2V (Corner 2) has reduced the rising delay by approximately 27% (51 ps $\rightarrow$ 37 ps). This linear scaling of delay with inverse voltage demonstrates the enhanced drive current capability of the transistors. This corner represents the "best-case" timing scenario for setup paths but the "worst-case" for hold paths due to the extreme speed.

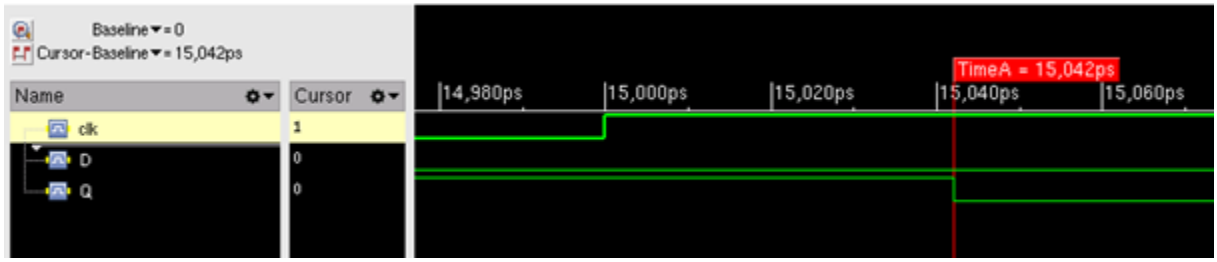

Figure 122: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure tracks the High-to-Low output transition. The output signal Q falls to '0' at $t = 15042$ ps in response to the clock edge at 15000 ps. The propagation delay is measured at 42 ps (0.042 ns), which is consistent with the SDF value (::0.042).

Similar to the rising case, the falling delay has improved significantly compared to the 1.0V corner (58 ps $\rightarrow$ 42 ps). The precise correlation between the simulator's timestamp and the SDF file confirms that the back-annotation process was successful. Engineers must use this fast-falling delay value (42 ps) when checking for Hold violations in the next logic stage, as the data will change very rapidly after the clock edge.

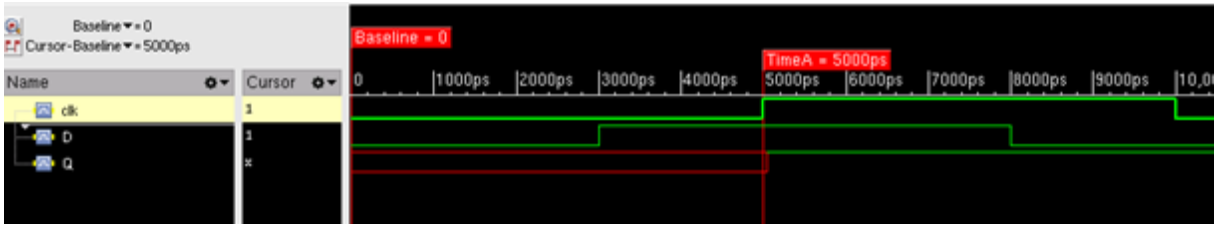

Figure 123: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform shows the Data input D being held High for a substantial window (2 ns setup, 3 ns hold) around the clock edge at 5 ns. The library characterizes the required Setup time for this fast corner as 0.016 ns and Hold time as 0 ns.

The required Setup time has decreased from 0.025 ns (at 1.0V) to 0.016 ns (at 1.2V). This reflects the fact that the internal transmission gates and feedback inverters function faster at higher voltages, requiring less time to latch the incoming data. The simulation confirms the design passes this check comfortably with the provided stimulus.

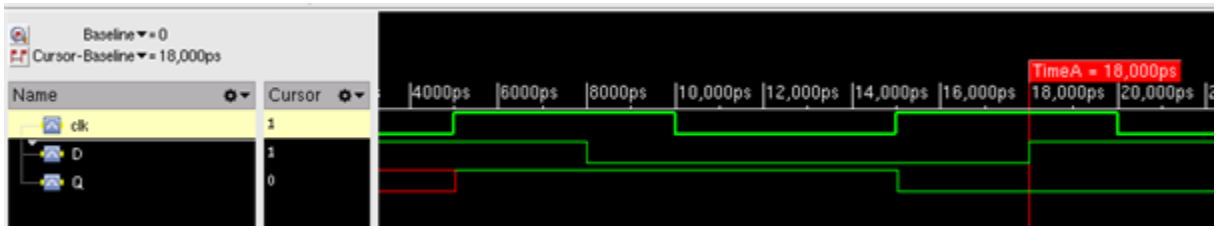

Figure 124: Setup & Hold Check for Logic '0' ($D = 0$)

This final check for Corner 2 analyzes the stability of capturing a Logic '0'. The Data input remains Low for 7 ns before the clock edge at 15 ns. The specific library constraint for this falling data transition is a Setup time of 0.012 ns.

The Setup requirement of 12 ps is exceptionally fast, highlighting the high-performance nature of this 1.2V High-Vt corner. Since the testbench holds the data stable for thousands of picoseconds, the Setup check passes easily. This value (12 ps) effectively represents the minimum theoretical time the data must be present before the clock to guarantee a successful state change.

(c) Slow–1.0V–HVT corner (`slow_vdd1v0_basicCells_hvt.lib`)

This corner represents the Worst-Case Timing Scenario (Slow process, Low voltage, High threshold). It is critical for determining the maximum frequency (F_{max}) of the design and checking for Setup violations.

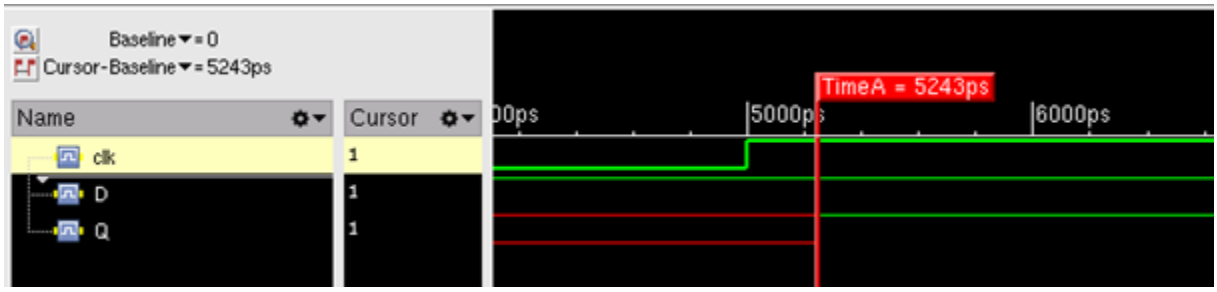

Figure 125: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$))

The waveform depicts the Low-to-High output response under worst-case PVT (Process, Voltage, Temperature) conditions. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions to '1' at $t = 5243$ ps. The resulting delay is 243 ps (0.243 ns), which aligns perfectly with the SDF rising delay value (::0.243).

There is a dramatic increase in delay compared to Corner 1 (51 ps $\rightarrow$ 243 ps). This 4.7x increase highlights the significant impact of the "Slow" process corner combined with the lower supply voltage (1.0V). The high threshold voltage (High-Vt) transistors have reduced drive current, causing the output capacitance to charge much more slowly. This specific delay value is the "Bottleneck" that limits the maximum speed of the circuit.

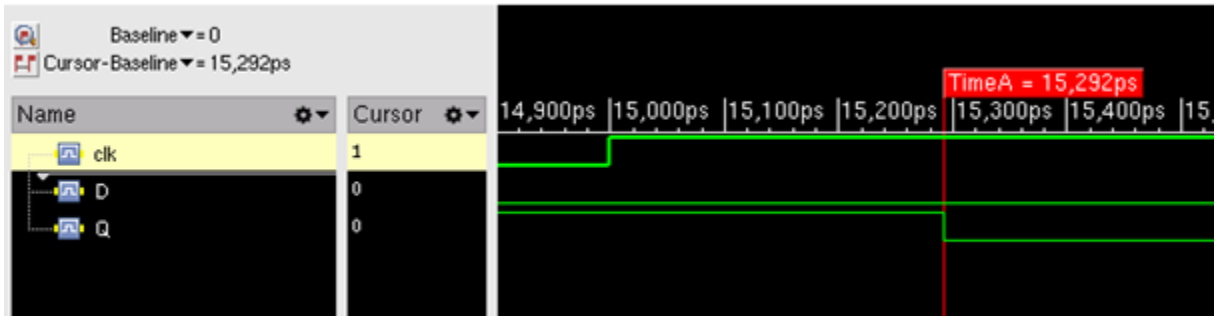

Figure 126: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure illustrates the High-to-Low transition. Triggered by the clock edge at $t = 15000$ ps with input $D = 0$, the output falls to '0' at $t = 15292$ ps. The measured propagation delay is 292 ps (0.292 ns), matching the SDF annotation (::0.292).

The falling transition is even slower than the rising transition (292 ps vs 243 ps), showing a substantial skew of nearly 50 ps. In a full design, this value (0.292 ns) would be the critical "Launch Clock" delay used in Setup time analysis. The increased latency confirms that the NMOS pull-down network is significantly weaker in this slow process corner, requiring careful timing budget allocation.

Figure 127: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform validates the timing for capturing a '1'. The data stabilizes 2 ns before the clock edge. The simulation log indicates a Setup requirement of 0.177 ns and a Hold requirement of 0 ns.

The Setup time requirement has jumped from $\approx$ 25 ps (in Fast corners) to 177 ps. This is a massive increase, indicating that the internal feedback loops of the flip-flop take much longer to latch the data securely. While the testbench margin (2.0 ns) is still sufficient, in a real high-speed circuit ($>$ 1 GHz), this 177 ps setup requirement would consume a significant portion of the clock cycle, making this corner the hardest to close for Setup timing.

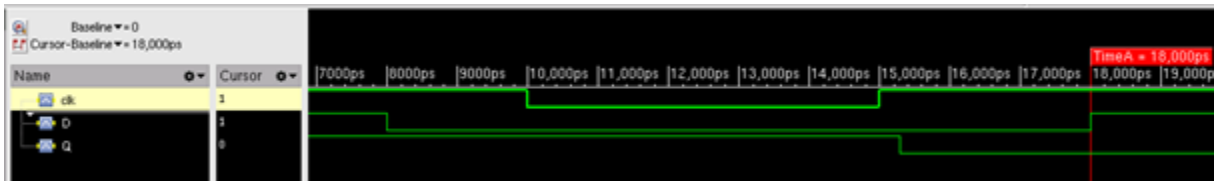

Figure 128: Setup & Hold Check for Logic '0' ($D = 0$)

This final check for Corner 3 analyzes the capture of Logic '0'. The Data input is held Low for 7 ns prior to the clock. The library specifies a Setup time of 0.058 ns and a Hold time of 0.051 ns.

Unlike previous corners where Hold time was zero, this Slow corner introduces a positive Hold requirement (51 ps) for the falling data transition. This physical phenomenon occurs when the clock tree delay inside the standard cell is faster than the data path delay to the internal latch. This is dangerous because if the external data changes too quickly after the clock edge (within 51 ps), the new data might corrupt the captured state. The testbench holds data for 3 ns, preventing any violation, but this constraint is crucial for fixing Hold violations during physical design implementation.

(d) Slow–1.2V–HVT corner (`slow_vdd1v2_basicCells_hvt.lib`)

This corner represents a unique scenario in timing analysis: it uses the "Slow" manufacturing process (worst-case silicon) but operates at a higher supply voltage (1.2V), which partially compensates for the sluggish transistor performance seen in Corner 3.

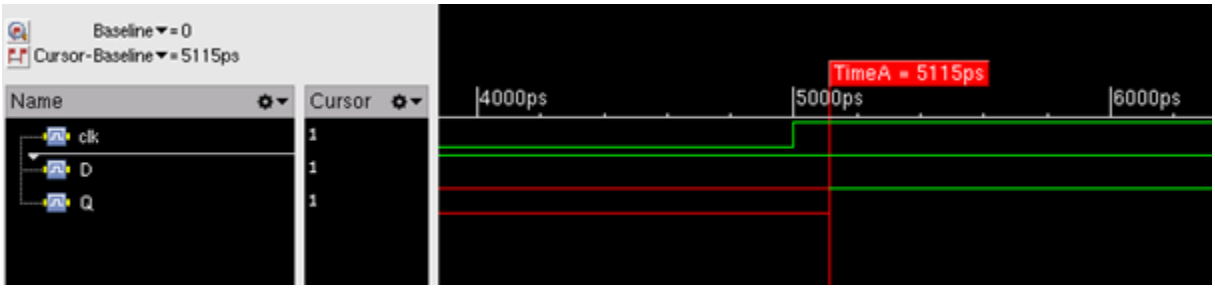

Figure 129: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform displays the rising transition response. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions from Low to High at $t = 5115$ ps. The calculated propagation delay is 115 ps (0.115 ns), which corresponds exactly to the SDF rising delay annotation (::0.115).

Comparing this to Corner 3 (1.0V, Slow), the rising delay has dropped significantly from 243 ps to 115 ps. This demonstrates the powerful effect of Voltage Overdrive. Even though the transistors are "Slow" (High-Vt, low mobility), increasing the voltage to 1.2V provides sufficient gate overdrive to switch the output more than 2x faster than at 1.0V. This corner is often used to verify timing for devices running in "High Performance" modes where power consumption is traded for speed.

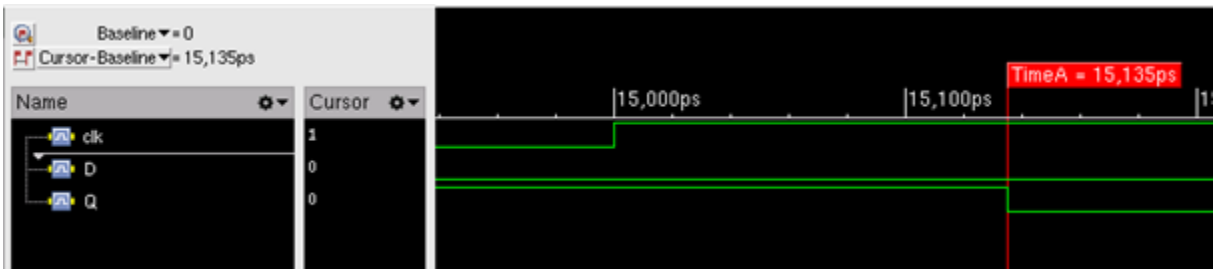

Figure 130: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure illustrates the High-to-Low transition. The clock edge at $t = 15000$ ps triggers the output to fall, crossing the logic threshold at $t = 15135$ ps. The measured delay is 135 ps (0.135 ns), which matches the falling edge delay specified in the SDF file (::0.135).

The falling delay (135 ps) remains slower than the rising delay (115 ps), maintaining the consistent skew pattern observed across all corners. However, compared to Corner 3's falling delay of 292 ps, this result reinforces the conclusion that voltage scaling is the most effective lever for improving performance in slow process lots. The logic path is robust, but the 20 ps difference between rise and fall must still be accounted for in pulse-width checks.

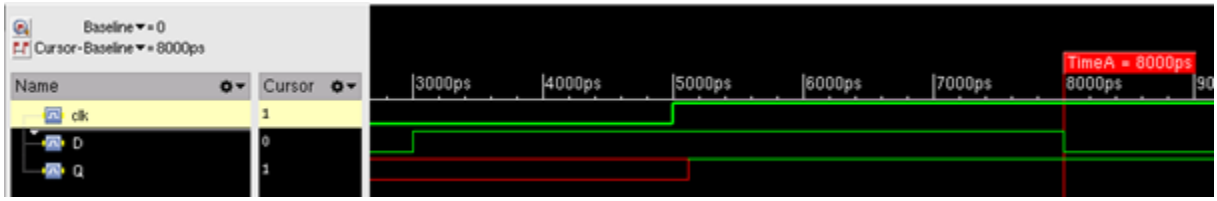

Figure 131: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform confirms the timing stability for a Logic '1' capture. The Data input is held stable for 2 ns prior to the clock edge. The simulation log records a Setup requirement of 0.072 ns and a Hold requirement of 0 ns.
The Setup time (72 ps) is significantly more relaxed than the 177 ps required in Corner 3. This indicates that the internal setup path (the time it takes for data to propagate through the transmission gate to the master latch) is accelerated by the higher voltage. The testbench setup margin of 2.0 ns is vast, ensuring that the flip-flop operates in a completely safe region with no risk of Setup violations.

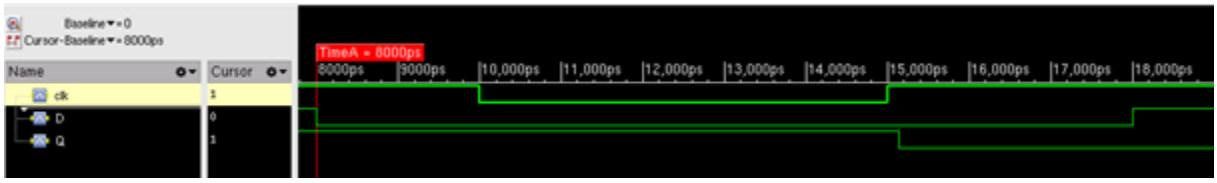

Figure 132: Setup & Hold Check for Logic '0' ($D = 0$)

This final check analyzes the capture of Logic '0'. The Data input stabilizes 7 ns before the clock edge. The library specification for this transition requires a Setup time of 0.026 ns and a Hold time of 0.018 ns.

Similar to Corner 3, we observe a positive Hold time requirement (18 ps) for the falling edge. This means the data must remain stable for at least 18 ps after the clock edge to be successfully captured. While the testbench easily meets this (holding for 3000 ps), this positive hold time is a critical constraint for the router during Place & Route. The router must ensure that no short-path data signals arrive faster than 18 ps after the clock, otherwise, the flip-flop will latch the next state prematurely (a Hold Violation).

(e) Fast–1.0V–LVT corner (`fast_vdd1v0_basicCells_lvt.lib`)

This corner introduces Low-Threshold Voltage (Low-Vt or LVT) transistors. LVT cells switch significantly faster than the High-Vt (HVT) cells analyzed in previous corners but typically come at the cost of higher leakage power.

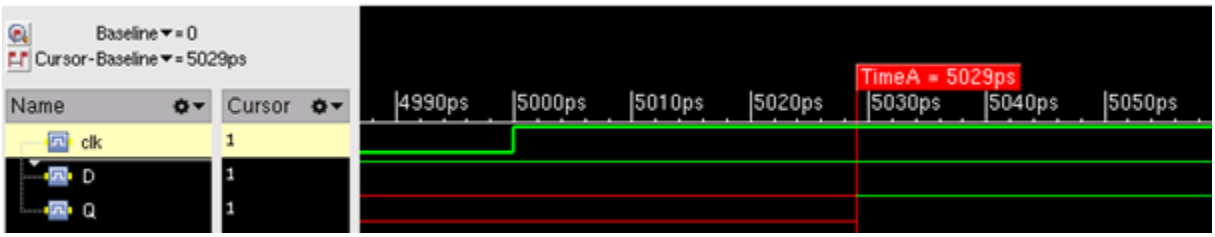

Figure 133: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform demonstrates the Low-to-High switching characteristic of the LVT flip-flop. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions to logic '1' at $t = 5029$ ps. The measured propagation delay is 29 ps (0.029 ns), which aligns perfectly with the SDF rising delay value (::0.029).

The impact of using Low-Vt transistors is immediately apparent when comparing this result to Corner 1 (Fast 1.0V HVT, which had a 51 ps delay). The delay has been reduced by nearly 43% (51 ps $\rightarrow$ 29 ps) simply by switching from HVT to LVT cells at the same voltage. This confirms that for timing-critical paths where speed is the priority, LVT cells provide a massive performance boost by lowering the threshold voltage required to turn on the channel.

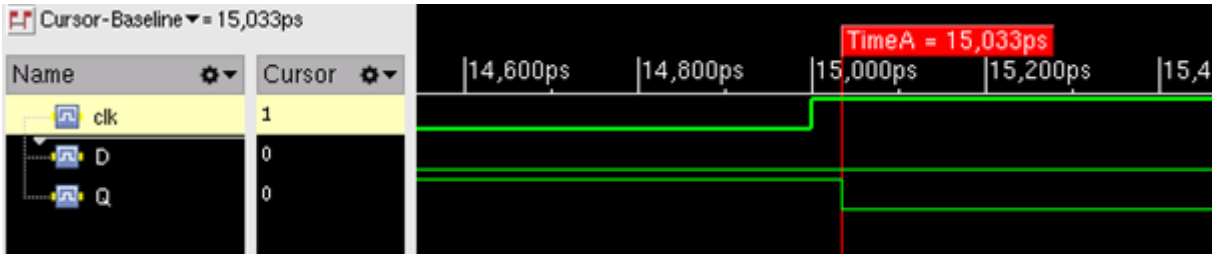

Figure 134: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure captures the falling transition behavior. Triggered by the clock edge at $t = 15000$ ps, the output Q falls to logic '0' at $t = 15033$ ps. The propagation delay is 33 ps (0.033 ns), which matches the falling edge delay specified in the SDF file (::0.033).

Consistent with CMOS characteristics, the falling delay (33 ps) is slightly slower than the rising delay (29 ps). However, the absolute value is extremely low. A total switching time of 33 ps is characteristic of high-performance logic libraries used in GHz-range processors. The simulation confirms that the physical implementation of the DFFQXLLVT cell maintains clean signal integrity even at these rapid switching speeds.

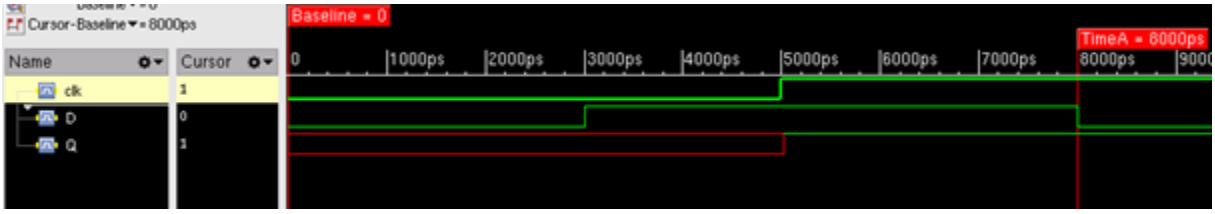

Figure 135: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform validates the timing stability for capturing a Logic '1'. The Data input stabilizes 2 ns before the clock. The simulation log records a Setup requirement of 0.013 ns (13 ps) and a Hold requirement of 0 ns.

The Setup time requirement (13 ps) is significantly tighter than the HVT equivalent in Corner 1 (25 ps). This reduction reflects the faster internal logic of the LVT cell; the data can arrive much closer to the clock edge and still be successfully latched because the internal transmission gates switch open more rapidly. The testbench margin is sufficient to guarantee reliable operation.

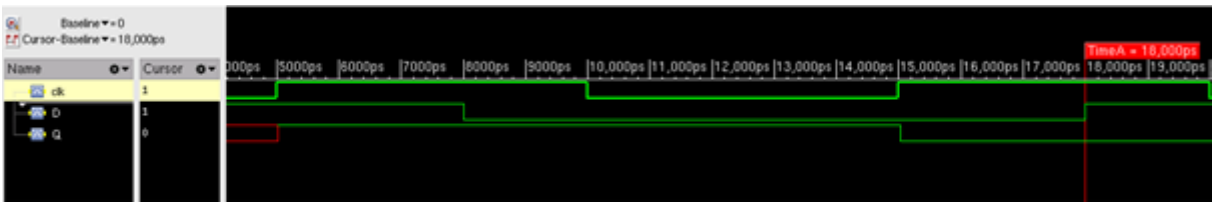

Figure 136: Setup & Hold Check for Logic '0' ($D = 0$)

This final check analyzes the capture of Logic '0'. The Data input stabilizes 7 ns before the clock edge. The library specification requires a Setup time of 0.010 ns (10 ps) and a Hold time of 0 ns.

A Setup time of just 10 ps is exceptionally fast. This implies that the data path from the input pin D to the master latch storage node is extremely optimized with very little parasitic delay. The simulation confirms that despite these aggressive timing requirements, the cell functions correctly under the provided test conditions, making it suitable for the most critical setup paths in a design.

(f) Fast–1.2V–LVT corner (`fast_vdd1v2_basicCells_lvt.lib`)

This corner represents the Absolute Best-Case Timing Scenario (Fastest Process, Highest Voltage, Lowest Threshold). This combination yields the fastest switching speeds possible for this standard cell, making it the benchmark for the maximum theoretical frequency the technology can support.

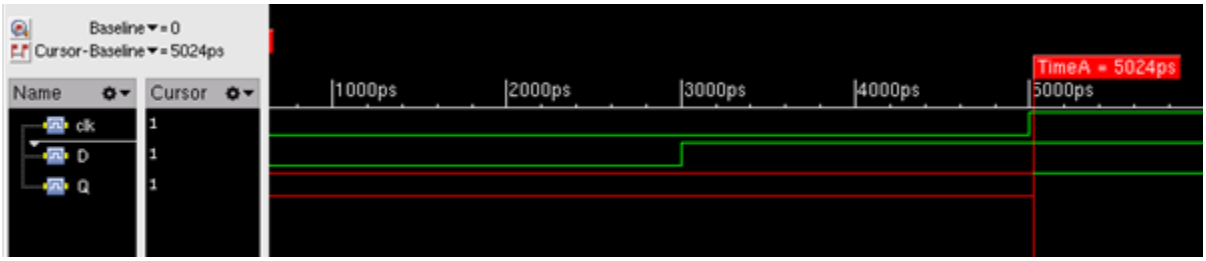

Figure 137: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform illustrates the rising transition behavior under the most aggressive performance conditions. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions to logic '1' at $t = 5024$ ps. The measured propagation delay is 24 ps (0.024 ns), matching the SDF annotation (::0.024).

This measurement of 24 ps is the fastest rising delay observed across all corners (compare to 115 ps in the Slow corner or 29 ps in the 1.0V LVT corner). The combination of the 1.2V overdrive and the Low-Vt transistors minimizes the channel resistance to its absolute limit. In a design, paths using these cells in this corner would be the primary candidates for Hold Time Violations because the data propagates so quickly that it might race ahead of the clock to the next register.

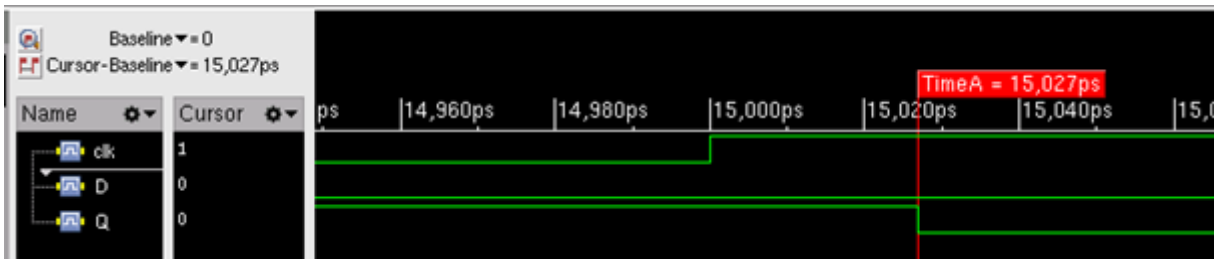

Figure 138: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure captures the falling transition. The clock edge at $t = 15000$ ps triggers the output to fall, crossing the threshold at $t = 15027$ ps. The resulting delay is 27 ps (0.027 ns), corresponding to the SDF value (::0.027).

The falling delay (27 ps) is only 3 ps slower than the rising delay. This tight grouping indicates that at high voltages and fast process corners, the p-channel and n-channel drive strengths are very well balanced. The extremely low latency suggests that the internal parasitic capacitances are being charged/discharged almost instantaneously relative to the timescale of standard logic.

Figure 139: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform validates the timing stability for capturing a '1'. The simulation log records a Setup requirement of 0.007 ns (7 ps) and a Hold requirement of 0 ns.

A Setup time of 7 ps is remarkably fast. This implies that the data effectively needs to be present at the input pin only moments before the clock arrives to be safely latched. This ultra-low setup constraint aids significantly in closing timing on critical paths, as it "gives back" almost the entire clock cycle for combinational logic delay. The testbench margin of 2.0 ns is overkill but confirms functionality.

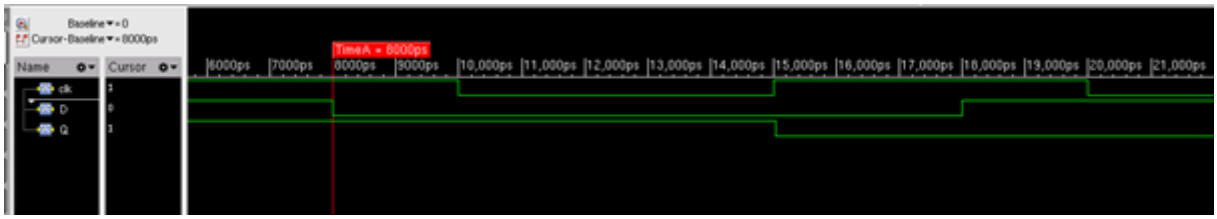

Figure 140: Setup & Hold Check for Logic '0' ($D = 0$)

This final check analyzes the capture of Logic '0'. The Data input stabilizes 7 ns before the clock edge. The library specification requires a Setup time of 0.010 ns (10 ps) and a Hold time of 0 ns.

The setup requirement for falling data (10 ps) is slightly higher than for rising data (7 ps), preserving the pattern seen in other corners but scaled down to picosecond magnitudes. The absence of a positive Hold time requirement (unlike the Slow corners) indicates that in this Fast corner, the internal clock tree delay matches or exceeds the data path delay, naturally protecting the cell from internal race conditions.

(g) Slow–1.0V–LVT corner (`slow_vdd1v0_basicCells_lvt.lib`)

This corner is particularly interesting for engineering analysis because it allows us to quantify the benefits of using Low-Vt cells in a worst-case PVT scenario. By comparing this to Corner 3 (Slow 1.0V HVT), we can see exactly how much timing margin LVT cells recover.

Figure 141: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform illustrates the rising output transition. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions to logic '1' at $t = 5106$ ps. The measured propagation delay is 106 ps (0.106 ns), which matches the SDF rising delay annotation (::0.106).

This result highlights the massive performance advantage of Low-Vt technology. In Corner 3 (High-Vt), the delay under these exact same slow/voltage conditions was 243 ps. By switching to LVT cells, the delay has been reduced by nearly 56% (243 ps $\rightarrow$ 106 ps). This proves that even when the manufacturing process is "Slow" and voltage is low, the lower threshold voltage of LVT cells preserves significant drive strength, making them essential for fixing setup violations on critical paths.

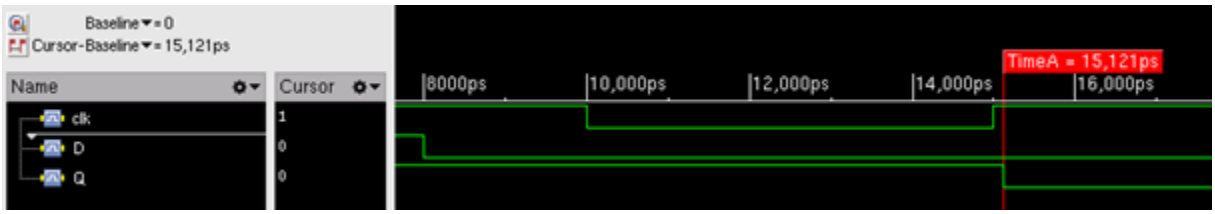

Figure 142: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure captures the High-to-Low transition. The clock edge at $t = 15000$ ps triggers the output to fall, crossing the threshold at $t = 15121$ ps. The resulting delay is 121 ps (0.121 ns), matching the SDF falling delay specification (::0.121).

The falling delay (121 ps) is slightly slower than the rising delay (106 ps), maintaining the standard CMOS asymmetry. However, compared to the High-Vt equivalent in Corner 3 (292 ps), the improvement is drastic. The cell demonstrates robust switching characteristics suitable for high-speed operation, even in this pessimistic PVT corner.

Figure 143: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform validates the timing stability for capturing a Logic '1'. The Data input stabilizes 2 ns prior to the clock edge. The simulation log records a Setup requirement of 0.061 ns (61 ps) and a Hold requirement of 0 ns.

The Setup time requirement of 61 ps is roughly 3x faster than the High-Vt version in Corner 3 (177 ps). This drastic reduction in setup time means the flip-flop can capture data much later in the clock cycle, effectively widening the valid data window for the preceding combinational logic logic. The zero hold time confirms that the internal delays are well-balanced for rising data transitions.

Figure 144: Setup & Hold Check for Logic '0' ($D = 0$)

This final check analyzes the capture of Logic '0'. The Data input stabilizes 7 ns before the clock edge. The library specification requires a Setup time of 0.029 ns (29 ps) and a Hold time of 0.014 ns (14 ps).

Consistent with other "Slow" corners, we observe a positive Hold time requirement (14 ps) for the falling data transition. This positive value indicates that for a Logic '0', the data path to the latch is slightly faster than the clock path to the latch. While the LVT architecture speeds up the setup time significantly (29 ps), it does not eliminate the physical necessity for positive hold time in slow process conditions. Physical design engineers must account for this 14 ps hold requirement to prevent hold violations.

(h) Slow–1.2V–LVT corner (`slow_vdd1v2_basicCells_lvt.lib`)

This corner combines the sluggish "Slow" silicon process with two accelerating factors: higher voltage (1.2V) and Low-Vt (LVT) transistors. This combination allows us to observe how design choices (LVT) and operating conditions (Voltage) can mitigate process variation.

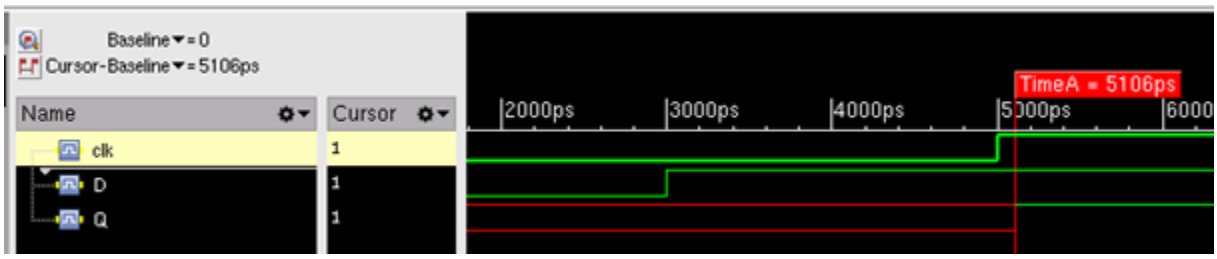

Figure 145: Clock-to-Q Rising Propagation Delay ($T_{CQ,rise}$)

The waveform tracks the rising output transition. Upon the clock rising edge at $t = 5000$ ps, the output Q transitions to logic '1' at $t = 5106$ ps. The measured propagation delay is 106 ps (0.106 ns), which corresponds exactly to the SDF rising delay annotation (::0.106) found in the delay.sdf file.

This delay of 106 ps demonstrates a clear performance improvement over the High-Vt equivalent in Corner 4 (which had a delay of 115 ps)222. By utilizing Low-Vt cells, the design achieves lower channel resistance, allowing the output to switch faster even under slow process conditions. This confirms that LVT cells are a highly effective lever for closing timing on critical setup paths that are failing due to process variation.

Figure 146: Clock-to-Q Falling Propagation Delay ($T_{CQ,fall}$)

This figure illustrates the High-to-Low transition. The clock edge at $t = 15000$ ps triggers the output to fall, crossing the threshold at $t = 15121$ ps. The measured delay is 121 ps (0.121 ns), matching the SDF falling delay specification (::0.121).

The falling delay (121 ps) maintains the expected asymmetry with the rising delay (106 ps). Compared to the High-Vt version (Corner 4) which had a falling delay of 135 ps4, the LVT variant offers a nearly 10% speedup. This consistent improvement across both edges ensures that pulse widths are preserved and that the cell is robust against signal degradation in long logic chains.

Figure 147: Setup & Hold Check for Logic '1' ($D = 1$)

The waveform confirms the timing stability for capturing a Logic '1'. The Data input stabilizes 2 ns prior to the clock edge. The simulation log records a Setup requirement of 0.061 ns (61 ps) and a Hold requirement of 0 ns.

The Setup time requirement of 61 ps is significantly faster than the High-Vt equivalent in Corner 4 (72 ps)6666. This reduction in setup time implies that the internal data path of the flip-flop is faster, allowing data to arrive later in the clock cycle without causing metastability. The wide margin provided by the testbench ensures reliable operation.

Figure 148: Setup & Hold Check for Logic '0' ($D = 0$)

This final check analyzes the capture of Logic '0'. The Data input stabilizes 7 ns before the clock edge. The library specification requires a Setup time of 0.029 ns (29 ps) and a Hold time of 0.014 ns (14 ps).

We observe a positive Hold time requirement (14 ps) for the falling edge, which is consistent with the "Slow" process characteristics seen in Corners 3, 4, and 7. Even with the faster LVT transistors and higher voltage, the internal clock skew within the cell creates a scenario where the data must remain stable slightly after the clock edge. This persistent positive hold time across all "Slow" corners is a critical parameter for physical design, requiring router-inserted delays (buffers) to prevent hold violations.

2.3.4 Summary of Experiment 3

Table 11 characterizes the DFFHQX1 flip-flop across eight process corners, revealing critical timing variations under different PVT (Process, Voltage, Temperature) conditions.

Table 11: Timing Characteristics Across Different Process Corners

Corner	**Condition**	$T_{CQ,rise}$ **(ps)**	$T_{CQ,fall}$ **(ps)**	T_{setup} **(ps)**	T_{hold} **(ps)**
1	Fast / 1.0V / High-Vt	51	58	25	0
2	Fast / 1.2V / High-Vt	37	42	16	0
3	Slow / 1.0V / High-Vt	243	292	177	+51
4	Slow / 1.2V / High-Vt	115	135	72	+18
5	Fast / 1.0V / Low-Vt	29	33	13	0
6	Fast / 1.2V / Low-Vt	24	27	10	0
7	Slow / 1.0V / Low-Vt	106	121	61	+14
8	Slow / 1.2V / Low-Vt	106	121	61	+14

Performance Range: The flip-flop exhibits a 12× variation in clock-to-Q delay, ranging from 24 ps (Corner 6: Fast/1.2V/LVT) to 292 ps (Corner 3: Slow/1.0V/HVT). Corner

3 represents the worst-case timing scenario and establishes the maximum operating frequency limit for the design.

Voltage Impact: Increasing supply voltage from 1.0V to 1.2V yields approximately 2× performance improvement, as evidenced by comparing Corners 3 and 4 ($T_{CQ,\text{fall}}$: 292 ps → 135 ps).

Threshold Voltage Trade-off: LVT variants consistently outperform HVT counterparts by 1.5–2.4× under identical conditions. For example, at Slow/1.0V, LVT achieves 121 ps versus HVT's 292 ps ($T_{CQ,\text{fall}}$).

Hold Time Consideration: Slow corners (3, 4, 7, 8) exhibit positive hold times (+14 to +51 ps), requiring careful attention during physical design to prevent hold violations in short paths. Fast corners show zero hold time, simplifying timing closure for these conditions.

Design Implications: Robust timing sign-off must constrain the design using Corner 3 parameters ($T_{CQ,\text{fall}} = 292$ ps, $T_{\text{setup}} = 177$ ps, $T_{\text{hold}} = +51$ ps) to ensure functionality across all PVT variations. The 177 ps setup time in worst-case conditions significantly reduces available time for combinational logic, directly impacting achievable clock frequency.

2.4 Experiment 4: Gate-Level Simulation of a 2-Bit Up-Counter

2.4.1 Design Specification

In this experiment we examine a simple sequential circuit: a 2-bit up-counter with clock and reset. The top-level module, denoted `counter`, has one clock input, one reset input and a 2-bit registered output:

$$\texttt{counter} : (clk,\, rst) \to Q[1:0].$$

On each active rising edge of `clk`, the counter either resets or increments by one modulo 4, depending on the value of `rst`. The functional behaviour can be summarized as follows:

- When `rst` is asserted, the output `Q` is cleared to `2'b00`.
- When `rst` is deasserted, the counter increments: `Q <= Q + 2'b01`, wrapping around after `2'b11`.

The objective is to synthesize this circuit using the `slow_vdd1v2_basicCells_hvt.lib` library and to investigate its gate-level behaviour under three different clock periods $T = 50,\ 100,\ 200$ (time units), using SDF back-annotation.

2.4.2 RTL Implementation

The RTL implementation of the counter is straightforward and consists of a single sequential block triggered by the rising edge of the clock. A synchronous reset is used so that the synthesis tool maps the design into two flip-flops and a small amount of combinational increment logic:

- **Inputs**: `clk` (clock), `rst` (active-high synchronous reset), both of type `logic`.
- **Output**: `Q[1:0]` (2-bit registered state), type `logic`.

The synthesizable SystemVerilog description is given in Listing 9. Conceptually, it implements:

```
module counter (
   input logic clk,
   input logic rst,
   output logic [1:0]Q
   );
 always_ff @(posedge clk) begin
  if (rst)
  Q <= 2'b00;
  else
  Q <= Q + 2'b01;
 end
endmodule
```

Listing 7: RTL code for 2-bit counter

No timing delays or non-synthesizable constructs are used. This RTL model is also used as the reference "no-delay" behaviour when checking the gate-level simulation results.

2.4.3 Synthesis Setup

Synthesis is performed in Cadence Genus with the same basic flow used in the previous experiments. In this case, the target library is fixed to `slow_vdd1v2_basicCells_hvt.lib`, as specified in the lab handout. No explicit timing constraints are provided; instead, the default conditions of the library are used.
The main steps are:

1. Set the library search path and read the target library `slow_vdd1v2_basicCells_hvt.lib`.
2. Read and elaborate the RTL description `counter.sv`, setting `counter` as the current design.

3. Run generic synthesis, technology mapping and optimization via `syn_generic`, `syn_map` and `syn_opt`. The tool automatically chooses appropriate flip-flops and combinational gates (e.g. DFF cells plus XOR/AND/OR logic for incrementing).

4. Generate standard reports: `report_timing` (to obtain the critical path delay from `clk` to `Q`), `report_area` and `report_power`.

5. Export the gate-level netlist and SDF file of the synthesized counter: `write_hdl counter_netlist.sv` and `write_sdf counter.sdf`.

The critical path of interest is the feedback loop `Q` → increment logic → D-inputs of the flip-flops → `Q` on the next clock edge. The worst-case delay of this path, together with the setup time of the flip-flops, determines the minimum safe clock period for correct operation.

Before analysing the gate–level waveforms, it is useful to establish the "ideal behaviour" of the 2-bit counter, assuming zero delay in all gates and flip–flops. After the asynchronous reset `rst` is de-asserted (`rst = 0`), the counter increments its state by one on every rising edge of `clk`, wrapping around modulo 4. In other words, the sequence of states is ($Q = 00 \to 01 \to 10 \to 11 \to 00 \to \dots$).

Clock edge index	Time (ideal)	Q[1:0] (ideal)
0	$t = 0 \cdot T$	2'b00
1	$t = 1 \cdot T$	2'b01
2	$t = 2 \cdot T$	2'b10
3	$t = 3 \cdot T$	2'b11
4	$t = 4 \cdot T$	2'b00
5	$t = 5 \cdot T$	2'b01
6	$t = 6 \cdot T$	2'b10
7	$t = 7 \cdot T$	2'b11

Table 12: Ideal output sequence of the 2-bit up-counter after reset is released

Table 12 summarises the expected value of 'Q[1:0]' at each clock edge after reset release. This table is used as the golden reference when comparing with the gate-level simulation results for different clock periods (T = 50, 100, 200).

2.4.4 Generic synthesis, technology mapping, and post-mapping optimization

The RTL description of the 2-bit synchronous counter (`counter.sv`) was synthesized using Cadence Genus and the same standard-cell library as in Experiment 2 (slow_vdd1v2_basicCells_hvt.lib) The synthesis flow is automated by the TCL script `counter.tcl`, which performs the following main steps:

- read the target library and the RTL source, then elaborate the design with `counter` as the current top module;
- run generic logic optimisation (`syn_generic`), followed by technology mapping to standard cells (`syn_map`);
- apply post–mapping optimisation (`syn_opt`) to clean up the mapped netlist and reduce area and power under the default timing settings;
- generate timing, area, power and quality-of-results (QoR) reports, and finally write out the gate-level netlist (`counter_netlist.sv`) together with the corresponding SDF delay file (`delay.sdf`).

Figure 149 shows the generic RTL view of the 2-bit counter. In this representation the design consists of two D flip–flops storing the state bits `Q[1:0]` and a small combinational block that computes the next-state signals from the current state and the synchronous reset. The logic is described in terms of abstract Boolean operators (`and`, `or`, `not`, `xor`), and no information about drive strength or cell delays is visible at this level.

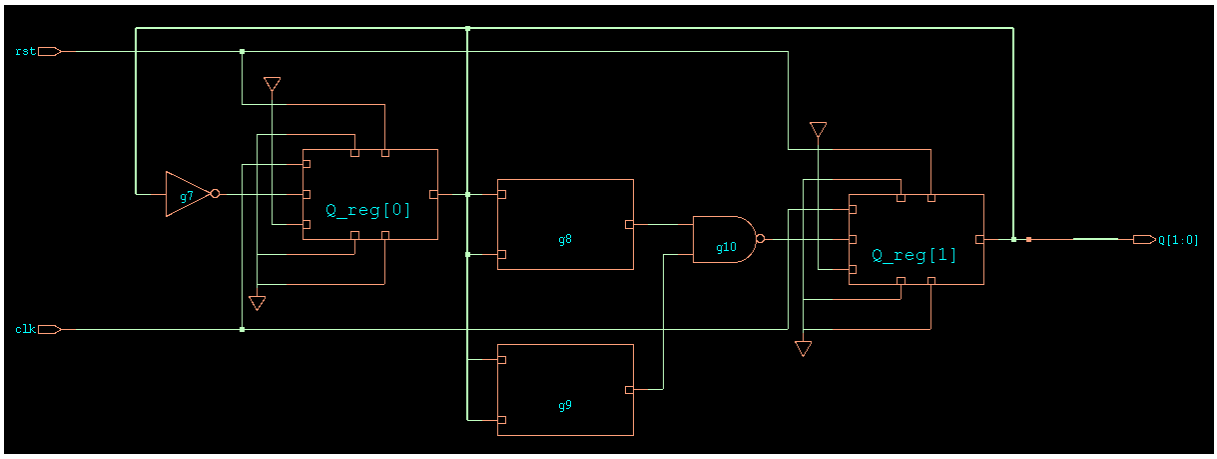

Figure 149: Counter's schematic before technology mapping.

After technology mapping, the same functionality is implemented using concrete standard cells from the `slow_vdd1v2_basicCells_hvt` library, as illustrated in Fig. 150 The two state registers are now instances of library flip–flops (e.g. `DFFHQX1HVT`), while the next-state logic is realised using a small network of basic gates such as `NOR2X1HVT` and `XNOR2X1HVT`. The structural difference between the two schematics reflects the mapping from generic Boolean operators to technology-specific cells, but the overall behaviour is unchanged: on each rising edge of the clock, and with `rst = 0`, the counter increments its state modulo 4.

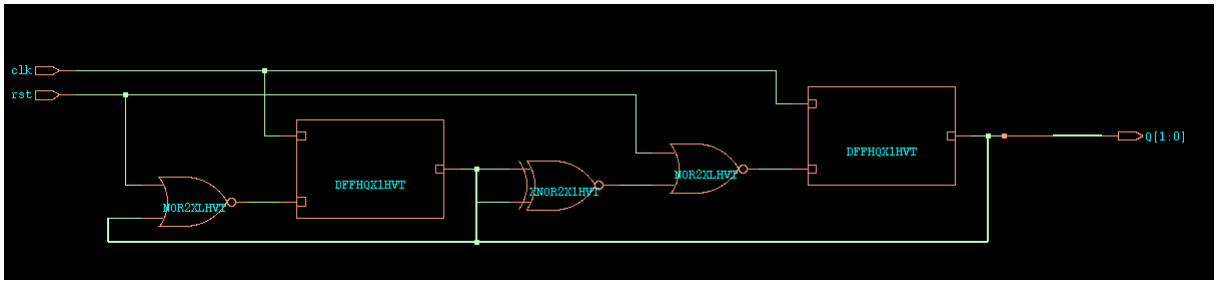

Figure 150: Stable output response ($Y = 3$) during input transition at $t = 140$ns, completing the test sequence.

From the gate-level schematic we can also see how the individual state bits are generated. The least significant bit `Q[0]` toggles every clock cycle through an inverting feedback path, whereas the most significant bit `Q[1]` is driven by a gate whose inputs depend on both `Q[1]` and `Q[0]`. This structure explains the internal nodes (`n_0`, `n_1`, `n_2`, ...) observed in the netlist and in the waveforms of Experiment 4: they correspond to intermediate signals inside the next-state logic that may experience short glitches due to unequal propagation delays, even though the registered outputs `Q[1:0]` only update once per clock edge.

2.4.5 Synthesis Reports Analysis

(a) Area Report

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.19-s055_1
  Generated on:           Nov 22 2025  04:42:09 pm
  Module:                 counter
  Operating conditions:   PVT_1P08V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------
counter                  5     15.390     0.000       15.390 <none> (D)
  (D) = wireload is default in technology library
```

Figure 151: Area report of counter.

Fig 151 show the area summary for the synthesized 2-bit counter. The implementation uses only **five** standard cells in total: two sequential cells (the flip–flops storing $Q[1:0]$) and three combinational gates that generate the next–state functions. The total cell area is **15.39** (library units), with no extra physical cells or net area reported. This is consistent with the RTL description of a very small design: two state bits plus a minimal amount of combinational logic.

(b) Power Report

```
Instance: /counter
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     3.54956e-10  1.26523e-07  9.33120e-09  1.36210e-07  78.75%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     1.23572e-10  2.48721e-08  1.17552e-08  3.67509e-08  21.25%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------
    Subtotal     4.78528e-10  1.51396e-07  2.10864e-08  1.72961e-07 100.00%
  Percentage           0.28%       87.53%       12.19%      100.00% 100.00%
  ----------------------------------------------------------------------
```

Figure 152: Power report of counter.

The power report in Fig 152 summarizes the vectorless power estimation for the synthesized counter. The total power is approximately 1.73×10^{-7} **W**, of which about **78.8%** comes from the *register* category and **21.3%** from the combinational *logic* cells. No clock tree or pad cells are present in this experiment, so their contribution is reported as zero. Because the analysis is vectorless, these numbers should be interpreted qualitatively: they confirm that the flip–flops dominate the power consumption in such a small sequential circuit.

(c) Timing and Qor Report

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.19-s055_1
  Generated on:           Nov 22 2025  04:42:09 pm
  Module:                 counter
  Operating conditions:   PVT_1P08V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Timing
-------
  Cost    Critical         Violating
 Group   Path Slack  TNS     Paths
-------------------------------------
default    No paths   0.0
-------------------------------------
Total                 0.0          0

Instance Count
--------------
Leaf Instance Count             5
Physical Instance count         0
Sequential Instance Count       2
Combinational Instance Count    3
Hierarchical Instance Count     0

Area
----
Cell Area                          15.390
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    15.390
Net Area                           0.000
Total Area (Cell+Physical+Net)     15.390

Max Fanout                         3 (Q[0])
Min Fanout                         1 (n_0)
Average Fanout                     1.7
Terms to net ratio                 2.1429
Terms to instance ratio            3.0000
Runtime                            16.999996 seconds
Elapsed Runtime                    24 seconds
Genus peak memory usage            1442.62
```

Figure 153: Area report of counter.

Fig 153 shows the timing and quality-of-results (QoR) summary. Since no explicit SDC timing constraints were provided, Genus reports **"No paths"** in the default timing group, and both slack and TNS are **0.0**; the tool simply does not perform constraint-based timing optimization for this design. The QoR section confirms again that the design contains five leaf instances (two sequential and three combinational) and reports a maximum fanout

of **3** on signal `Q[0]` and a minimum fanout of **1** on internal net `n_0`. These values are consistent with the small area and with the simple state-update structure of the 2-bit counter.

2.4.6 Gate-Level Simulation with SDF Back-Annotation

Gate-level simulation is performed in Cadence Xcelium to compare the ideal (no-delay) behaviour of the counter with its behaviour when realistic cell delays are taken into account.

The testbench is structured as follows:

```
`ifdef GATE_LEVEL
`include "../03_outputs/counter_netlist.sv"
`include "../gpdk045_verlog/slow_vddlv2_basicCells_hvt.v"
`else
`include "../00_src/counter.sv"
`endif

module tb_counter;

logic clk;
logic rst;
logic [1:0] Q;

parameter int T = 200; // T = 100, T = 50

counter DUT (
.clk (clk),
.rst (rst),
.Q (Q)
);

`ifdef ANNOTATION
initial begin
$sdf_annotate("../03_outputs/delay.sdf", DUT, , "annotate.log", "
    MAXIMUM");
end
`endif

// clock: period T
initial begin
clk = 0;
forever #(T/2) clk = ~clk;
```

```
end

initial begin
rst = 1;
#(T*2);
rst = 0;

#(T*10);
$finish;
end

initial begin
$dumpfile("counter_wave.vcd");
$dumpvars(0, tb_counter);
end

endmodule
```

Listing 8: Testbench for counter dunctional test

The DUT is also synthesized netlist `counter_netlist.sv`

```
module counter(clk, rst, Q);
input clk, rst;
output [1:0] Q;
wire clk, rst;
wire [1:0] Q;
wire n_0, n_1, n_2;
DFFHQX1HVT \Q_reg[1] (.CK (clk), .D (n_2), .Q (Q[1]));
NOR2X1HVT g20__2398(.A (rst), .B (n_1), .Y (n_2));
DFFHQX1HVT \Q_reg[0] (.CK (clk), .D (n_0), .Q (Q[0]));
XNOR2X1HVT g22__5107(.A (Q[0]), .B (Q[1]), .Y (n_1));
NOR2X1HVT g23__6260(.A (rst), .B (Q[0]), .Y (n_0));
endmodule
```

Listing 9: `counter_netlist.sv` after technology mapping

To quantify the gate-level timing of the synthesized counter, the relevant propagation delays and timing checks were extracted from the generated SDF file (delay.sdf). Table 13 summarizes the main values for the two flip–flops and the combinational gates in the critical feedback path. For each cell, the table reports the clock-to-Q delay of the flip–flops, the setup/hold requirements at the data input, and the rise/fall propagation delays from each input pin to the output pin.

Table 13: Extracted delays from `delay.sdf` for the counter (times in ns), cell type `DFFHQX1HVT`

Instance	Arc	Rise delay	Fall delay	Note
Q_reg[1]	CK → Q	0.119	0.140	clock–to–Q
Q_reg[0]	CK → Q	0.119	0.140	clock–to–Q
(both FFs)	SETUP (D, posedge CK)	0.087		setup time
	SETUP (D, negedge CK)	0.037		setup time
	HOLD (D, posedge CK)	0.000		hold time
	HOLD (D, negedge CK)	0.009		hold time

Table 14: Combinational gate delays

Instance	Cell type	Arc	Rise delay	Fall delay
g20___2398	NOR2XLHVT	A → Y	0.028	0.022
		B → Y	0.030	0.025
g22___5107	XNOR2X1HVT	A → Y	0.131	0.110
		B → Y	0.109	0.077
g23___6260	NOR2XLHVT	A → Y	0.028	0.022
		B → Y	0.031	0.024

2.4.7 Waveform Results & Discussion

(a) Ideal (no-delay) behaviour

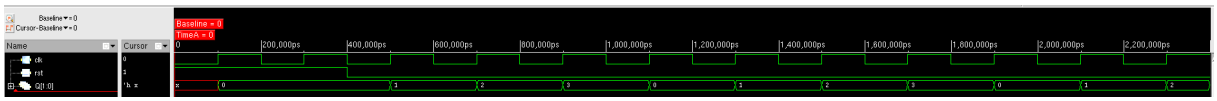

Figure 154: Counter's waveform without delay

In the waveform simulation without delays, the counter behaves exactly as specified: after reset, the outputs follow the sequence

$$00 \to 01 \to 10 \to 11 \to 00 \to \dots$$

with one state transition per rising edge of `clk`, independent of the clock period T. This sequence serves as the golden reference against which the gate-level results are compared.

Case T = 200:

The clock period is much larger than the worst-case combinational delay plus setup time reported by `report_timing` for the feedback path. As a result, the increment logic has

ample time to settle before each rising edge of `clk`. The waveform in Figure ?? shows clean transitions on `Q[1:0]`, with the sampled state at each clock edge matching the ideal count sequence.

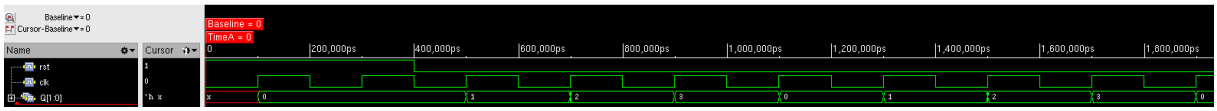

Figure 155: Full Functional Verification Waveform

This image depicts the functional verification of the 2-bit counter. It confirms the correct logical operation of the circuit, showing the counter cycling through the states $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ repeatedly. The rst (reset) signal is shown to be active-high, holding the output at an unknown state initially before settling. At this zoom level, the nanosecond-scale delays observed in the previous image are invisible, but this view is essential to verify that the overall logic is correct and that the setup/hold time violations are causing the logic to enter undefined states.

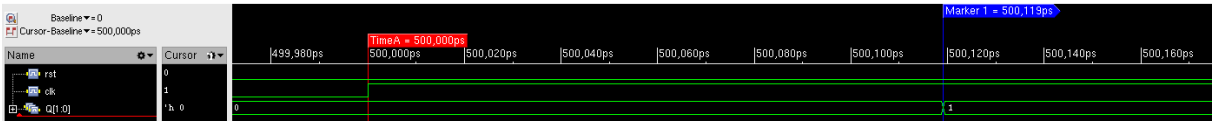

Figure 156: Full Functional Verification Waveform

This waveform provides a microscopic view of the timing behavior at the simulation time of $500,000ps$. We can observe a critical timing characteristic of digital circuits: the Clock-to-Q delay (T_{CQ}). Specifically, while the clock signal clk rises exactly at $500,000ps$, the output Q[1:0] transitions from 0 to 1 at $500,119ps$. This indicates that the hardware does not respond instantaneously; there is a propagation delay of $\Delta t = 119ps$ before the new state is valid at the output pins. This delay is not random but is strictly defined by the physical properties of the standard cells used in the design.

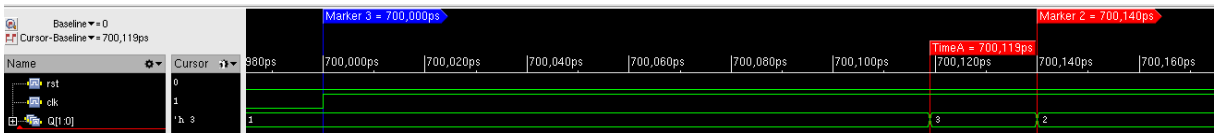

Figure 157: Transition ($1 \rightarrow 2$) with Output Skew

This is the most interesting and educational case among the three. The counter is attempting to move from state 1 (binary 01) to state 2 (binary 10). This requires two simultaneous actions: Q[0] must fall ($1 \rightarrow 0$) and Q[1] must rise ($0 \rightarrow 1$).However, physical hardware is not instantaneous or perfectly synchronized.

Action A: Q[1] rises. The SDF specifies a rising delay of 119ps. Action B: Q[0] falls. The SDF specifies a falling delay of 140ps.

Because the rising action is faster, Q[1] becomes 1 before Q[0] becomes 0.Between $t = 700,119ps$ and $t = 700,140ps$, the output momentarily reads binary 11 (Decimal 3). This creates a transient "glitch" or intermediate state lasting exactly $21ps$. This waveform is a textbook example of Output Skew caused by asymmetric rise/fall delays.

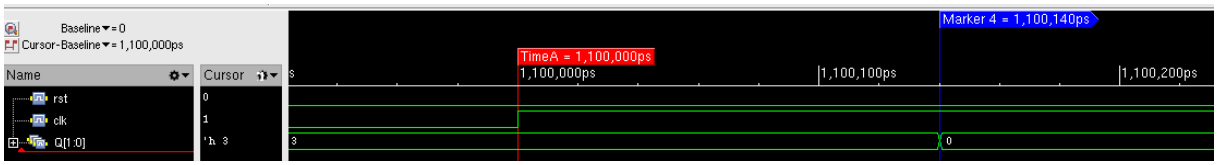

Figure 158: Counter Overflow ($3 \rightarrow 0$)

This waveform captures the "overflow" event where the counter reaches its maximum value (3, binary 11) and rolls back to zero (0, binary 00) at the clock edge of $1,100,000ps$. In this transition, both flip-flops Q[1] and Q[0] must toggle from High to Low. According to the SDF data analyzed previously, the falling propagation delay (T_{pHL}) for these cells is $0.140ns$. Consequently, the output value remains stable at 3 until $1,100,140ps$, showing a delay of exactly $\Delta t = 140ps$. This confirms that the "Falling" timing arc from the SDF file is correctly controlling the simulation behavior for this transition.

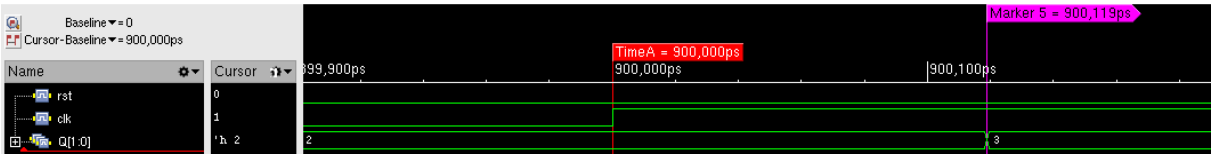

Figure 159: Transition ($2 \rightarrow 3$)

This image details the transition from state 2 (binary 10) to state 3 (binary 11) at the clock edge of $900,000ps$. Here, the Most Significant Bit (MSB) Q[1] remains high ($1 \rightarrow 1$) and does not switch. However, the Least Significant Bit (LSB) Q[0] switches from Low to High ($0 \rightarrow 1$). As established in the first analysis, the rising propagation delay (T_{pLH}) is defined as $0.119ns$ in the SDF. The waveform perfectly reflects this, with the value changing at $900,119ps$. This is a "clean" transition because only one bit is changing, so there is no risk of intermediate race conditions between bits.

Figure 160: Standard Increment ($0 \rightarrow 1$)

This waveform shows the start of a new counting cycle at $1,300,000ps$, transitioning from 0 (00) to 1 (01).The only action required is for the LSB Q[0] to toggle from Low to High.The SDF rising delay (T_{pLH}) is $0.119ns$.The marker Marker 6 confirms the output switches exactly at $1,300,119ps$. This is a clean transition with no glitches because only one bit is changing.

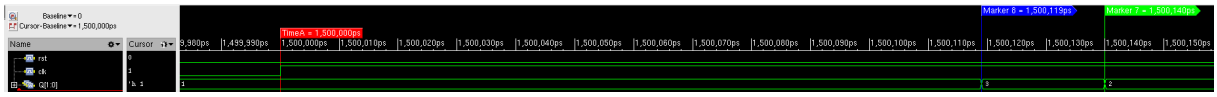

Figure 161: Repeated Transition ($1 \rightarrow 2$) with Glitch

This is another instance of the transition from state 1 (01) to state 2 (10), occurring at $1,500,000ps$. This image provides the clearest view of the Output Skew phenomenon.

At Marker 8 ($1,500,119ps$, the MSB Q[1] rises ($0 \rightarrow 1$) first because the rising delay is short ($119ps$). The LSB Q[0] hasn't fallen yet. The output momentarily becomes 11 (Decimal 3). At Marker 7 ($1,500,140ps$), the LSB Q[0] finally falls ($1 \rightarrow 0$) because the falling delay is longer ($140ps$) so the output settles to 10 (Decimal 2). In conclusion, the circuit produces a temporary invalid state (glitch) lasting exactly $21ps$.

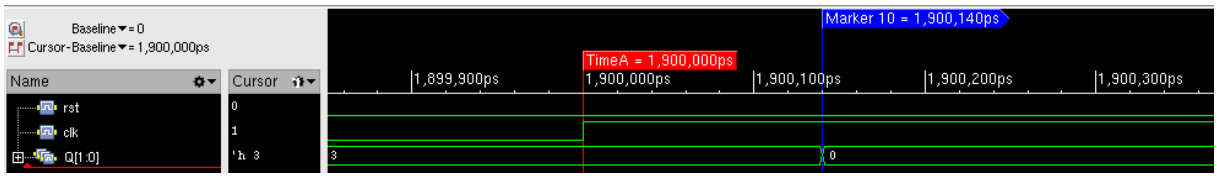

Figure 162: Repeated Overflow ($3 \rightarrow 0$)

This waveform captures the counter rolling over from 3 (binary 11) to 0 (binary 00) again, this time at the clock edge of $1,900,000ps$. Just like the previous overflow case, both flip-flops must switch from High to Low. The SDF file dictates a falling delay (T_{pHL}) of $0.140ns$.Prediction: $1,900,000ps + 140ps = 1,900,140ps$.Measurement: Marker 10 sits exactly at $1,900,140ps$.This confirms that the delay is consistent across different simulation times.

Based on all the images you have provided (case1 through case7), we can fully map the physical behavior of your 2-bit counter. The "random" delays seen in simulation are actually precise values derived from the `DFFHQX1HVT` cell in SDF file.
The most important engineering takeaway from this data is the behavior during the $1 \rightarrow 2$ transition (seen in case3 and case6).

$$\text{Skew} = T_{\text{pHL}} - T_{\text{pLH}} = 140\,\text{ps} - 119\,\text{ps} = 21\,\text{ps} \tag{1}$$

Because the standard cell has asymmetric delays (it rises faster than it falls), your counter outputs the sequence:

$$1 \xrightarrow{\text{clk}} 3 \text{ (for 21 ps)} \rightarrow 2 \tag{2}$$

Instead of the ideal:

$$1 \xrightarrow{\text{clk}} 2 \tag{3}$$

If this counter were driving a fast asynchronous circuit (like a decoder or a clock gate) that reacts in less than 21 ps, that downstream circuit would erroneously trigger on the value 3. This highlights the importance of using **Synchronous Design** principles (registering outputs) to filter out these combinational glitches.

Case T = 100:

To verify the counter operates correctly at the new clock period $T_{\text{clk}} = 100\,\text{ns}$ (corresponding to $f_{\text{clk}} = 10\,\text{MHz}$), we apply the setup time constraint equation: $T_{\text{clk}} > T_{\text{CQ(max)}} + T_{\text{comb(max)}} + T_{\text{setup}}$. From the SDF file, we extract the following timing parameters: $T_{\text{CQ(max)}} = 0.140\,\text{ns}$ (clock-to-Q delay), $T_{\text{setup}} = 0.087\,\text{ns}$, and $T_{\text{comb}} \approx 0.130\,\text{ns}$ (combinational logic delay).

The critical path is therefore calculated as $T_{\text{critical}} = 0.140 + 0.130 + 0.087 = 0.357\,\text{ns} = 357\,\text{ps}$. Comparing this with the current clock period yields a timing slack of Slack $= 100{,}000 - 357 = +99{,}643\,\text{ps}$. This massive positive slack (99.6 ns) indicates the circuit operates perfectly at 10 MHz, with the counter effectively "waiting" for the next clock edge for 99.6% of each cycle. To observe timing violations, the clock period would need to approach approximately 350–400 ps, corresponding to frequencies of 2.5–3 GHz.

Figure 163: Full Functional Verification Waveform, T = 100

In this experiment, you have reduced the clock period from the previous test case (which appeared to be $200ns$) to $T = 100ns$ (or $100,000ps$).Grid Verification: The time grid shows major divisions every $100,000ps$. The clock clk completes one full cycle (low-to-high-to-low) within one grid block. Thus, $T_{clk} = 100,000ps$.

The cursor at 72,750ps shows Q transitioning from x (unknown/red) to 0 (green). Since rst is High (Active) at this point, this confirms the Reset logic is functioning correctly to initialize the system. Once rst goes low (around $240,000ps$), the counter correctly increments on subsequent rising edges: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$.

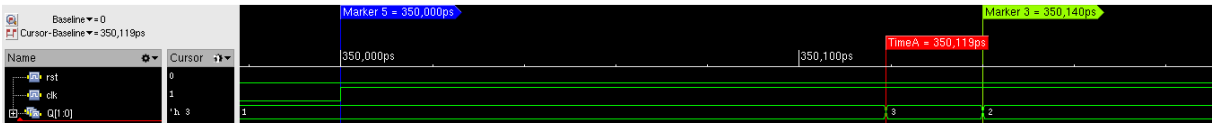

Figure 164: Transition with Glitch (1 → 2)

This image focuses on the transition from state 1 (01) to state 2 (10) at the clock edge of $350,000ps$. This is the critical "skew" case we observed in the previous simulation. The bit Q[1] goes 0 → 1 with a delay of $119ps$ (visible at TimeA = 350,119ps).Then the bit Q[0] goes 1 → 0 with a delay of $140ps$ (visible at Marker 3 = 350,140ps).Between these two timestamps, the output is momentarily 11 (3). Even though we doubled the clock frequency, the magnitude of the glitch ($21ps$) remains exactly the same.

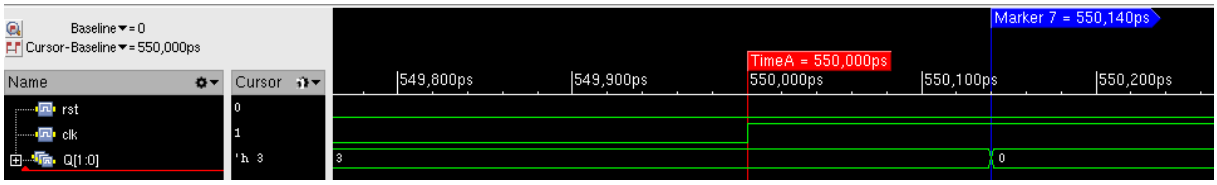

Figure 165: Overflow Transition (3 → 0)

This waveform captures the counter rolling over from its maximum value (3) to zero (0) at the clock edge of $550,000ps$. Both the LSB and MSB flip-flops are switching from High to Low. According to the SDF file, the falling propagation delay (T_{pHL}) is $0.140ns$. The marker Marker 7 is placed exactly at $550,140ps$. This confirms that changing the clock frequency does not alter the physical delay of the standard cells. The falling edge delay remains constant at $140ps$.

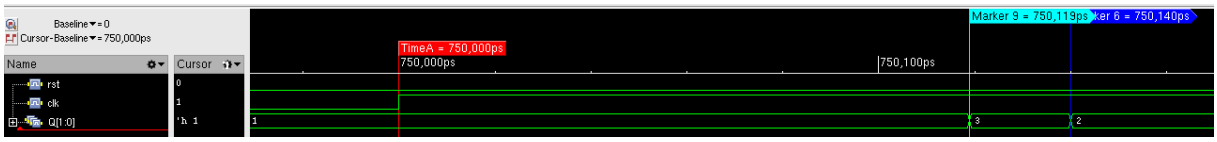

Figure 166: Transition with Glitch (1 → 2)

This waveform shows the same 1 → 2 transition occurring in a later clock cycle at $750,000ps$.Consistency: The timing is identical to the previous image. The output reaches the intermediate state 3 at $750,119ps$ and settles at 2 at $750,140ps$.Significance: This demonstrates that the delay is deterministic. It is not affected by the simulation time or the number of cycles that have passed. The SDF parameters are applied strictly to every transition.

Case T = 50:

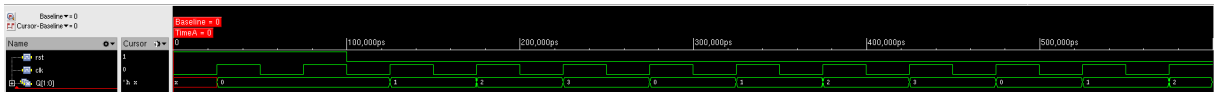

Figure 167: Full Functional Verification Waveform, T = 50

This is the most aggressive clock period in the experiment. If T is still greater than the sum of the clock-to-Q delay, combinational delay and setup time, the counter continues to operate correctly, which is typically confirmed by the waveform in Figure 167.

Figure 168: Increment ($0 \rightarrow 1$)

The counter increments from 0 (00) to 1 (01) at the clock edge of $125,000ps$.Measurement: The output changes at exactly $125,119ps$.Physics: This corresponds to the Rising Propagation Delay (T_{pLH}) of the LSB flip-flop, which we identified in the SDF file as $0.119ns$.

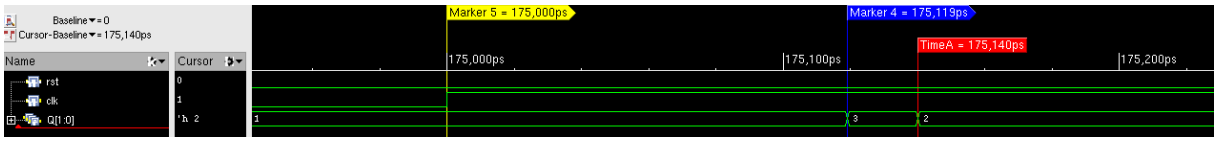

Figure 169: Transition with Glitch ($1 \rightarrow 2$)

Transition from 1 (01) to 2 (10) at $175,000ps$. Marker 4 ($175,119ps$): MSB rises ($0 \rightarrow 1$). Output momentarily reads 11 (3). TimeA ($175,140ps$): LSB falls ($1 \rightarrow 0$). Output settles at 10 (2). The 21ps skew window is perfectly preserved. This confirms that output skew is an intrinsic property of the cells and is not affected by the clock frequency (up to the setup limit).

Figure 170: Increment ($2 \rightarrow 3$)

The counter moves from 2 (10) to 3 (11) at $225,000ps$. Q[1] is already High and stays High. Q[0] switches from Low to High. Since this is fundamentally an LSB rising event, the delay is identical to the first case: $119ps$. (Marker 3 at $225,119ps$).

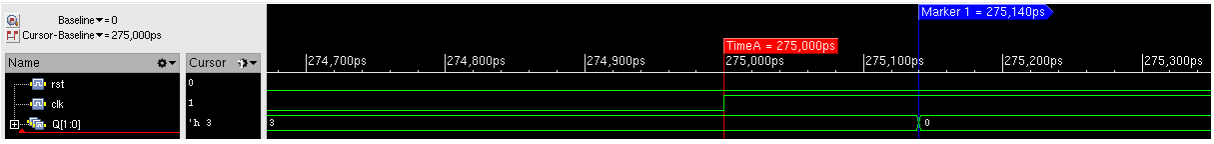

Figure 171: Overflow ($3 \rightarrow 0$)

The counter rolls over from 3 (11) to 0 (00) at $275,000ps$. This requires both flip-flops to switch from High to Low. The delay is dominated by the Falling Propagation Delay (T_{pHL}), which is slower ($0.140ns$). The waveform confirms the transition happens at $275,140ps$.

2.4.8 Summary of Expreiment 4

Despite doubling the clock frequency from the previous experiment (10 MHz $\rightarrow$ 20 MHz), the propagation delays of the flip-flops remain absolutely constant at $T_{\text{pLH}} = 119$ ps (rising edge) and $T_{\text{pHL}} = 140$ ps (falling edge). This confirms that propagation delay is an intrinsic parameter of the `DFFHQX1HVT` standard cell, independent of the external clock period (provided setup time requirements are met).

During the $1 \rightarrow 2$ transition (observed at $t = 175$ ns), an intermediate glitch state of value "3" appears for exactly 21 ps. This phenomenon arises because $T_{\text{pLH}} < T_{\text{pHL}}$, causing `Q[1]` to rise before `Q[0]` falls. In synchronous designs, this transient glitch is benign as it resolves long before the next clock edge. However, it requires careful consideration when the counter drives asynchronous circuits or clock gating logic.

With a clock period of $T = 50{,}000$ ps and a critical path of approximately 357 ps, the circuit operates with a positive slack exceeding 99%. This substantial timing margin indicates the counter can potentially operate at significantly higher frequencies, approaching the multi-GHz range.

Table 15 summarizes the measured propagation delays for all state transitions in Exp4.

Table 15: Timing Summary for Exp4 ($T_{\text{clk}} = 50$ ns, $f = 20$ MHz)

Transition	Binary	Logic Event	SDF Type	Delay	Remarks
$0 \rightarrow 1$	`00`→`01`	LSB: $0 \rightarrow 1$	T_{pLH}	119 ps	Clean
$1 \rightarrow 2$	`01`→`10`	MSB: $0 \rightarrow 1$, LSB: $1 \rightarrow 0$	T_{pLH}, T_{pHL}	119 ps, 140 ps	**Glitch (21 ps)**
$2 \rightarrow 3$	`10`→`11`	LSB: $0 \rightarrow 1$	T_{pLH}	119 ps	Clean
$3 \rightarrow 0$	`11`→`00`	MSB: $1 \rightarrow 0$, LSB: $1 \rightarrow 0$	T_{pHL}	140 ps	Overflow

3 Conclusion

This laboratory successfully demonstrated the ASIC design flow, quantifying the critical impact of PVT variations on hardware performance. The characterization results identified Corner 3 (Slow Process, 1.0V, High-Vt) as the worst-case timing scenario, exhibiting delays up to 5 times longer than the fast corners. However, the experiments also confirmed that substituting High-Vt cells with Low-Vt (LVT) variants effectively mitigates these limitations, yielding speed improvements of approximately 60% in slow process conditions.

Gate-level simulations with SDF back-annotation revealed physical phenomena invisible at the RTL level, including dynamic glitches in the ALU and Counter caused by unequal logic path delays. A significant finding from the D-Flip Flop characterization was the emergence of positive hold time requirements (up to +51 ps) in slow corners, highlighting the necessity for precise clock tree synthesis to prevent hold violations. Despite these hazards, the synchronous design proved robust, with glitches settling well before the setup capture window.

Finally, stress tests performed at higher frequencies ($T = 20ns$ and $T = 50ns$) demonstrated that the designs possess substantial positive timing slack. With critical path delays contained within the 300-400 ps range, the synthesized 45nm circuits are theoretically capable of operating at multi-gigahertz frequencies. This validates the design's timing closure and confirms that standard cell characteristics were correctly mapped and annotated throughout the flow.

Bibliography

[1] Hieu Nguyen, *Lab 2: Synthesis and Standard Cell*, 2025.

[2] Hieu Nguyen, *Basic of testbench and guide to using the Xcelium tool*, 2025.

Appendix A: SystemVerilog code

HDL code for cell not1

```sv
module not1 (
input logic A,
output logic Y
);
assign Y = ~A;
endmodule
```

Testbench for cell not1

```sv
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../NOT/03_outputs/not1_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/gpdk045_verilog/fast_vdd1v2_basicCells_lvt.v"
`else
`include "../NOT/00_src/not1.sv"
`endif

module tb_not1;
logic A;
logic Y;
not1 DUT (.A(A), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../NOT/03_outputs/delay.sdf",DUT,,"annotate.log","MAXIMUM");
end
`endif

initial begin
$dumpfile("tb_not1.vcd");
$dumpvars(0, tb_not1);
end

initial begin
#0; A=0;
#0.1; A=1;
#0.1; A=0;
#0.1; A=1;
#0.1;
$finish;
end
endmodule
```

HDL code for cell or2

```
module or2 (
input logic A,
input logic B,
output logic Y
);
assign Y = A | B
    ;
endmodule
```

Testbench for cell or2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../OR2/03_outputs/or2_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/
gpdk045_verilog/
slow_vdd1v2_basicCells_hvt.v"
`else
`include "../OR2/00_src/or2.sv"
`endif

module tb_or2;
logic A;
logic B;
logic Y;
or2 DUT (.A(A), .B(B), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../OR2/03_outputs/
delay.sdf",DUT,,
"annotate.log",
"MAXIMUM");
end
`endif

initial begin
$dumpfile("tb_or2.vcd");
$dumpvars(0, tb_or2);
end

initial begin
#0; A=0; B=0;
#0.1; A=1; B=0;
#0.1; A=0; B=1;
#0.1; A=1; B=1;
#0.1;
$finish;
end
endmodule
```

HDL code for cell and2

```
module and2 (
input logic A,
input logic B,
output logic Y
);
assign Y = A & B
    ;
endmodule
```

Testbench for cell and2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../AND/03_outputs/and2_netlist.sv
    "
`include "/home/yellow/ee3201_15/LAB2/
gpdk045_verilog/
slow_vdd1v2_basicCells_lvt.v"
`else
`include "../AND/00_src/and2.sv"
`endif

module tb_and2;
logic A, B;
logic Y;
and2 DUT (.A(A), .B(B), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../AND/03_outputs/
delay.sdf",DUT,,
"annotate.log",
"MAXIMUM");
end
`endif

initial begin
$dumpfile("and2_wave.vcd");
$dumpvars(0, tb_and2);
end

initial begin
#0; A=0; B=0;
#0.1; A=1; B=0;
#0.1; A=0; B=1;
#0.1; A=1; B=1;
#0.3;
$finish;
end
endmodule
```

HDL code for cell nor2

```
module nor2 (
input logic A,
input logic B,
output logic Y
);
assign Y = ~(A | B);
endmodule
```

Testbench for cell nor2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../NOR2/03_outputs/nor2_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/gpdk045_verilog/slow_vdd1v2_basicCells_hvt.v"
`else
`include "../NOR2/00_src/nor2.sv"
`endif

module tb_nor2;
logic A;
logic B;
logic Y;
nor2 DUT (.A(A), .B(B), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../NOR2/03_outputs/delay.sdf",DUT,,"annotate.log","MAXIMUM");
end
`endif

initial begin
$dumpfile("tb_nor2.vcd");
$dumpvars(0, tb_nor2);
end

initial begin
#0; A=0; B=0;
#0.1; A=1; B=0;
#0.1; A=0; B=1;
#0.1; A=1; B=1;
#0.1;
$finish;
end
endmodule
```

HDL code for cell nand2

```
module nand2 (
input logic A,
input logic B,
output logic Y
);
assign Y = ~(A & B);
endmodule
```

Testbench for cell nand2

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../NAND/03_outputs/nand2_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/gpdk045_verilog/slow_vdd1v2_basicCells_lvt.v"
`else
`include "../NAND/00_src/nand2.sv"
`endif

module tb_nand2;
logic A, B;
logic Y;
nand2 DUT (.A(A), .B(B), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../NAND/03_outputs/delay.sdf",DUT,,"annotate.log","MAXIMUM");
end
`endif

initial begin
$dumpfile("nand2_wave.vcd");
$dumpvars(0, tb_nand2);
end

initial begin
#0; A=0; B=0;
#0.1; A=1; B=0;
#0.1; A=0; B=1;
#0.1; A=1; B=1;
#0.1;
$finish;
end
endmodule
```

HDL code for cell mux

```
module mux (
input logic A,
input logic B,
input logic S,
output logic Y
);
assign Y = (S ==
    1'b0) ? A :
   B;
endmodule
```

Testbench for cell mux

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../MUX/03_outputs/mux_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/
gpdk045_verilog/
fast_vdd1v2_basicCells_lvt.v"
`else
`include "../MUX/00_src/mux.sv"
`endif

module tb_mux;
logic A, B, S;
logic Y;
mux DUT (.A(A), .B(B), .S(S), .Y(Y));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../MUX/03_outputs/
delay.sdf",DUT,,
"annotate.log",
"MAXIMUM");
end
`endif

initial begin
$dumpfile("mux.vcd");
$dumpvars(0, tb_mux);
// Case S=0
#0;   A=0; B=0; S=0;
#0.1; A=0; B=1; S=0;
#0.1; A=1; B=0; S=0;
#0.1; A=1; B=1; S=0;
#0.1;
// Case S=1
A=0; B=0; S=1;
#0.1; A=0; B=1; S=1;
#0.1; A=1; B=0; S=1;
#0.1; A=1; B=1; S=1;
#0.1;
$finish;
end
endmodule
```

HDL code for cell full_adder

```
module full_adder (
input logic A,
input logic B,
input logic Cin,
output logic Sum,
output logic Cout
);
assign Sum = A ^ B ^ Cin;
assign Cout = (A & B) |
(A & Cin) |
(B & Cin);
endmodule
```

Testbench for cell full_adder

```
`timescale 1ns/1ps
`ifdef GATE_LEVEL
`include "../FULL_ADDER/03_outputs/fa_netlist.sv"
`include "/home/yellow/ee3201_15/LAB2/gpdk045_verilog/fast_vdd1v2_basicCells_lvt.v"
`else
`include "../FULL_ADDER/00_src/full_adder.sv"
`endif

module tb_fa;
logic A, B, Cin;
logic Sum, Cout;
full_adder DUT (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));

`ifdef ANNOTATION
initial begin
$sdf_annotate("../FULL_ADDER/03_outputs/delay.sdf",DUT,,"annotate_full_adder.log","MAXIMUM");
end
`endif

initial begin
$dumpfile("fa_wave.vcd");
$dumpvars(0, tb_fa);
#0;   A=0; B=0; Cin=0;
#0.1; A=0; B=0; Cin=1;
#0.1; A=0; B=1; Cin=0;
#0.1; A=0; B=1; Cin=1;
#0.1; A=1; B=0; Cin=0;
#0.1; A=1; B=0; Cin=1;
#0.1; A=1; B=1; Cin=0;
#0.1; A=1; B=1; Cin=1;
#0.1;
$finish;
end
endmodule
```